

HDF5 Audit Report

Gerd Heber¹ and Elena Pourmal²

HDF5 is a foundational component of the scientific software ecosystem, yet its safety, security, and privacy properties warrant analysis across component boundaries. This report presents a targeted, ecosystem-level technical audit of the core library and file format, command-line tools, extension mechanisms (VOL connectors, filter plugins, and virtual file drivers), selected language bindings and applications, the upstream and downstream software supply chain, representative independent format implementations, and long-term data-accessibility concerns.

Two cross-cutting themes recur; this thematic organization is not a risk ranking. First, the selected HDF5 CVE history contains recurring failures to validate file-derived sizes, counts, offsets, and related state before low-level C operations. Patching or containment is necessary treatment for affected scope but is insufficient to eliminate those classes. We therefore propose a prevention program built on an explicit invariant registry for each on-disk structure, a two-phase parser boundary separating decode from object construction, centralized checked arithmetic, structure-aware fuzzing, and file-open security profiles that make hostile input a first-class concern. Second, privacy exposure can arise during normal, authorized operation: the format's self-describing metadata, structural layout, workflow artifacts, and extension mechanisms can disclose sensitive information even when raw data are protected. Encryption may be required at some disclosure boundaries, but it is one partial control alongside minimization, recipient restriction, and lifecycle governance.

A related source-level observation found candidate deserializer invariants derived from in-file bytes but enforced only inside assertions, which release builds may disable. The report assigns this perimeter a 30-day catalog and debug/release

¹The HDF Group, gheber@hdfgroup.org

²Lifeboat, LLC, elena.pourmal@lifeboat.llc

differential-test target; each confirmed adverse path is routed by its consequence without waiting for the longer parser migration.

Throughout, we frame the recurring failure as trust-boundary confusion and give risk-informed mitigations together with a distinct implementation-planning framework for library maintainers, extension authors, downstream projects, and end users. The resulting priority order is bounded and scenario-specific: equivalent exposed services that follow attacker-selected external resources require Immediate containment; confirmed affected High and upper-bound-High scopes require Near-term treatment; Moderate findings receive Planned treatment; and unresolved safety, extension-privacy, and supply-chain observations require evidence rather than an assumed Low rating. Longer-horizon structural work prevents recurrence but does not defer those response clocks. Although the subject is HDF5, the analysis is intended to generalize to comparable open-source scientific data ecosystems.

Acknowledgments: This material is based upon work supported by the U.S. National Science Foundation under Federal Award No. 2534078. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation.

Revision History

Version Number	Date	Comments
v1	July 4, 2026	Initial draft released for internal review.
v2	August 31, 2026	Public release.

Executive Summary

What this report is

This is a targeted, ecosystem-level technical assessment of safety, security, and privacy risk in the HDF5 software and data ecosystem, with an evidence cutoff of August 27, 2026. It is not a certification, attestation, penetration test, or exhaustive source review, and it expresses no assurance opinion. The full engagement type, scope, and limitations are stated in Sections 1.1–1.10.

Ratings apply to specific adverse-event scenarios at specific boundaries, never to HDF5 as a technology. The rating method is defined once, in Section 1.7, and Appendix A is authoritative for every formal rating in this revision.

Four ecosystem-level judgments

1. **Historically affected core and official-tool paths repeatedly failed to validate file-derived values** — sizes, counts, offsets, message state, layout state, filter and datatype parameters — before using them in trusted C operations. Local fixes can treat a known variant; eliminating the class requires invariant-driven parsing and continuing regression evidence.
2. **Applications can turn documented HDF5 capabilities into attack mechanisms** when they process untrusted files while holding filesystem, plugin-loading, deserialization, or resource authority that the file’s originator does not have. The defect is in the authority boundary, not necessarily in the parser.
3. **Reader acceptance, format conformance, and safe deployment are three separate decisions.** Permissive historical acceptance is not proof that a file is consistent, and stricter rejection is not automatically a regression.
4. **Privacy harm can arise during correct, authorized operation**, because protecting the payload does not protect metadata, structure, derived information, caches, logs, or workflow artifacts.

Registered findings at a glance

Table 1 is a navigation aid. The controlling records — scenario statements, scope, evidence, and rating bases — are in Appendix A; the proposed owners come from Appendix C and are role labels, not present assignments. Counts here describe bounded scenarios only. They are not prevalence estimates and must not be aggregated into a score.

Table 1 – Registered findings, ratings, and response routes at a glance

ID	Scenario shorthand	Dim.	Tier	Status	Response route	Owner
APP-SEC-001	Affected dataset service followed attacker-selected external raw-storage paths with worker authority and returned local files.	Sec.	Critical	Rated	Immediate (24 h from identification)	APP
CORE-SEC-001	Malformed on-disk metadata reaches memory-unsafe core-library paths in historically affected releases.	Sec.	High	Provisional	Near-term (30 d)	CORE
APP-SEC-002	Affected Keras legacy-HDF5 model loading executes embedded code despite <code>safe_mode=True</code> .	Sec.	High	Rated	Near-term (30 d)	APP
APP-SEC-003	Affected Keras model loading follows attacker-controlled external dataset references and discloses local files.	Sec.	High	Rated	Near-term (30 d)	APP
APP-SEC-005	TensorFlow 2.17.0 loads HDF5 plugins from an uncontrolled search location, enabling local privilege escalation.	Sec.	High	Rated	Near-term (30 d)	APP
CORE-TOOL-001	A malicious file triggers the reported <code>h5dump</code> use-after-free in HDF5 1.14.1-2 and earlier.	Sec.	Mod.–High	Provisional	Near-term (30 d)	CORE
CORE-PRIV-001	Sensitive plaintext metadata is disclosed when a file is shared under payload-only protection.	Priv.	Mod.–High	Provisional	Near-term (30 d)	PRIV
CORE-SAFE-001	Storage exhaustion during write or close leaves an incomplete or corrupted file; no library mechanism detects or recovers from it.	Safety	Mod.–High	Provisional	Near-term (containment)	CORE
CORE-SAFE-002	Abrupt process or host termination during write leaves a file corrupted or not reopenable.	Safety	Mod.–High	Provisional	Near-term (containment)	CORE
LONG-SAF-001	An archived file becomes unreadable because a required filter, VFD, or VOL connector is gone. Decadal horizon.	Safety	Mod.–High	Provisional	Planned (recorded departure)	INTEROP

Table 1 – Registered findings, ratings, and response routes at a glance, continued

ID	Scenario shorthand	Dim.	Tier	Status	Response route	Owner
CORE-SEC-002	File-controlled work, allocation, traversal, or error paths terminate or exhaust an affected reader.	Sec.	Moderate	Provisional	Planned (90 d)	CORE
APP-SEC-004	Affected Keras weight loading allocates from an extreme file-declared shape and exhausts memory.	Sec.	Moderate	Rated	Planned (90 d)	APP
FMT-SAF-001	The normative specification is incomplete, ambiguous, or contradicted by the reference library, so conformance is not determinable.	Safety	Mod.–High	Provisional	Near-term (adjudicate disputes)	INTEROP
IND-SAF-001	HDF5 2.0/2.1 exact-width validation rejects a sufficient but nonminimal chunk encoding permitted by the specification.	Safety	Moderate	Rated	Planned (90 d)	INTEROP
IND-SAF-002	Older permissive readers accept contradictory PureHDF 2.1.2 chunk metadata that HDF5 2.0 rejects after upgrade.	Safety	Moderate	Rated	Planned (90 d)	INTEROP
CORE-SEC-003	Candidate deserializer invariants derived from in-file bytes are enforced only inside <code>assert(...)</code> and may be absent from <code>-DNDEBUG</code> builds. One instance is confirmed and promoted into CORE-SEC-002; extent unknown.	Sec.	<i>None</i>	Backlog	Evidence action (RM-1E, 30 d target)	CORE
EXT-PRIV-001	Extensions can move data and metadata across storage, cache, logging, network, and executable-plugin boundaries.	Priv.	<i>None</i>	Backlog	Evidence action (RM-2E)	PRIV
SUP-SAFE-001	Downstream builds and workflows can disagree about ABI, linkage, filters, plugins, VFDs, parallel features, or file locking.	Safety	<i>None</i>	Backlog	Evidence action (RM-2F)	REL

Fifteen formal findings are registered: seven Rated and eight Provisional. Their tiers comprise one Critical, four High, six Moderate–High ranges, and four Moderate. Three further observations remain in the evidence-and-scoping backlog below the rule in the table. **A backlog record is an evidence gap, not a determination of Low risk.**

One finding, FMT-SAF-001, is about the normative artifact rather than any implementation: where the specification and the reference library disagree and no procedure exists to rule between them, conformance cannot be established. Its

exposure rises with the number of independent implementations, because each unresolved ambiguity is resolved independently by every one of them.

Three further findings — CORE-SAFE-001, CORE-SAFE-002, and LONG-SAF-001 — are write-side and lifecycle safety risks that an earlier revision of this report either left unrated or did not register at all. They are called out here because they are the conditions HDF5 users encounter most often in ordinary operation, and because the rest of this report is organised around a different question: hostile input arriving from outside, rather than data failing to survive.

Where the priority actually falls

Response urgency comes from Table 34 and nowhere else. Chapter order, recommendation numbering, CVE counts, publicity, effort, and dependencies do not create or revise it.

- **The single Immediate route belongs to downstream service operators, not to the reference-library maintainers.** It is conditional: it applies to any service that still automatically follows file-selected external resources while holding access to sensitive local files. The affected July 2026 deployment is reported remediated, and this report assigns no current residual tier to it.
- **For The HDF Group specifically, there is no Immediate action in this revision.** The near-term core obligations are the deployment census (RM-1A), the affected-version treatment of CORE-SEC-001 and CORE-TOOL-001 (RM-1B), and the 30-day assert-site catalog (RM-1E).
- **Two response clocks are distinct.** A finding's clock starts when an affected deployment or equivalent exposure is *identified*. The census and assert-catalog clocks start when the roadmap is *adopted*. Adoption must set due dates for both inventories, so that a response clock cannot stay latent through unbounded non-discovery.

If only a few things can be funded

The roadmap in Appendix C is scoped to risk, not to budget. Section C.5 identifies a capacity-constrained subset that preserves the response clocks and delivers the highest control coverage per unit of effort. It is a funding aid and explicitly not a risk rank.

What this report does not establish

No proposed mitigation is shown to be adopted, implemented, or effective. Every roadmap package is *Proposed / implementation not verified* at publication, and no residual-risk acceptance is recorded. Current installed populations, exact fix coverage across supported releases, and deployment-specific control effectiveness remain incomplete. Unless a record expressly states otherwise, findings rest on document, artifact, and architecture examination rather than independent reproduction. The absence of a reported finding does not establish the absence of defects.

Contents

Executive Summary	4
1. Introduction	13
1.1. Purpose and Engagement Type	13
1.2. Objectives and Intended Audience	13
1.3. Scope	14
1.4. Assessment Dimensions	16
1.5. Temporal Boundary and Evidence Basis	17
1.6. Methodology	18
1.7. Report-wide Risk Rating and Mitigation Planning	19
1.8. Identifier Scheme and Relationship to SSP SIG Artifacts	25
1.9. Reconciliation with the SSP SIG Risk Registries	27
1.10. Limitations and Assurance	30
1.11. Privacy Context	30
2. Assessment Surfaces	32
3. HDF5 Core Library and File Format	34
3.1. HDF5 Safety Observations and Evidence Gaps	34
3.2. Malformed Input as a Security Boundary	38
3.3. HDF5 Privacy: Assets and Exposure	42
3.3.1. Introduction	42
3.3.2. HDF5 Assets and their Exposure	43
3.4. HDF5 Tools	47
3.4.1. Privacy exposure through HDF5 command-line tools	47
3.5. Audit Findings and Mitigation Planning	50
3.5.1. Security and Safety	51
3.5.2. Prior Art: What Already Exists	53
3.5.3. Safety: Durability and Recovery	70
3.5.4. Privacy	73

4. HDF5 Library Extensions	80
4.1. Overview	80
4.2. Virtual Object Layer (VOL) Connectors	80
4.2.1. Privacy-Relevant Mechanisms for VOL Connectors	80
4.3. Filter Plugins	84
4.3.1. Privacy-Relevant Mechanisms for Filter Plugins	84
4.4. Virtual File Drivers (VFD)	88
4.4.1. Privacy-Relevant Mechanisms for VFDs	88
4.5. Architectural Observations and Mitigation Planning	91
4.5.1. Privacy	92
5. HDF5 Wrappers and Language Bindings	99
5.1. Overview	99
5.2. Python	99
5.3. Java	100
5.4. Audit Findings and Mitigation Planning	101
6. Applications using HDF5	102
6.1. Boundary Case Study: External Raw Storage and the Hugging Face Incident	103
6.2. Audit Findings and Mitigation Planning	105
7. HDF5 in the Software Supply Chain	113
7.1. Upstream Dependencies	113
7.2. Downstream Dependencies	114
7.3. Audit Findings and Mitigation Planning	117
8. Independent HDF5 Implementations	120
8.1. Overview	120
8.2. Two Distinct 2026 Chunk-Layout Cases	122
8.2.1. Sufficient but Nonminimal Encoding	122
8.2.2. Contradictory PureHDF Metadata	122
8.3. When the Specification Itself Is the Defect	123
8.4. Audit Findings and Mitigation Planning	124

9. Long-term Data Accessibility and Interpretability	128
9.1. Why Extensions Are the Sharp Edge	128
9.2. Audit Findings and Mitigation Planning	129
10. Conclusion	132
10.1. Overall Assessment Conclusion	132
10.2. Reconciled Mitigation Priorities	133
10.3. Significance of the 2026 Developments	136
10.4. Assurance, Closure, and Residual Risk	137
References	139
A. Findings Register	146
A.1. Correspondence with the SSP SIG Registries	147
A.2. Evidence and Scoping Backlog	150
A.3. Response Routing	151
A.4. Evidence Actions Without Assigned Urgency	152
A.5. Core Library, Tool, and Format Findings	153
A.6. Application-Boundary Findings	161
A.7. Independent-Implementation and Interoperability Findings	163
A.8. Format and Specification Findings	165
A.9. Long-Term Accessibility Findings	167
A.10. Evidence-Backlog Record Details	169
B. Recommendation Register	171
B.1. Correspondence with SSP SIG Policy Controls	177
B.2. Recommendations Not Tied to a Formal Finding	180
C. Phased Mitigation Roadmap	181
C.1. Accountability Roles	181
C.2. Phase Gates and Clocks	183
C.3. Licensing Constraint on Reuse of Existing Work	185
C.4. Work-Package Register	186
C.5. Minimum Viable Subset Under Constrained Capacity	192

C.6. Acceptance and Closure Evidence	196
C.7. Finding and Evidence-Action Coverage	200
C.8. Change Control and Residual Risk	204
D. CVE History Summary	206
E. Software Supply Chain Details	217
F. Resources and Supporting Tooling	227

1. Introduction

1.1. Purpose and Engagement Type

This report presents a targeted, evidence-based technical assessment of safety, security, and privacy risks in the HDF5 software and data ecosystem. Its purpose is to identify systemic risk patterns, explain the trust boundaries at which those risks arise, and recommend mitigations for the organizations and individuals that develop, distribute, deploy, or use HDF5.

The term *audit* denotes a documented technical examination. This engagement is not a compliance certification, attestation engagement, penetration test, or exhaustive line-by-line source-code review. It expresses no independent assurance opinion and does not certify that any HDF5 release, application, file, or deployment is safe, secure, privacy-preserving, conforming, or free from defects. Authorship and sponsorship are disclosed in the front matter.

1.2. Objectives and Intended Audience

The objectives of the assessment are to:

1. identify important assets, interfaces, capabilities, trust boundaries, and failure modes across the HDF5 ecosystem;
2. distinguish accidental safety and data-integrity failures from adversarial security failures and privacy harms arising during authorized operation;
3. identify recurring root-cause classes rather than treating individual crashes, CVEs, disclosures, and interoperability failures as unrelated events;
4. examine how risk propagates between the file format, core library, tools, extensions, bindings, applications, supply chain, and independent implementations;
5. distinguish file-format conformance, acceptance by a particular implementation, and safe use in a particular deployment;

6. recommend mitigations attributable to the actors able to implement and enforce them; and
7. identify lessons applicable to comparable open-source scientific data ecosystems.

The primary audience is The HDF Group's maintainers, security responders, release engineers, and technical governance bodies. Additional audiences include extension and plugin authors, binding and independent-implementation maintainers, downstream application and distribution maintainers, operators of data-ingestion services, research-infrastructure teams, data stewards, and security and privacy reviewers.

Recommendations are assigned, where possible, to the layer able to enforce them. A control failure at an application or deployment boundary is not classified as a core-library defect merely because an HDF5 mechanism was involved.

1.3. Scope

Component scope is not binary in this report. The assessment distinguishes the following levels of coverage:

Primary assessment objects. These are the HDF5 on-disk format; the core C library, with emphasis on its handling of file-controlled metadata; HDF5 command-line tools; the capability, trust, and privacy characteristics of VOL connectors, filter plugins, and Virtual File Drivers as extension classes; and HDF5 build, release, and dependency configuration. The report states findings and mitigations for these objects.

Boundary case studies. Selected downstream incidents and interoperability disputes are examined to determine how HDF5 capabilities interact with application authority, deployment policy, or independent implementations. The cases include model and dataset ingestion, external-resource resolution, plugin loading, and conformance disputes involving files produced by independent writers. The referenced projects are not audited as products.

Contextual references. Other components are identified to describe ecosystem reach, dependency relationships, compatibility consequences, or potential exposure. A contextual reference does not constitute an assessment or assurance conclusion about that component.

Case studies and samples are selected purposively according to exposure to untrusted input or ambient authority, evidence of a recurring failure class, interoperability or archival consequence, and relevance to cross-cutting mitigations. They are not a statistical sample and do not support prevalence or frequency estimates.

In this report, *ecosystem-wide* describes the breadth of surfaces considered. It does not mean that every implementation, configuration, or downstream product was examined exhaustively.

Out of scope. The following are outside the engagement:

- HSDS source code, client behavior, REST endpoints, authentication and authorization, tenant isolation, deployment configuration, storage backends, and service availability;
- a complete safety, security, or privacy assessment of HDFView, Keras, TensorFlow, Hugging Face, PureHDF, or any other application, service, framework, or independent implementation cited as a case study or ecosystem dependency;
- general AI or machine-learning safety, model behavior, training, inference, and framework functionality unrelated to an HDF5 integration boundary;
- detailed assessment of MPI semantics, collective-I/O correctness, concurrency, fault tolerance, performance, or parallel-filesystem security in Parallel HDF5;
- exhaustive review or testing of every HDF5 release, platform, build configuration, compiler, filter, connector, binding, application, package, or independent implementation;
- penetration testing of production services, exploit development, or proof that every defect or vulnerability class has been identified;

- assessment of a particular organization's production infrastructure, credentials, identity systems, network controls, or operational procedures, except as context for a cited incident or mitigation; and
- legal, regulatory, contractual, research-ethics, or functional-safety certification.

References to HDFView identify downstream exposure or reader behavior, not an HDFView finding. References to machine-learning frameworks or services concern only their handling of HDF5-controlled inputs. References to Parallel HDF5 are limited to the configuration, dependency, extension, or trust-boundary characteristics expressly discussed.

HSDS may appear in ecosystem diagrams for context, but no conclusion about HSDS should be inferred.

1.4. Assessment Dimensions

Safety. Safety means operational robustness and protection of data integrity, availability, recoverability, interoperability, and long-term accessibility in the absence of a malicious actor. Relevant conditions include malformed, truncated, inconsistent, or partially written files; resource exhaustion; I/O failure; incompatible implementations; and foreseeable misuse. Safety in this report does not mean regulated functional or physical safety.

Security. Security concerns the possibility that an adversary can compromise confidentiality, integrity, or availability, or cause a process to exercise authority contrary to the operator's intent. The principal threat scenario is an HDF5 file, metadata field, external reference, plugin requirement, archive, or model artifact controlled by an untrusted party and processed by an application possessing filesystem, network, compute, plugin-loading, or credential authority.

The assessment includes memory-safety defects, denial of service, resource exhaustion, external-resource resolution, dynamic code loading, downstream deserialization, and supply-chain compromise. A security consequence does not require

a defect in the HDF5 format or parser: an intentional HDF5 capability can become an attack mechanism when a downstream application interprets file-controlled instructions using authority unavailable to the file's originator.

Privacy. Privacy concerns adverse consequences arising from the disclosure, retention, inference, aggregation, correlation, or secondary use of information. Such consequences may arise during authorized and otherwise correct operation, without a security breach. The assessment includes scientific data, descriptive and structural metadata, object names and attributes, provenance, workflow artifacts, caches, temporary copies, external resources, and information that becomes sensitive when datasets are combined.

The report evaluates privacy-exposure mechanisms and possible technical or governance controls. It does not determine compliance with privacy statutes, regulations, contracts, consent requirements, institutional policies, or research-ethics obligations.

1.5. Temporal Boundary and Evidence Basis

The evidence cutoff for this revision is August 27, 2026. Public evidence available on or before that date is eligible for consideration. Later developments are outside this revision unless expressly incorporated as subsequent-event updates.

The historical analysis has no fixed lower date and includes earlier releases where necessary to examine cited vulnerabilities, compatibility behavior, and implementation changes. Because this is a historical, ecosystem-level assessment, no single software baseline applies. A conclusion concerning one release, commit, build configuration, platform, or third-party version is not presumed to apply to others. Version-specific conclusions are bounded by the artifact identifiers cited with them. Where an exact artifact is not identified, a statement should be read as an architectural or historical class-level observation, not as a release-specific conclusion.

The principal evidence consists of published HDF5 specifications and API documentation; selected source code and build configurations; security policies,

advisories, and CVE records; issue and pull-request discussions; cited commits and regression artifacts; and authoritative documentation or incident reports from representative downstream and independent projects.

1.6. Methodology

The engagement uses a qualitative, risk-informed technical-assessment method:

1. **Surface and trust-boundary mapping.** The ecosystem is decomposed into format, parser, tool, extension, binding, application, supply-chain, independent-implementation, and data-lifecycle surfaces.
2. **Asset and harm analysis.** The assessment identifies relevant data, meta-data, code, credentials, compute and storage resources, availability, interoperability, and interpretability, together with plausible safety, security, and privacy consequences.
3. **Document and artifact examination.** Specifications, documentation, public source artifacts, vulnerability records, fixes, issue records, build configuration, and representative ecosystem documentation are examined.
4. **Root-cause and pattern analysis.** Defects and incidents are grouped by the earliest violated invariant or trust-boundary failure supported by the evidence, rather than only by crash type, CVE identifier, or affected application.
5. **Conformance and interoperability analysis.** Acceptance by one HDF5 reader is not treated as proof that a file conforms to the applicable specification, and rejection by another reader is not treated as proof that it does not. Where implementations disagree, the analysis distinguishes the normative requirement, writer behavior, reader validation, and compatibility policy.
6. **Risk and mitigation analysis.** Adverse-event scenarios are evaluated under the report-wide inherent- risk method in Section 1.7. Recommendations are then analyzed separately for control coverage, enforcing actor, response urgency, implementation dependencies, effort, sequence, and acceptance ev-

idence. Cross-cutting controls and prerequisite protections are distinguished from local remediations.

For incidents involving multiple components, the analysis distinguishes the HDF5 mechanism, the authority supplied by the host application, the failed control or policy boundary, and the resulting impact. The presence of HDF5 in an exploit chain is not, without additional evidence, classified as a parser or file-format defect.

The report distinguishes, where practicable, among:

Documented observation: directly supported by a cited specification, source artifact, test, advisory, incident record, or maintainer diagnosis; the applicable discussion distinguishes source-reported evidence from independent reproduction;

Reasoned inference: a conclusion drawn from observations, with material assumptions and uncertainty identified; and

Recommendation: a proposed control to be validated through implementation and acceptance evidence.

Unless a finding expressly records an independent reproduction or test, it should be understood as the result of document, artifact, or architecture examination rather than independent dynamic verification.

1.7. Report-wide Risk Rating and Mitigation Planning

This subsection defines the only risk-rating method used in this report. A formal rating applies to a specific adverse-event scenario, not to a technology, chapter, CVE count, root-cause family, recommendation, or implementation task. Each rated finding states, directly or by reference, the assessment object and affected version or surface; the initiating actor or non-adversarial event; the condition, exposure, and material preconditions; the assets or people affected; the credible consequence; and, when time materially affects the rating, the assessment horizon.

Materially different versions, deployments, preconditions, or consequences are rated separately.

A risk scenario can be summarized as follows:

Given [actor or non-adversarial event], [assessment object and scope], and [exposure and preconditions], [condition or event] may cause [harm] to [assets, people, operations, or other organizations] [during the material assessment horizon, if any].

Inherent risk. Inherent risk is the risk presented by the stated scenario in its defined assessed population and deployment. It includes only controls that are mandatory and intrinsic to the identified implementation, version, and scoped default configuration; it gives no credit for optional, compensating, organization-specific, or proposed controls. A feature, configuration, privilege, or user action required to reach the condition is a scenario precondition and informs exposure; it is not an implementation-effort factor. A disabled-by-default or opt-in state constrains exposure only when that state is part of the stated scope. Deployments in which the feature is enabled are rated separately when the difference is material.

Where the effectiveness and coverage of existing controls are supported by evidence, a finding may also state *current residual risk*. A *target residual risk* may be projected for a recommendation, but it is not treated as achieved until the control passes its acceptance criteria. Neither current nor target residual risk replaces the inherent-risk rating. When a residual rating is stated, it uses the same criteria and identifies the controls credited in the assessment.

Inherent risk combines the impact and exposure levels in Table 2 using Table 3. Exposure is the rubric's reachability and recurrence axis: it expresses how readily and commonly the scenario arises under the stated conditions, not a numerical probability. The axes and tiers are ordinal; their labels are not numeric scores and are not multiplied.

Table 2 – Report-wide impact and exposure criteria

Level	Name	Decision rule
I1	Limited impact	Transient or localized effect with routine recovery; no material incorrect result, durable data loss, sensitive disclosure, or harm to individuals.
I2	Material impact	Meaningful workflow interruption; recoverable data loss or corruption; a persistent but bounded compatibility or scientific-validity problem; or limited disclosure, inference, or privacy harm affecting a defined population.
I3	Serious impact	Major data loss, corruption, or outage; process compromise or unauthorized resource access; consequential scientific invalidity; or substantial sensitive- information disclosure or privacy harm.
I4	Severe or systemic impact	Widespread, cross-tenant, or privileged compromise; irreversible loss of unique or critical scientific records; large-scale disclosure of highly sensitive information; or ecosystem- or mission-level disruption.
E1	Exceptional exposure	The scenario requires an unusual configuration and several restrictive preconditions, affects a narrow surface, and is exceptional within the stated assessed population.
E2	Constrained exposure	The scenario is realistic but limited to an opt-in feature, restricted actor capability, uncommon workflow, or multiple material preconditions.
E3	Broad exposure	The scenario arises in a common workflow or deployment, has few material preconditions, commonly crosses a trust boundary, or recurs within the stated assessed population.
E4	Routine exposure	The scenario arises during default or normal operation, is broadly reachable and repeatable, or is straightforward to trigger or automate. Malicious intent is not required for safety or privacy harm.

Table 3 – Report-wide inherent-risk matrix

Impact	E1	E2	E3	E4
I4 Severe/systemic	High	High	Critical	Critical
I3 Serious	Moderate	High	High	Critical
I2 Material	Low	Moderate	Moderate	High
I1 Limited	Low	Low	Moderate	Moderate

The following decision rules apply:

1. Use the greatest credible impact supported by evidence, not an unconstrained theoretical worst case.
2. Judge exposure from the stated reachability, preconditions, deployment prevalence, and recurrence. A confirmed incident establishes feasibility but does not by itself establish ecosystem-wide exposure. Absence of adequate prevalence or recurrence evidence changes confidence or rating status; it does not establish E1.
3. Rate the scenario at the boundary where authority and control reside. Participation in an exploit chain does not assign the full impact to every component in that chain.
4. Split findings when affected versions, deployment profiles, or consequences would produce materially different ratings.
5. Do not add or average safety, security, and privacy impacts. Record each applicable dimension and identify the substantiated impact that controls the overall rating.
6. Do not translate CVSS scores, CVE counts, publicity, privacy-exposure category codes, or recommendation numbers into this rating.
7. State a time horizon when it materially affects impact or exposure. Do not directly order ratings that use materially different horizons.

Evidence confidence and rating status. Confidence records uncertainty in the evidence and is separate from both exposure and risk:

High confidence. Direct, scope-specific primary evidence supports the condition and material rating assumptions through reproduction or mutually consistent specification, source, fix, and test artifacts.

Moderate confidence. Credible primary evidence supports the scenario, but an important element such as reproduction, prevalence, version boundaries, or causal attribution remains incomplete.

Low confidence. The scenario or rating depends materially on incomplete, disputed, secondary, or uncorroborated evidence.

Confidence is not multiplied into, added to, or subtracted from inherent risk. If uncertainty could change the tier, the report gives the plausible range and the controlling assumptions rather than silently lowering the rating.

A rating record also states one of the following statuses:

Rated. Both axes and the resulting tier are supported by the cited evidence.

Provisional. Both axes can be estimated, but material uncertainty remains and is expressly stated.

NE—not evaluated. At least one axis lacks sufficient evidence. An NE finding is not implicitly Low risk.

N/A—not applicable. The item is not a risk-bearing finding, such as a recommendation, implementation task, or program theme.

Narrative findings that do not state impact, exposure, tier, status, and confidence carry no formal report-wide rating.

Risk and implementation planning are separate. The risk record and the recommendation record are separate. The following planning fields never change the inherent-risk rating:

Table 4 – Treatment-planning fields kept separate from inherent risk

Field	Recording rule
Response urgency	Records <i>Immediate</i> , <i>Near-term</i> , <i>Planned</i> , or <i>Monitor</i> , with the rationale and target horizon. It is determined from current residual risk, time to harm, active exploitation or recurrence, and stated risk tolerance—not from dependencies or effort.
Implementation dependencies	Lists concrete prerequisite recommendation or decision identifiers. Use <i>None known</i> when none are identified and <i>TBD</i> when dependency analysis has not been performed; do not encode dependencies as severity.
Effort	Records <i>Not estimated</i> , <i>Small</i> (localized change), <i>Medium</i> (multi-module or coordinated change), or <i>Large</i> (architectural, cross-project, compatibility, or migration work). Estimate work independently of elapsed duration or delivery horizon. Effort is not subtracted from risk.
Implementation sequence	Records a delivery wave or order together with its concrete prerequisite or enabling rationale. Recommendation identifiers alone do not establish sequence, and delivery order is not a risk rating.
Owner and target horizon	Names the actor with authority to implement or accept the recommendation and the intended completion date or release.
Acceptance evidence	States the observable test, artifact, or operational evidence required to mark the recommendation complete and reassess residual risk.
Residual-risk decision	Records the verified remaining rating and the accountable decision to accept, further reduce, avoid, or transfer it.

Unless a stricter stated risk tolerance applies, the default response-urgency rule maps Critical current residual risk to *Immediate*, High to *Near-term*, Moderate to *Planned*, and Low to *Monitor*. A finding whose rating is expressed as a tier range is routed at the upper bound of that range; the report refers to such a case as *upper-bound-High* when the range is Moderate–High. Routing at the upper bound does not collapse the range, which remains the stated rating. Active exploitation or recurrence, imminent harm, or a credible irreversible consequence can raise urgency. When current residual risk has not been assessed, inherent risk is the conservative provisional baseline. Dependencies and effort may shape the delivery plan or require an interim containment action, but they never lower

response urgency. Any departure from this mapping requires a recorded rationale and accountable decision.

A Critical scenario remains Critical even when its mitigation is high effort. An enabling action may appear earlier because it is a prerequisite; the action does not thereby acquire a risk rating. Immediate containment and a later architectural treatment may therefore address the same finding on different delivery horizons.

Every formal finding record should contain a stable identifier, scenario, dimension, affected scope, evidence, I/E rationale, an inherent-risk tier when evaluated or explicit no-tier/NE notation, rating status, confidence, and linked recommendation identifiers. Every recommendation record should separately identify the findings addressed, enforcing actor or owner, urgency, dependencies, effort, sequence and target horizon, acceptance evidence, status, and residual-risk decision. Numbered recommendation identifiers in later sections identify items only; they do not establish risk, urgency, effort, dependency, or delivery order.

The populated report-wide register is Appendix A. That appendix is authoritative for formal ratings in this revision: chapter narrative, recommendation numbering, CVE metadata, and descriptive uses of words such as “high” or “important” do not create an additional rating unless they map to a registered finding. Appendix C separately groups the recommendations into proposed work packages with accountable roles, dependencies, effort, phase gates, acceptance evidence, and residual-risk decision requirements. Those planning fields do not revise the register’s inherent ratings or represent stakeholder-approved commitments until adopted and assigned.

1.8. Identifier Scheme and Relationship to SSP SIG Artifacts

This report and the HDF5 Safety, Security & Privacy (SSP) Special Interest Group repositories are separate artifacts of the same programme, and both use the word *SHINES*. They do not share an identifier namespace. A reader who assumes otherwise will misread both.

This report's finding identifiers. CORE-SEC-nnn, CORE-TOOL-nnn, CORE-PRIV-nnn, CORE-SAFE-nnn, APP-SEC-nnn, IND-SAF-nnn, EXT-PRIV-nnn, and SUP-SAFE-nnn. These name the adverse-event scenarios registered in Appendix A and exist only in this report.

This report's recommendation identifiers. CORE-Sn, CORE-PRIV-Sn, APP-Sn, VOL-Sn, FILTER-Sn, VFD-Sn, IND-Sn, and SUP-Sn, registered in Appendix B. Work-package identifiers RM-nX in Appendix C are likewise report-local.

SSP SIG policy controls. HDF5-SHINES-FAMILY-nn, in families including SDLC, BUILD, REL, DEP, TEST, TM, VULN, CRYPTO, COMP, HARD, PLUG, IR, and GOV. These are *policy controls* maintained in the SSP SIG governance repository [48]. They are not recommendations of this report, and no identifier in this report is one of them.

SSP SIG registry entries. ATK-nnn (security threats), HAZ-nnn (safety hazards), and EXP-nnn (privacy exposures) [51, 50, 49]. These are threat- and hazard-model entries, rated under a different method (Section 1.9). They are not findings of this report.

Citing an identifier outside this report. Prefix the report-local short form with this revision's identifier when quoting it elsewhere: the canonical global form of CORE-S1 is SHINES-2026-001: CORE-S1, and of CORE-SEC-001 is SHINES-2026-001: CORE-SEC-001. The short forms are used throughout this document for readability. They must not be abbreviated to "SHINES control" in any context that also refers to HDF5-SHINES-* policy controls.

A specific hazard for readers. The SSP SIG maintains a coverage matrix from the OpenSSF Open Source Project Security Baseline to "SHINES controls" [52]. That matrix scores the HDF5-SHINES-* *policy controls*. It does not score the recommendations in this report, and its COVERED, PARTIAL, and GAP counts say nothing about the coverage of any finding registered here.

Crosswalk. Appendix B records, for each recommendation, the SSP SIG policy control it supports where one exists, and Appendix A records the corresponding ATK, HAZ, or EXP registry entry for each finding. Those mappings are editorial correspondences established by this report; neither repository has adopted them, and a correspondence is not a claim that the control is satisfied or that the ratings agree.

1.9. Reconciliation with the SSP SIG Risk Registries

The SSP SIG registries rate their entries with a method that is **incompatible with Section 1.7**, and the two must not be compared numerically or merged into a single ordering.

SSP SIG method. Severity (1–5) *multiplied by* Likelihood (1–5) yields a score of 1–25, banded as 1–5 Low, 6–10 Moderate, 11–15 High, 16–20 Very High, 21–25 Critical.

This report's method. Ordinal impact (I1–I4) and exposure (E1–E4) resolved through Table 3 to Low, Moderate, High, or Critical. The axes are not numeric scores, are not multiplied, and decision rule 6 of Section 1.7 expressly forbids translating an external score into this rating.

Three consequences follow. First, the band vocabularies differ: *Very High* has no counterpart in Table 3, and the two *Critical* bands are not the same threshold. Second, an SSP SIG score cannot be converted into a tier here, and a tier here cannot be converted into a score there; a crosswalk may state that two records describe the same mechanism, never that they agree on magnitude. Third, and most important, the registries and this report answer different questions. An ATK or HAZ entry rates a *mechanism* across the ecosystem as a threat- or hazard-model artifact. A finding here rates a *bounded adverse-event scenario* in a stated population, and withholds a rating when exposure is not evidenced for that population.

That difference, not disagreement about the underlying facts, explains most of the divergence in Table 5. The registries are deliberately inclusive; this report is

deliberately conservative about exposure. Neither posture is an error, but a reader comparing the two documents must know which question each is answering.

Table 5 – Known rating divergences between the SSP SIG registries and this report

SSP SIG entry	Their rating	This report	Why they differ
ATK-001 metadata size and address fields corrupt parser memory	25, Critical (likelihood “almost certain”)	CORE-SEC-001, High, Provisional	The registry rates the mechanism’s recurrence across the CVE history. This report rates a bounded population — deployments on an affected version-and-path pair processing untrusted files — and holds exposure at E2–E3 because installed-population prevalence is not established. Closing the version-and-path gap in Appendix D is what would let the exposure axis move.
ATK-009 malformed files exhaust memory or CPU	16, Very High	CORE-SEC-002, Moderate	Impact, not exposure. This report assigns I2 because the evidenced consequence is a recoverable process or workflow interruption; a mission-level outage requires a separate deployment-specific record. The registry’s severity reflects service-denial in a hostile deployment.
ATK-011 malicious plugin resolution executes untrusted extension code	20, Very High	no core finding; APP-SEC-005 High at the application boundary	A genuine methodological disagreement, not an evidence gap. Decision rule 3 of Section 1.7 rates a scenario at the boundary where authority resides, and the evidenced instance is a downstream search-path policy failure. The registry rates the library capability itself. Both agree the capability requires a distinct control domain (CORE-S10).
ATK-013 trojanized packages or binaries	15, High	no finding; SUP-SAFE-001 unrated	Scope. This report registers a build-and-feature <i>mismatch</i> observation and never developed a distribution-compromise scenario. The registry entry is the better starting point, and RM-2F should adopt it rather than re-derive one.

Table 5 – Known rating divergences between the SSP SIG registries and this report, continued

SSP SIG entry	Their rating	This report	Why they differ
HAZ-005 out-of-space during write or close	15, High, with a full accident chain and three safety constraints	CORE-SAFE-001, NE, no tier	The most consequential divergence. This report declined to rate for want of a bounded API sequence, version, and recoverability evidence. The registry supplies the hazard analysis but not those measurements, so both records are waiting on the same RM-2D study; the entry's safety constraints SC-1 to SC-3 should be adopted as that study's hypotheses.
EXP-001 no standard whole-file encryption by default	20, Very High	no finding; CORE-PRIV-S4, conditional RM-2G	A framing difference. This report does not rate the absence of a feature as a scenario, and treats encryption as one partial control whose credit depends on tested recovery. The registry rates the resulting exposure directly.
EXP-002 metadata leakage despite encrypted payload	12, High	CORE-PRIV-001, Moderate-High	Consistent. The two records describe the same mechanism and land in comparable bands.
EXP-004, EXP-005, EXP-006, EXP-007 extension and storage- boundary exposures	12, 12, 10, 9	EXT-PRIV-001, unrated backlog	This report requires a named deployment, sensitive-data population, recipient, and harm before rating, and has none. The registry rates the mechanisms. RM-2E should take these four entries as its enumeration rather than starting from a blank scenario.

Where this report is silent and a registry entry exists, the registry entry stands as the SSP SIG's own assessment. Silence here is not disagreement, and Section 1.10 applies.

1.10. Limitations and Assurance

This assessment is time-bounded and based predominantly on public evidence. It is not an exhaustive review of the HDF5 source tree, every file-format feature, every configuration, or every ecosystem component. Representative sampling supports conclusions about observed mechanisms and recurring classes; it does not establish their frequency throughout the ecosystem.

Public CVE and issue records may be incomplete, duplicated, disputed, or subsequently corrected. Deployment-specific exposure and impact depend on application privileges, isolation, resource limits, enabled features, data sensitivity, and provenance. Proposed mitigations are evaluated analytically unless an implementation and verification procedure is expressly cited.

This report provides no formal assurance or certification opinion. Its conclusions are bounded by the assessment objects, methods, evidence, and cutoff described above. The absence of a reported finding does not establish the absence of defects, vulnerabilities, safety failures, or privacy risk.

Inherent-risk ratings express the authors' evidence-based judgment under Section 1.7; an unrated finding is not assumed to be Low risk. Numbered recommendation identifiers do not express risk, urgency, effort, dependency, or sequence. Recommendations become auditable commitments only when paired with an owner, affected scope, target horizon, implementation status, acceptance criteria, and residual-risk decision.

1.11. Privacy Context

The National Institute of Standards and Technology (NIST) defines privacy as “a condition that safeguards human autonomy and dignity through various means, including confidentiality, predictability, manageability, and disassociability.” Historically, privacy was interpreted as preventing unauthorized access to or disclosure of Personally Identifiable Information (PII). However, NIST emphasizes that contemporary privacy risks extend far beyond traditional notions of confidentiality. Modern privacy concerns increasingly arise from inference, aggregation,

behavioral profiling, secondary use of data, context changes, and surveillance effects [73]. NIST emphasizes that privacy risks are distinct from cybersecurity risks and may emerge even when systems are secure, lawful, and functioning exactly as intended [23]. HDF5 inherits many of the same classes of privacy risks identified by NIST, although these risks manifest differently in scientific and engineering environments.

HDF5 was designed to maximize data shareability, extensibility, portability, and long-term usability, and doesn't enforce data privacy or provide data protection mechanisms. Its rich metadata model, self-describing structure, plugin architecture, and interoperability layers can expose information extending beyond the raw data stored within the file. Sensitive information in HDF5 ecosystems may emerge through HDF5 internal and user-defined metadata, for example, HDF5 attributes, object names, file organization, and object properties. It may be exposed during the HDF5 data lifecycle through provenance records, workflow artifacts, and correlations across multiple datasets and files.

A common misconception in scientific computing is that privacy concerns are resolved once data access is restricted or datasets are anonymized. However, privacy risks often persist even when traditional security mechanisms are correctly implemented. Restricting access does not prevent inference from metadata. Anonymization may remove direct identifiers while leaving contextual information sufficient for re-identification. Similarly, data reduction or aggregation does not eliminate the possibility that sensitive information may be reconstructed through correlations across files, workflows, or external repositories. In modern scientific ecosystems, privacy exposure frequently results from accumulation of data and metadata over time rather than disclosure of a single sensitive piece of raw data or metadata.

The privacy risks are amplified by current scientific practice of data openness and shareability. HDF5 datasets increasingly move across organizations, systems, cloud platforms, workflows, and public and private repositories. Data are routinely repurposed beyond their original intent and analyzed by users who were not involved in their creation. Machine learning systems further amplify these concerns by discovering hidden relationships across data stored in HDF5 and enabling new

forms of inference that may not have been anticipated when data were originally collected. As a result, data stored in the HDF5 files may experience evolving privacy characteristics through its lifecycle.

Therefore, privacy exposure in HDF5 is not limited to unauthorized access, data breaches, unsafe use, or malicious behavior; it is distinct from, but can overlap with, data security and safety. By design, the HDF5 file format and library do not guarantee data privacy. In addition, privacy risks may emerge during legitimate scientific workflows, through data sharing, reuse, and analysis. In this document, we audit privacy assets and different privacy exposure mechanisms (or surfaces) in HDF5 with the goal of raising awareness about privacy risks when using HDF5.

2. Assessment Surfaces

Figure 1 maps the principal components and interfaces considered when identifying HDF5 risk. Inclusion in the figure provides ecosystem context and does not override the scope levels and limitations defined in Section 1.3. In particular, HSDS is shown for context but remains outside the engagement.

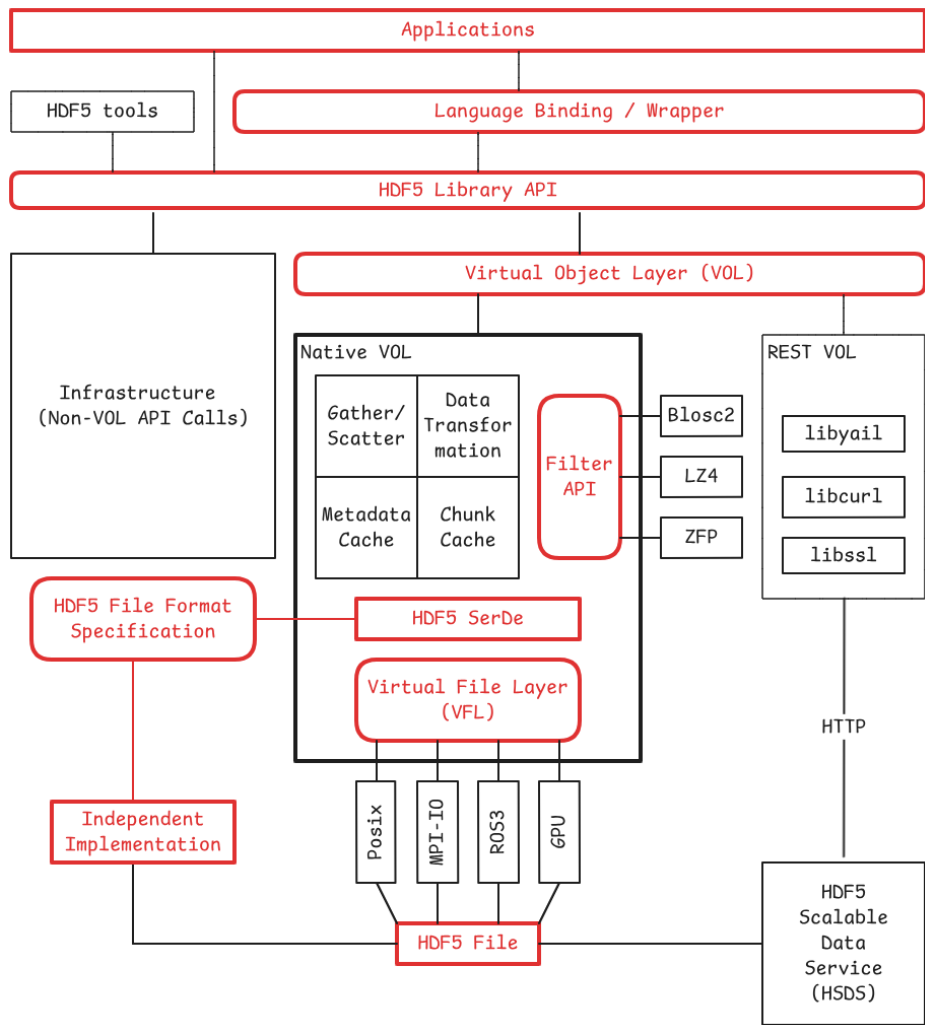


Figure 1 – HDF5 assessment surfaces (interfaces: rounded rectangles)

3. HDF5 Core Library and File Format

3.1. HDF5 Safety Observations and Evidence Gaps

Safety in this chapter means robustness and data integrity in the absence of a malicious actor, as defined in Section 1.4. Three conditions were examined, and none reached the evidence threshold for a rated finding in this revision.

Four conditions are registered as formal findings in this revision. The first three are write-side and lifecycle risks and were the least developed part of an earlier revision of this report; the discussion below states each as a distinct trigger, because conflating them — as an earlier draft did by merging storage exhaustion and abrupt termination into one observation — obscures that they need different containment and different structural treatment.

Storage exhaustion during write or close.

Storage fills during a write, flush, or close; the write fails with `ENOSPC`; and the library offers no notification, no recovery procedure, and no way to mark the file suspect. The controlling evidence is a maintainer statement that there is no HDF5 story for the condition and no callback for it, together with four candidate treatments and a documented cache workaround used in its place [20]. Registered as `CORE-SAFE-001`, **Moderate-High**. An earlier revision recorded this as NE for want of a bounded recovery path; the recovery path is in fact bounded — it does not exist — and that fact constrains exposure rather than leaving it open.

Corruption from abrupt termination.

A writer is killed, crashes, or loses power with metadata updates in flight, and in-memory and durable state diverge with no journal or write-ahead log to reconcile them. Community reports recur across sixteen years, and The HDF Group’s own 2024 discussion states that a crash “can lead to data loss or corruption of the file” and describes restarting crash-proofing work [18, 16, 17, 11, 12, 8, 15, 9]. Registered as `CORE-SAFE-002`, **Moderate-High**. This condition had no record of any kind in an earlier revision, which was the largest single omission in this chapter.

Extension-dependent loss of interpretability.

An archived file requires a filter, VFD, or VOL connector that is later unavailable. Registered as LONG-SAF-001, **Moderate-High** at a stated ten-year-or-more horizon, and developed in Section 9. Because the horizon is material, decision rule 7 forbids ordering this rating directly against ratings that state no horizon.

Reader/writer disagreement.

The two 2026 chunk-layout cases in Section 8, registered as IND-SAF-001 and IND-SAF-002. These are the short-horizon, fully evidenced instance of the same family as LONG-SAF-001: a file that a reader can no longer interpret.

The shape of the gap this closes. It is worth naming why safety was the thinnest dimension. The rest of this report is organised around *read-side hostile input* — the rubric’s principal threat scenario, the CVE analysis, the invariant registry, the parser boundary, the fuzzing programme, and the security profiles all answer the question “a malicious file has arrived.” The three findings above answer a different question, “my file did not survive,” and no structural recommendation in the security track addresses it. That asymmetry, rather than any judgment that the write path is sound, is why these conditions were previously unregistered or unrated.

Coverage limitation, stated against an existing hazard analysis. Safety is by a wide margin the least developed dimension in this revision. The extent of the gap can be stated precisely rather than vaguely, because the SSP SIG has already published a sixteen-entry safety hazard registry built on a control-system hazard model with unsafe control actions, derived safety constraints, and per-entry ratings [50, 45].

Table 6 maps that registry to this report. **Of sixteen hazards, this revision registers a corresponding formal finding for four, an unrated evidence record for two, and nothing at all for ten.** An earlier revision covered one. That is the honest measure of this chapter’s safety coverage. The registry entries are not reproduced or re-rated here — their method is incompatible with Section 1.7, as Section 1.9 explains — and where this report is silent, the registry entry

stands as the SSP SIG's own assessment. Silence here is not disagreement and is emphatically not a Low-risk conclusion.

Table 6 – SSP SIG safety hazard registry mapped to this report's coverage

ID	Hazard	Their rating	Coverage in this report
HAZ-001	Crash during metadata update	20, Very High	Covered. CORE-SAFE-002, Moderate-High, Near-term containment with structural treatment under CORE-SAFE-S2. Registered in this revision; absent from the previous one.
HAZ-002	Concurrent access without safe coordination	15, High	Not assessed. No concurrency scenario was developed.
HAZ-003	Extension boundary violates core invariants	15, High	Partially covered as an unrated record. The privacy aspect is EXT-PRIV-001; the <i>safety</i> aspect — an extension breaking an invariant the core relies on — has no counterpart here.
HAZ-004	Metadata and artifacts leak sensitive information	12, High	Covered. CORE-PRIV-001, Moderate-High, Near-term. This is the one hazard with a corresponding rated finding.
HAZ-005	Out-of-space during write or close	15, High	Covered. CORE-SAFE-001, Moderate-High, Near-term containment with structural treatment under CORE-SAFE-S1. The registry's accident chain and safety constraints SC-1 to SC-3 are adopted as the RM-2D hypotheses.
HAZ-006	False sense of durability from flush	12, High	Not separately assessed, but within the evidence base for CORE-SAFE-002: the community record includes reports that flushing itself can leave a file corrupted. A distinct durability-signaling finding is still warranted.
HAZ-007	SWMR deployment misfit	12, High	Not assessed.
HAZ-008	File-locking pitfalls or disabling via environment variable	12, High	Not assessed. File locking appears only as one item in the SUP-SAFE-001 mismatch list, which is itself unrated.
HAZ-009	Thread-safe build uses a global lock	12, High	Not assessed.

Table 6 – SSP SIG safety hazard registry mapped to this report’s coverage, continued

ID	Hazard	Their rating	Coverage in this report
HAZ-010	Chunk size too small	12, High	Not assessed as a safety hazard. Pathological chunk shapes appear here only as an adversarial resource-consumption mechanism under CORE-SEC-002; the non-adversarial operational hazard is absent.
HAZ-011	Deleting data does not shrink files by default	12, High	Not assessed. Note the privacy adjacency: the registry separately rates data remanence after deletion as EXP-003.
HAZ-012	Recovery tooling limits	12, High	Not separately assessed. Recovery capability is a controlling assumption of the CORE-SAFE-002 impact range — whether the file reopens after <code>h5clear</code> is what separates I2 from I3 — so the limits of that tooling bear directly on the rating. The evidence-gated repair planner in H5Lens is relevant prior art (Section 3.5.2).
HAZ-013	High-level API thread-safety gaps	9, Moderate	Not assessed.
HAZ-014	Portability hazard from missing filter plugins	9, Moderate	Covered, and extended. LONG-SAF-001 rates the archival scenario at a stated decadal horizon and extends it to VFD and VOL dependency, which this registry does not cover.
HAZ-015	Misuse of metadata-cache flush controls	8, Moderate	Not assessed.
HAZ-016	Chunk cache mis-sizing	6, Moderate	Not assessed.

Consequence for the reader and for the roadmap. Three things follow. First, a reader must not treat this report’s small number of safety findings as evidence that HDF5’s safety surface is small; it is evidence that this engagement examined it least. Second, the natural next revision of this report should adopt the hazard registry as its safety enumeration and work through it scenario by scenario, rather than re-deriving a list. Third, RM-2D is currently scoped to storage exhaustion alone; the registry shows at least HAZ-001, HAZ-006, HAZ-012, and HAZ-015 share its evidence apparatus — fault injection at write, flush, and close bound-

aries, followed by durable-state and recovery inspection — and it has accordingly been widened to cover the whole cluster, including the abrupt-termination trigger behind `CORE-SAFE-002`. Ten hazards remain with no record here; that number, not the four now covered, is the honest measure of what a safety-focused revision would still have to do.

3.2. Malformed Input as a Security Boundary

Three mechanisms recur across the security evidence for the core library and official tools: arithmetic on file-derived quantities that is not checked for overflow or zero denominators; file-format parsing that constructs internal objects before validating the bytes that describe them; and executable extension code reached through plugin loading, which is treated in Section 4. The first two are examined here.

A CVE history summary is included in Appendix D. The list mixes real HDF5-library issues, removed-tool issues, duplicate/rejected/not-HDF5 entries, and entries with no commit URL. A recurring class represented in the selected history is failure to validate file-derived structure sizes, offsets, counts, message lengths, continuation chunks, datatype metadata, or filter parameters before using them in low-level C memory operations. The 2025 entries include heap overflows, use-after-free, null dereference, memory leak, and resource-consumption defects across object headers, file-space metadata, type conversion, filters, and metadata cache code. This qualitative clustering does not estimate ecosystem frequency.

The fix commentary confirms that several CVEs were symptoms of corrupted metadata not being rejected early. (Examples: the file-space page-size fix added a sanity check immediately after reading `page_size` from the file; the object-header-continuation fix rejects zero continuation chunk sizes; the self-referential continuation-message fix checks expected versus actual deserialized object-header chunks; and the cache-image fix added buffer-size arguments, overflow checks, input validation, and cleanup.)

An August 2026 source-review observation exposes a related but distinct control gap. HDF5 Pickles issue #87 records candidate deserializer invariants whose truth

depends on in-file bytes but whose checks are masked inside `assert(...)` expressions; those expressions provide no runtime validation when assertions are disabled with `-DNDEBUG`. The open catalog task names version-2 B-trees, dataset chunk records, extensible arrays, fractal heaps, free-space managers, shared object-header messages, and metadata-cache images, with a cursory estimate of 50–100 instances [57]. A fixed August 25 develop snapshot corroborates the mechanism: its build configuration notes that CMake adds `NDEBUG` for Release builds, while representative source sites assert a decoded filtered- chunk address and byte count before returning success, assert extensible-array parameters before geometry and allocation calculations, and assert a reverse-filter output size before copying the expected fractal-heap block size [87, 82, 83, 84]. This static sample confirms a source-level assurance gap, but the 50–100 estimate is not a verified site count, and the available evidence does not establish that every candidate is a sole guard, release-reachable in a supported version, or capable of a particular adverse outcome. This report therefore tracks the condition as evidence record `CORE-SEC-003`, not as a separately rated formal finding.

One member of that population has since been carried to reproduction, and it is worth stating precisely because it bounds what the rest of the record does and does not claim. A version-2 B-tree header declaring a zero record size reaches an integer division while the leaf record capacity is derived; the sole guard is an assertion in `H5B2hdr.c`, absent under `-DNDEBUG`. The reported measurement records `SIGFPE` from a shipped `h5dump-H` against HDF5 2.3.0 and from the public `H5Fopen/H5Gopen2/H5Gget_info` path, at both 32- and 64-bit widths [33]. That instance meets all three conditions the record leaves open, so it is promoted into `CORE-SEC-002` by its consequence; see Appendix A. It does not establish the size of the remaining population, the affected version range — which its reporters expressly decline to assert — or that other candidates share its consequence class. A second candidate, the cache-image LRU rank relation, is reported without a measured release-build consequence [31]. One confirmed instance is a reason to complete the catalog, not a substitute for it. Regardless of the final count, a predicate whose validity depends on file bytes must have an always-active runtime check; an assertion may duplicate that check for diagnostics but must not enforce it alone.

The observed root-cause categories are more compact than a CVE-by-CVE list and are summarized in Table 7. They were derived independently in this assessment, and they are consistent with the SSP SIG security threat model, which decomposes the same evidence into thirteen mechanism entries [47, 51]; that decomposition is the finer-grained one and is the better basis for the invariant tranches proposed in CORE-S1.

Table 8 maps each category to the standard weakness vocabulary, using the SSP SIG’s top-CWE list for HDF5 contexts [53]. The mapping is worth stating explicitly because it makes the report’s central security claim checkable against an external taxonomy rather than only against its own categories: nearly every entry resolves to CWE-20 and its children as the *cause*, with the memory-safety CWEs appearing as *consequences*. Per-CVE tags are in Appendix D.

Table 7 – Observed root-cause categories in the selected HDF5 CVE history

Root-cause category	What it means for HDF5
unchecked file-derived sizes / counts / offsets	A recurring evidenced class. HDF5 metadata controls object layout, heap blocks, chunks, dataspace, filters, links, and type descriptors. When those fields are malformed, C code later trusts them.
inconsistent internal bookkeeping	Attribute tables, cache images, continuation chunks, and free lists often needed separate “allocated capacity” versus “actual count” tracking.
decode/encode asymmetry	Several issues happen because decode did not reject bad messages, then encode/flush later assumed valid native structures.
arithmetic precondition failures	Divide-by-zero and multiplication-overflow checks were missing in chunking, layout, b-tree, filter, and dataspace paths.
unsafe cleanup/lifetime paths	Use-after-free, double-free, leaks, and null dereferences show that malformed input often breaks unwinding logic as much as the primary decode logic.
unbounded traversal, recursion, loop-progress, and resource-limit failures	File-controlled graphs, dimensions, and metadata can drive recursion, repeated work, or allocation without a structural progress rule or explicit budget.
legacy tool attack surface	gif2h5, HDF to GIF conversion, h5dump, and h5repack appear repeatedly. Removal of gif2h5 was a structural mitigation, not just a patch.
triage quality, duplicate/variant clustering, reachability, and fix-completeness metadata	Several entries have <code>commit_url:null</code> , “not filed against HDF5,” “cannot reproduce,” “not applicable,” or tentative confidence. Those should not be counted as reviewed HDF5 fixes without qualification.

Table 8 – Observed root-cause categories mapped to standard weaknesses

Root-cause category	Cause (CWE)	Consequence (CWE) and note
unchecked file-derived sizes, counts, offsets	CWE-20, CWE-1284, CWE-1285	CWE-787, CWE-125. The single largest group. A quantity or an offset taken from the file is used before it is validated.
inconsistent internal bookkeeping	CWE-20, CWE-1284	CWE-787, CWE-908. Allocated capacity and actual count are conflated, so a validated bound does not describe the buffer actually held.
decode/encode asymmetry	CWE-20, CWE-1286	CWE-787. Decode accepts a structure that encode or flush later assumes is well-formed; the syntactic check that was never made surfaces on the write path.
arithmetic precondition failures	CWE-20, CWE-1284	CWE-369, CWE-190. Divisors and multiplications derived from file bytes without a zero or overflow precondition.
unsafe cleanup and lifetime paths	CWE-20	CWE-416, CWE-415, CWE-476, CWE-401. Distinctive in that the unwinding path, not the decode path, is what malformed input breaks.
unbounded traversal, recursion, and resource limits	CWE-20, CWE-1284	CWE-400, CWE-674. File-controlled graphs and counts drive work with no structural progress rule or explicit budget.
legacy tool attack surface	CWE-20	CWE-787, CWE-125. Not a distinct weakness class; a distinct <i>exposure</i> class, since these are the programs users run first on untrusted files.
type-confusion on message and datatype flags	CWE-20, CWE-1287	CWE-787. A message or datatype is cast to a structure its declared type does not support — validation of a specified <i>type</i> , distinct from validating a quantity or an index.
triage and fix-completeness metadata	<i>n/a</i>	<i>n/a</i> . A records-quality category, not a weakness. It affects what can be concluded from the history, not what the library does.

Read together, the two tables state the report’s core security claim in standard terms: the memory-safety weaknesses that dominate the CVE history are consequences, and the cause they share is CWE-20. That is why CORE-S1 through

CORE-S3 target validation and checked arithmetic rather than the individual crash classes, and why closing a CWE-787 instance does not close the class that produced it.

3.3. HDF5 Privacy: Assets and Exposure

3.3.1. Introduction

HDF5 design features create predictable privacy-exposure mechanisms, but realized harm, exposure, and any lifecycle trend are deployment-specific. The file format and library are one source of exposure surfaces; scientific workflows, software ecosystems, interoperability mechanisms, and changing data contexts can add others.

Before describing the assets that require protection in HDF5 and the mechanisms through which privacy exposure may occur, it is important to emphasize several fundamental characteristics of HDF5 and the audit approach adopted in this document.

- *HDF5 privacy primarily concerns accidental or unintended exposure of sensitive information rather than malicious attacks.* Malicious attacks and unauthorized access are primarily addressed through cybersecurity mechanisms and are outside the scope of this section.
- *HDF5, by design, does not provide privacy protections.* When privacy is a concern, users and organizations are responsible for protecting data throughout the entire data lifecycle. In this document, the term data refers not only to individual HDF5 objects within a single file, but also to collections of HDF5 files and external data accessible through the HDF5 library, connectors, and related software components.
- *HDF5 neither implements nor enforces privacy standards.* Furthermore, even when privacy standards and technical safeguards are in place, sensitive information may still be exposed through legitimate data processing, contextual interpretation, pattern discovery, inference, aggregation, and other analytical activities.

- *HDF5 does not manage authentication, authorization, or filesystem protection.* However, privacy risks extend beyond traditional security concerns. Sensitive information may be exposed during legitimate data use by authorized users, even within highly secure computing environments.
- *Additional lifecycle stages can add privacy-exposure surfaces.* Sensitive information stored in HDF5 may be exposed during data creation, processing, storage, sharing, transfer, reuse, or archival; classification, minimization, access restrictions, and lifecycle controls can increase or reduce the realized deployment risk.
- *HDF5 extensibility mechanisms and associated software ecosystems may alter the privacy characteristics of data.* Features such as filters, Virtual Object Layer (VOL) connectors, Virtual File Drivers (VFDs), and external applications, including conversion and analysis tools, may introduce new privacy exposure surfaces or modify existing ones.

The remainder of this section focuses on the assets that require protection in HDF5, the mechanisms through which sensitive information may be exposed, and potential approaches for mitigating privacy risks.

3.3.2. HDF5 Assets and their Exposure

This subsection identifies five classes of asset that may require protection in an HDF5 deployment: the scientific data themselves, user-defined metadata, structural metadata, information derived or reconstructed from combinations of the former, and artifacts produced by the surrounding tools and workflows. Each is treated in turn below. The classes are cumulative rather than alternative — a single sharing boundary can disclose through all five — and the enumeration in `CORE-PRIV-S1` expects a disposition for every class, including a positive statement where a class was examined and found empty.

Data Assets

The primary assets requiring protection are the scientific data themselves. In

HDF5, these include data stored in HDF5 objects, such as files, groups, datasets, datatypes, and dataspace, which represent experimental observations, measurements, simulations, images, sensor outputs, and computational results.

Scientific data may be exposed through direct access using HDF5-based software, including command-line utilities, graphical viewers, and programming interfaces such as Python. However, privacy-sensitive information may also be inferred through examination of the low-level storage organization and auxiliary metadata embedded within the file to support portability, interoperability, and efficient data access. For example, string attributes visible through standard operating system utilities (e.g., the Unix strings command) may disclose dataset names, experimental identifiers, or other descriptive information. Similarly, inspection of dataset headers may reveal storage layout, chunk organization, compression methods, or data acquisition patterns (e.g., frame-based acquisition stored as chunked datasets). Examination of embedded strings may also disclose the use of external storage or references to datasets located in other HDF5 files.

It is important to distinguish between protecting access to data values and protecting information about data representation. Even when raw data are protected through mechanisms such as compression or encryption, information describing their storage organization, layout, or access methods may remain accessible and reveal sensitive contextual information about the underlying data or the processes that generated them.

Metadata Assets

User-defined metadata often constitute one of the most privacy-sensitive asset classes in HDF5. The format was intentionally designed as a self-describing data model that allows extensive contextual information to be stored alongside scientific data. While HDF5 attributes are the most recognizable form of user-defined metadata, they represent only one category of descriptive information maintained within an HDF5 file.

User-defined metadata include all user-created HDF5 objects and object properties that organize, identify, or describe stored data. These include groups, datasets, their associated datatypes and dataspace, committed datatypes, and attributes

attached to HDF5 objects. Each HDF5 attribute is itself a structured metadata object with a name, datatype, and dataspace. Additional metadata assets include file and object names (link names), committed datatype names, compound datatype member names, enumeration labels, object comments, and information stored in the user block. Collectively, these metadata describe the logical organization, semantics, and relationships of the stored data and frequently contain experiment-specific, operational, or organizational information.

Like scientific data, metadata assets may be exposed not only through HDF5 applications but also through direct examination of the binary file format. Inspection of file structures or extraction of embedded strings without using HDF5 software may reveal object names, descriptive attributes, storage relationships, user block contents, and other contextual information that could disclose sensitive information about the data, their origin, or the organization that produced them.

Structural Metadata Assets

Privacy-sensitive information may also be inferred from the structural characteristics of HDF5 files. Structural metadata assets include user-defined organizational structures, such as the hierarchical arrangement of objects, dataset storage layouts (e.g., contiguous, chunked, compact, or external storage), filter pipelines, and relationships established through links, object references, region references, or virtual datasets (VDS). In addition, the HDF5 library maintains internal structural metadata that describe and manage these user-defined structures, including chunk indexing schemes, B-tree organizations, and other implementation-specific data structures.

Even when raw data are inaccessible, for example, because they are encrypted, compressed, stored externally, or protected by file-system access controls, structural metadata may disclose information about the purpose of the data, experimental design, workflow organization, relationships among datasets, or assumptions made by the generating application. Naming hierarchies, grouping strategies, storage layouts, and reference relationships may reveal semantic information about scientific activities or operational processes. Likewise, implementation-specific structures, such as chunk indexing formats or other internal metadata, may dis-

close characteristics of the HDF5 library version or software implementation used to create or process the file.

Like scientific data and user-defined metadata, structural metadata may be exposed through HDF5 applications or by direct examination of the binary file format without using HDF5 software.

Derived and Reconstructed Information

Privacy-sensitive information in HDF5 may also arise through inference or reconstruction from combinations of available data and metadata. Derived assets include relationships inferred from multiple HDF5 datasets, external links, object references, region references, virtual dataset (VDS) mappings, or information obtained by combining HDF5 data with external sources. These relationships may reveal information that is not explicitly represented within any single dataset or file.

For example, correlating quality-control datasets with experimental observations, combining temporal and spatial measurements, or reconstructing relationships across linked HDF5 files may disclose sensitive characteristics that are not apparent when individual datasets are examined in isolation. Consequently, privacy-sensitive assets extend beyond explicitly stored data and metadata to include information that can be inferred through aggregation, correlation, reconstruction, or other forms of analysis.

Workflow and Applications Artifacts

Privacy-sensitive assets in HDF5 extend beyond data and metadata stored within HDF5 files to include artifacts created by applications, processing workflows, and the surrounding software ecosystem. These artifacts include command outputs (e.g., h5dump output), diagnostic logs, temporary files, metadata snapshots, cached intermediate datasets, conversion artifacts, and outputs generated by plugins or connectors (e.g., binary files produced by an HDF5 VOL connector for another scientific data format).

Examples include VFD SWMR metadata snapshots, cached intermediate results, diagnostic logs, and metadata generated during translation between non-self-describing data formats and HDF5. Such artifacts may expose scientific data, metadata, storage organization, workflow decisions, or implementation details beyond what is contained in the original HDF5 file. Because they are often created automatically, these artifacts may persist beyond their intended lifetime and remain unnoticed by users focused primarily on the scientific datasets themselves.

Scientific data processing workflows routinely enrich, restructure, annotate, and transform datasets to improve interoperability, reproducibility, usability, or analytical value. However, these processes may also introduce or preserve contextual information that was not intended for disclosure. Automatically generated metadata, intermediate processing products, and derived datasets may reveal experimental assumptions, workflow decisions, data provenance, software implementation details, or scientific semantics that were absent from the original data. Consequently, privacy exposure may arise not from malicious activity or unauthorized access, but as an unintended consequence of routine scientific data processing.

3.4. HDF5 Tools

Not strictly part of the core library, but the tools are an important part of the ecosystem and have their own set of issues.

3.4.1. Privacy exposure through HDF5 command-line tools

HDF5 command-line tools provide convenient mechanisms for managing HDF5 files. They can be used to display or extract raw data, and user-defined and structural HDF5 metadata (e.g., `h5dump`, `h5ls`, `h5debug`), edit raw data and attributes with `h5edit`, extract or aggregate data using `h5copy`, change characteristics of HDF5 objects, for example, remove or add data filters and change storage properties using `h5repack`. HDF5 command-line tools are routinely employed throughout the HDF5 data lifecycle. While the command-line tools are

essential for working with HDF5 data, they are ultimate sources of unintended privacy exposure.

The HDF5 tools were designed to ease access to HDF5 data, to reveal information about the contents and organization of HDF5 files, and to manage that information while avoiding steep learning curve of HDF5 programming. I.e., HDF5 command-line tools were designed to expose HDF5 assets. Depending on the tool and the selected options, they may display object names, attributes, datatype definitions, storage layouts, chunking parameters, filter pipelines, external storage locations, object references, virtual dataset mappings, and portions or all of the stored data to name a few. Even when users intend only to verify file integrity or inspect metadata, command output may disclose information that is considered sensitive in a particular context.

Privacy exposure is not limited to information displayed on the terminal by `h5dump` or `h5ls`. Command output is frequently redirected to files, incorporated into automated workflow logs, attached to bug reports, archived as diagnostic information, or shared. The output can be used by other tools (e.g., `h5import`) or applications to generate new data. Consequently, metadata and scientific data extracted by HDF5 tools may persist outside the original HDF5 file and become accessible to individuals who were never intended to access the underlying information.

Several command-line tools may also create new HDF5 files or transform existing ones. `h5repack` and `h5copy` may generate additional copies of data and/or aggregate data, modify storage layouts, remove or add data filtering; `h5edit` may modify and introduce new metadata, change raw data values and preserve information that was not expected to be retained. I.e., the tools can alter the privacy characteristics of the resulting files, particularly when they are incorporated into larger workflows, backups, or public data repositories.

Because HDF5 command-line tools are widely used in automated scripts, continuous integration systems, and scientific workflows, their privacy implications should be evaluated alongside those of the HDF5 library and HDF5-based applications. The privacy mitigation strategies discussed in the next section apply not only to the HDF5 library itself but also, and often to an even greater extent, to

HDF5 command-line tools. This is because these tools are designed to expose HDF5 assets and to generate persistent outputs (e.g., new binary and text files).

The table below summarizes the assets exposed by the listed command-line tools.

Table 9 – Privacy exposure through HDF5 command-line tools

Tool	Exposed assets	Workflow artifacts	Privacy considerations
h5dump	Raw data; user-defined and structural metadata	Text dumps, redirected output, logs	Displays datasets, attributes, datatypes, storage layout, filter information, object references, and VDS mappings. May expose data stored in other HDF5 files through external links and Virtual Datasets (VDS). Output is frequently redirected to files or logs, creating persistent copies of sensitive information.
h5ls	User-defined and structural metadata (optionally raw data)	Terminal output, logs	Reveals object hierarchy, object names, datatypes, storage properties, links, and object relationships. With the <code>-d</code> option, also displays dataset values.
h5repack	Raw data; user-defined and structural metadata	New HDF5 file	Creates additional HDF5 copies while potentially modifying storage layout, filters, and metadata. The <code>-merge</code> option can aggregate datasets reachable through external links into a single file. May access non-HDF5 data through configured VFDs and VOL connectors.
h5diff	Raw data; user-defined metadata	Comparison reports	Reads and compares datasets and attributes. Reports may disclose sensitive values or structural differences.
h5copy	Raw data; user-defined and structural metadata	Additional HDF5 files	Copies objects within or between HDF5 files, increasing the number of persistent copies of sensitive information.
h5stat	Structural metadata	Statistics reports, logs	Reports object counts, storage statistics, chunk information, free space, and other implementation characteristics that may reveal dataset complexity or workflow properties.
h5debug	Structural metadata	Debug output	Displays low-level internal file structures, addresses, B-trees, heaps, chunk indexes, and other implementation metadata intended for debugging.
h5watch	Raw data; user-defined metadata	Monitoring output, logs	Monitors SWMR datasets and reports newly written data, timestamps, and update activity in real time.

Table 9 – Privacy exposure through HDF5 command-line tools, continued

Tool	Exposed assets	Workflow artifacts	Privacy considerations
h5import	Raw data; user-defined metadata	New HDF5 file	Imports external data into HDF5, creating additional copies while potentially preserving metadata and semantics from the source format.
h5jam/h5unjam	User block contents	Extracted or embedded files	Stores or extracts arbitrary information in the HDF5 user block, including scripts, XML documents, application metadata, or proprietary information.
h5mkgrp	Structural metadata	Modified HDF5 file	Creates or modifies the group hierarchy. Limited direct privacy exposure but changes descriptive metadata and namespace organization.
h5edit	Raw data; user-defined metadata	Modified HDF5 file	Creates or modifies attributes; modifies raw data. Limited direct privacy exposure but changes data and user-defined metadata.
h5format_convert	Raw data; structural metadata	Converted HDF5 file	Converts files between HDF5 format versions, creating additional copies while preserving or rewriting internal metadata structures.
h5clear	Structural metadata	Modified HDF5 file	Clears SWMR status and metadata cache information. Primarily affects recovery metadata and file consistency information.

Note: All HDF5 command-line tools are built on top of the HDF5 library and therefore inherit its capabilities. When VFDs or VOL connectors are configured, these tools may transparently access data stored in remote services, external storage systems, or non-HDF5 data sources exposed through connectors. Consequently, privacy exposure is not necessarily limited to the HDF5 file specified on the command line but may extend to the broader HDF5 ecosystem.

3.5. Audit Findings and Mitigation Planning

Formal core and official-tool findings are registered as CORE-SEC-001, CORE-SEC-002, CORE-TOOL-001, CORE-PRIV-001, CORE-SAFE-001, and CORE-SAFE-002 in Appendix A. The two shared-event security findings are provisional historical

parser-boundary scenarios; they do not assert that every current HDF5 release contains every represented defect. The assert-masking observation is separately registered as evidence record CORE-SEC-003 until exact source sites, supported release paths, and consequences are established.

Priority reconciliation.

The response order comes from Table 34, not from the order of recommendations in this chapter. Confirmed affected scope for CORE-SEC-001, CORE-TOOL-001, and CORE-PRIV-001 receives Near-term treatment; CORE-SEC-002 receives Planned treatment. CORE-SAFE-001 and CORE-SAFE-002 receive Near-term *containment* — capacity headroom and monitoring, flush and checkpoint discipline, a rehearsed recovery procedure, and independent copies of unregenerable data — with their structural treatment in CORE-SAFE-S1 and CORE-SAFE-S2 on the Planned and Phase 3 horizons; RM-2D produces their acceptance evidence rather than deciding whether they can be rated. The CORE-SEC-003 catalog and release-build differential study has a 30-day Phase 1 assurance target under RM-1E; this scheduling choice does not assign it a risk tier. A confirmed case is routed by its supported consequence and affected scope, including under CORE-SEC-001 for a memory-unsafe path or CORE-SEC-002 for termination or resource exhaustion. Phase 1 patching, isolation, and privacy controls treat concrete populations. Phases 2–4 provide reusable guardrails, class prevention, and sustainment; that structural work may begin in parallel but does not extend an earlier response clock.

3.5.1. Security and Safety

The selected CVE history shows repeated malformed-file parsing pressure against a complex binary metadata machine. Affected variants should be patched or contained promptly. Prevention must not stop at those local repairs: it should add systematic metadata validation at decode boundaries, explicit invariants for each on-disk structure, fuzz corpora tied to invariants, and hard separation between parser acceptance and internal object construction.

Goal: HDF5 should be able to parse any byte stream as hostile input and either reject it cleanly or produce a validated internal object graph. Anything else is security debt.

Structural prevention program: maintain current-scope fixes or containment while bootstrapping invariants, checked primitives, regression and fuzz harnesses, review gates, and metrics in parallel. Migrate each decode boundary in independently releasable tranches once its invariants and checked primitives exist, and make the gates enforce each accepted tranche. This program neither precedes nor delays Phase 1 treatment or Phase 2 release safeguards.

The `CORE-S` identifiers below are stable identifiers for recommendations in the core-security track. Their numbers do not express risk, urgency, effort, dependency, or implementation order.

Parser normalization is a central structural control: decode file bytes into a validated intermediate form before constructing internal objects so malformed metadata cannot reach trusted C paths. It is a recurrence-prevention control, not a substitute for patching or isolating a known affected release.

Our goal should be: *make malformed HDF5 files boring*. Every corrupted file should terminate in a controlled `H5E_BADVALUE`, `H5E_OVERFLOW`, `H5E_CORRUPT`, or `H5E_RESOURCE` path. No segfaults. No assertions. No leaks. No undefined behavior. No late-stage encode/flush crash from a bad decode.

The current CVE list includes many issues in `H5FS`, `H5C`, `H5O`, `H5T`, `H5Z`, and vector-copy paths, with repeated heap overflows, use-after-free, null dereference, and resource-consumption outcomes. HDF5's own format documentation explains why this happens: a file is a graph of groups, datasets, and named datatypes; on disk it is represented through lower-level structures such as the superblock, B-tree nodes, object headers, global heaps, local heaps, and free-space metadata. That means the library is parsing a dense graph of mutually referential, *attacker-controlled* metadata.

3.5.2. Prior Art: What Already Exists

Several recommendations below describe controls that have already been designed, and in some cases built and measured, in the H5Lens / `hdf5-pickles` project [30]. Presenting them as new proposals would misstate both the effort involved and the state of the art inside the programme, so this subsection records what exists before the recommendations restate what is still missing.

H5Lens describes HDF5 on-disk metadata as machine-readable format definitions and builds four things on top of them: `h5policy`, an independent preflight oracle for untrusted files with named security profiles and a regression corpus; a versioned invariant registry mapping each record family's invariants to finding codes, enforcement points, tests, fuzz targets, and migration status; `h5patch`, an evidence-gated repair planner; and a differential measurement harness that records, per record family, what a selected `libhdf5` build actually did with a malformed specimen [39, 38, 37]. The accompanying design document, *A CVE Strategy for the HDF5 Library*, sets out the decode-validate-construct architecture in more detail than this report does [56, 58].

Two places where the existing design is ahead of this report. This matters for sequencing, not just for credit.

1. **Three validation scopes, not two phases.** CORE-S2 below describes decode, validate, construct. The strategy document decomposes the middle step into *record-local*, *aggregate-object*, and *reference-graph* validation, which is the distinction that actually explains the harder CVE variants: a continuation message that references part of itself, a shared message that resolves through itself, or a compound member that extends past its enclosing datatype are all locally well-formed and fail only at a wider scope. A two-phase boundary does not by itself catch them.
2. **Materialization versus activation.** The strategy document separates constructing a native object from performing operations with side effects — cache publication, ID registration, plugin loading, decompression, user callbacks, and *external-file access*. Its rule is that successfully parsing fields

must not authorize any of those. This report has no equivalent concept, and the omission is consequential: that rule is the core-library-side statement of the same boundary failure this report rates as APP-SEC-001 at the service boundary. The two framings are complementary, and a reader of this report alone would conclude that external-resource policy is purely a downstream concern.

Re-scoped recommendations. Table 10 restates the affected recommendations against what exists. The revised effort in the last column supersedes the estimate carried in Appendix B for these six items, and the reuse is subject to the licensing boundary in Section C.3: identifiers, boundaries, fixtures, and measurements are consumable as data; GPLv3 implementation source is not consumable into a BSD-licensed library.

Table 10 – Recommendations re-scoped against existing H5Lens work

Rec.	What already exists	What this recommendation therefore is	Effort
CORE-S1	A versioned, machine-readable invariant registry with per-record-family invariants, finding codes, enforcement points, test and fuzz-target mappings, and per-family migration status.	<i>Adopt the vocabulary; build the enforcement.</i> The design and enumeration work is largely done. What remains is mapping each adopted invariant to an always-active enforcement point <i>inside libhdf5</i> and proving it in both build modes.	Large → Medium (design and enumeration reused; enforcement is new)
CORE-S2	A complete architectural design with three validation scopes, the materialization/activation rule, a seven-phase incremental migration plan, and a working bounded decoder demonstrating the contract.	<i>Adopt the design, including the two refinements above; migrate the library.</i> The remaining work is the migration itself, which is genuinely large and is not reduced by the existence of a design.	Large → Large (unchanged; design reuse does not reduce migration cost)

Table 10 – Recommendations re-scoped against existing H5Lens work, continued

Rec.	What already exists	What this recommendation therefore is	Effort
CORE-S4	h5policy implements four profiles — legacy, trusted-fast, untrusted-strict, forensic — with the same names and substantially the same semantics this recommendation proposes, including feature policy and resource budgets.	<i>Port a proven profile design into the library API.</i> The semantics have been exercised against a corpus; the open work is the H5P surface, default-state behavior, and enforcement inside libhdf5 rather than beside it.	Large → Medium
CORE-S5	A mutation harness with per-family recipes, truncation sweeps, and a boundary-value inventory identifying which values the corpus lacks. Its own coverage report records that OSS-Fuzz integration is absent for every family.	<i>Adopt the harness design and close the two named gaps:</i> OSS-Fuzz integration and per-family fuzz targets, which exist for only a small minority of families. This recommendation should stop proposing OSS-Fuzz as a new idea and start treating it as a known, scheduled gap.	Large → Medium
CORE-S6	A malformed-input corpus with per-fixture expected-outcome records, a valid control set, and a manifest — structurally the <code>expected.yml</code> design sketched below.	<i>Adopt the corpus and its expectation format;</i> add HDF5-side CI integration and the debug/-DNDEBUG parity requirement. Subject to per-specimen provenance review before redistribution.	Medium → Small
APP-S2	h5policy is a working preflight validator for untrusted files, invocable per profile and emitting structured results.	<i>Two distinct deliverables, previously conflated.</i> A validator exists and can be used today by deployments willing to run an external GPLv3 tool. What does not exist is the enforcement coupling this recommendation requires — binding the preflight verdict to the same open handle or verified file identity used at load time — which is the part that makes preflight an authorization rather than an advisory.	Large → Medium (validator reused; coupling is the real work)

What this report adds that the existing work does not cover. Stating the reuse honestly also requires stating the complement. The application-boundary findings, the independent-implementation and interoperability analysis, the sepa-

ration of inherent from residual risk with response clocks and accountable owners, and the attribution of recommendations to enforcing actors across downstream projects have no counterpart in H5Lens, which is deliberately a format-and-oracle project rather than an ecosystem-governance one. The two are complementary. The defect corrected here is that this report previously described the overlap as future work.

An unclosed loop. The existing case records report that a substantial majority of their tracked divergences are *oracle-hardened*: the oracle rejects the specimen and the libhdf5 side is unfixed. Two of those concern external-file-list handling, and one — an external data file opened before the wrapped segment is validated — is recorded as uninvestigated [32]. If substantiated, that is a core-library instance of the activation-before-validation rule above and bears directly on APP-SEC-001. This report did not evaluate it, and RM-1E should take the full set of oracle-hardened cases as triage input.

Recommendation CORE-S0: Separate defect closure from recurring-class closure

Adopt the following rule:

A verified local repair may close the affected defect, but it must not be represented as elimination of the recurring class without structural evidence.

For every new CVE or crash:

Table 11 – Required artifacts for defect closure and recurring-class tracking

Required artifact	Purpose
crashing input	becomes a permanent corpus seed
minimized input	used in quick PR regression
root-cause category	maps to a known invariant class
invariant update	records and advances prevention of sibling bugs
sanitizer regression	proves no crash, no leak, no UAF

Table 11 – Required artifacts for defect closure and recurring-class tracking, continued

Required artifact	Purpose
negative test	checks the same controlled failure in assertion-enabled and <code>-DNDEBUG</code> builds
reviewer checklist update	prevents reintroduction

Variant closure requires evidence that the affected path is repaired or contained and that its negative and sanitizer regressions pass. A local `if` may therefore be a valid treatment, including for an embargoed fix, while the corresponding **class-prevention record** remains open until the invariant framework and related gates cover sibling cases.

Recent commits show the right direction, but too locally. One fix added a sanity check immediately after reading file-space page size from the file and used the same fuzzer to verify that the previous assertion became a corrupted-file error. [35] Another hardened `H5C__reconstruct_cache_entry()` by adding buffer-size tracking, overflow checks, input validation, and safe cleanup. [34] Those are exactly the moves to systematize.

Recommendation CORE-S1: Build an invariant registry for the file format

Prior art: see Table 10; this item is re-scoped against existing work. Create a versioned document and machine-readable metadata file, for example:

```
security/invariants/
  superblock.yml
  btree_v2.yml
  chunk_records.yml
  extensible_array.yml
  object_header.yml
  dataspace.yml
  datatype.yml
  layout.yml
```

```
filters.yml
heaps.yml
fractal_heap.yml
free_space.yml
links.yml
shared_object_header_messages.yml
cache_image.yml
```

Each entry should define:

```
id: H5O-CONT-001
component: object_header
structure: continuation_message
condition: continuation_chunk_size > 0
failure: H5E_CORRUPT
checked_at:
  - H5O__cont_decode
  - H5O__chunk_deserialize
release_enforced: true
diagnostic_assert_sites: []
fuzz_targets:
  - fuzz_object_header
cves:
  - CVE-2025-6856
  - CVE-2025-6816
```

The following are candidate initial invariant-registry families. Tranche order must be recorded in the implementation plan from affected scope, coverage gaps, and concrete enabling dependencies; this list is not a risk rank.

Table 12 – Candidate initial invariant-registry families

Area	Candidate invariants
superblock	offset size and length size are valid; base address, EOF address, driver info address, and root object address are inside the file or explicitly undefined; EOF does not wrap

Table 12 – Candidate initial invariant-registry families, continued

Area	Candidate invariants
object headers	message length fits inside chunk; continuation chunks are nonzero; continuations do not self-reference; continuations are acyclic; message flags match message type; unknown messages cannot be cast into known message structures
free-space meta-data	page size is nonzero and below max; section size fits page/file bounds; serialized section count matches decoded section count
heaps	heap object offset and size are inside heap; free-list blocks do not overlap; free-block list is acyclic; no allocation uses unchecked <code>count*size</code>
dataspaces	rank is bounded; dimension product is checked; selected point count is checked; hyperslab stride, count, block, and start do not overflow
datatypes	storage size is nonzero; precision fits storage size; member offsets fit compound size; string length is bounded; vlen/ref types cannot escape validation
layouts	compact data size equals <code>dataspaceelements*datatype size</code> ; chunk dimensions are nonzero; chunk product is checked; chunk index references are inside file bounds
filters	parameter count is bounded; decompressed size is bounded; scale-offset precision is validated; filter output ratio cannot exceed configured resource limits
cache image	cache-entry count, address, size, and alignment are bounded; decoding consumes exactly the expected number of bytes; cleanup is safe after partial decode
tools	string rendering is length-bounded; conversion tools never trust input dimensions; tool-only parsers use the same checked arithmetic wrappers as the library

The seven component families named in issue #87 are the mandatory first audit perimeter for assert-masked checks. That perimeter is not itself a risk rank. Within it, first migrate a candidate that is a sole release-reachable guard for memory bounds, pointer arithmetic, allocation, copy, or traversal; then address termination-only paths and redundant diagnostics. Record the reachability and consequence evidence so a raw candidate count cannot substitute for prioritization.

This registry becomes the source of truth for code review, tests, fuzzing, and documentation.

Recommendation CORE-S2: Introduce a two-phase parser boundary

Prior art: see Table 10; this item is re-scoped against existing work. Right now, too many paths appear to do this:

```
file bytes -> native internal object
             -> later code assumes validity
```

Change the architecture to:

```
file bytes -> bounded raw decode
             -> semantic validation
             -> native object construction
```

Concrete rule: *No public or internal runtime object is constructed from file metadata until the corresponding validator has passed.*

Implement the pattern by component:

```
1 | herr_t H5O_msg_decode_raw(const uint8_t *buf, size_t buf_size,
   |   ↪ H5O_msg_raw_t *raw);
2 | herr_t H5O_msg_validate(const H5F_t *f, const H5O_msg_raw_t *raw,
   |   ↪ H5O_validation_ctx_t *ctx);
3 | herr_t H5O_msg_construct(const H5O_msg_raw_t *raw, H5O_msg_t *out);
```

Do not let decode functions allocate complex structures before basic bounds are known. A raw decoded object should hold values, offsets, and spans, not trusted pointers into long-lived internal state.

All decode functions that consume file bytes should take one of these forms:

```
1 | herr_t decode(const H5F_t *f, const uint8_t *buf, size_t buf_size,
   |   ↪ out_t *out);
2 |
3 | herr_t decode_cursor(H5_decode_cursor_t *cur, out_t *out);
```

Ban this pattern in security-sensitive decode code:

```
1 | const uint8_t **pp
```

unless there is also an explicit `p_end` or remaining-size counter. The cache-image fix already moved in this direction by adding a buffer-size argument and bounds checks. [34]

Recommendation CORE-S3: Centralize checked arithmetic and allocation

Most parser bugs become exploitable when metadata reaches allocation, copy, or pointer arithmetic. Add a small, mandatory checked-arithmetic layer.

Example API:

```
1 herr_t H5_checked_add_hsize(hsize_t a, hsize_t b, hsize_t *out);
2 herr_t H5_checked_mul_hsize(hsize_t a, hsize_t b, hsize_t *out);
3 herr_t H5_checked_addr_add(haddr_t base, hsize_t len, haddr_t eoa,
  ↪ haddr_t *end);
4 herr_t H5_checked_alloc_size(size_t count, size_t elem_size, size_t
  ↪ *bytes);
```

Then make these rules review-blocking:

Table 13 – Review-blocking parser arithmetic and allocation patterns

Ban	Replacement
malloc(n*size)	H5MM_calloc_checked(n, size)
addr+size	H5_checked_addr_add(addr, size, eoa, &end)
memcpy(dst, src, n) from file-derived size	H5_checked_memcpy(dst, dst_size, src, src_size, n)
pointer increment without p_end	cursor-based decode
raw recursion through object graph	bounded traversal context

The 2024 security commit added multiplication-overflow checks for dataset storage size and checks against file bounds. [36] That pattern should become a library-wide rule, not a local repair.

Recommendation CORE-S4: Create security profiles for file opening

Prior art: see Table 10; this item is re-scoped against existing work. Add explicit modes. Do not rely on users to infer safe behavior.

```
1 typedef enum {
2     H5_SECURITY_PROFILE_LEGACY,
```

```
3 |     H5_SECURITY_PROFILE_TRUSTED_FAST,  
4 |     H5_SECURITY_PROFILE_UNTRUSTED_STRICT,  
5 |     H5_SECURITY_PROFILE_FORENSIC  
6 | } H5_security_profile_t;
```

Suggested semantics:

Table 14 – Security profile semantics	
Profile	Behavior
legacy	current compatibility bias
trusted_fast	normal validation, generous resource limits
untrusted_strict	strict metadata validation, plugin loading disabled unless allowed, resource limits enforced
forensic	never repairs silently, reports all structural anomalies, best-effort inspection without unsafe traversal

Expose it through file-access property lists:

```
1 | H5Pset_security_profile(fapl, H5_SECURITY_PROFILE_UNTRUSTED_STRICT);  
2 | H5Pset_max_metadata_bytes(fapl, 256 * 1024 * 1024);  
3 | H5Pset_max_object_header_chunks(fapl, 1024);  
4 | H5Pset_max_btree_depth(fapl, 64);  
5 | H5Pset_max_filter_expansion_ratio(fapl, 100);  
6 | H5Pset_plugin_policy(fapl, H5_PLUGIN_POLICY_DISABLED);
```

This matters because HDF5 supports dynamically loaded plugins, and the runtime API can enable or disable plugin classes; the docs also state that `HDF5_PLUGIN_PRELOAD=: :` disables all dynamic plugins. Turn that capability into a first-class security policy, not an environment-variable trick.

Recommendation CORE-S5: Fuzz by structure, not just by whole-file crashes

Prior art: see Table 10; this item is re-scoped against existing work. Whole-file fuzzing is necessary but insufficient. It finds symptoms. Structure-aware fuzzing finds families.

Treat the following as an initial harness-investment portfolio, not a finding rank or response order. Known finding fixtures and changed paths enter regression immediately; new harnesses may proceed in parallel, with tranche order recorded from shared-enablement and coverage evidence.

Table 15 – Candidate structure-aware fuzzing portfolio

Candidate target	Coverage rationale
object-header message decoder	shared decode paths and continuation structures
datatype decoder and converter	complex nested metadata and conversion operations
dataspace and selection decoder	rank, count, hyperslab, and point arithmetic
layout and chunk index decoder	storage-size, address, and chunk-product checks
free-space and local/global heap decoders	historical overflow and list-corruption variants
filter pipeline and scale-offset filter	compression and decompression boundaries
metadata cache image reconstruction	complex bounds, lifecycle, and cleanup paths
h5dump/h5repack rendering paths	tool paths that add handling and rendering behavior

Use three fuzzing layers:

Table 16 – Fuzzing layers

Layer	Description
byte-level fuzzing	current randomized mutation against APIs
structure-aware mutation	valid superblock with corrupted object headers, valid object header with bad continuation, valid datatype with bad member offset
invariant-guided generation	generate files that violate exactly one invariant at a time

OSS-Fuzz already supports libFuzzer, AFL++, Honggfuzz, Centipede, sanitizers, and distributed execution for C/C++ projects. Use OSS-Fuzz for long-running public coverage and ClusterFuzzLite or local CI for PR-level checks.

Table 17 – Fuzzing acceptance criteria

Stage	Gate
PR	run minimized CVE corpus under ASan/UBSan/LSan
nightly	run structure fuzzers for core parsers
weekly	report coverage by component, not just total line coverage
release candidate	72-hour fuzz soak on changed parser areas
CVE closure	crashing input added to corpus and mapped to invariant

Recommendation CORE-S6: Make malformed-input regression cheap

Prior art: see Table 10; this item is re-scoped against existing work. Create this repository layout:

```
test/security/
  corpus/
    cve/
      CVE-2025-7069/
        input.h5
        expected.yml
      CVE-2025-6269/
        input.h5
        expected.yml
    invariants/
      H5O-CONT-001/
      H5T-SIZE-002/
    harnesses/
      fuzz_object_header.c
      fuzz_datatype.c
      fuzz_dataspace.c
      fuzz_layout.c
      fuzz_filters.c
    expected_errors/
```


corrupt_file.yml

Each expected.yml should include:

```
expected:
  exit: controlled_error
  allowed_errors:
    - H5E_CORRUPT
    - H5E_BADVALUE
    - H5E_OVERFLOW
  forbidden:
    - SIGSEGV
    - SIGABRT
    - leak
    - use_after_free
    - timeout
  invariant:
    - H5FS-PAGE-001
```

This gives maintainers a fast way to prove that old crashes remain dead.

Recommendation CORE-S7: Add a security-review checklist that blocks risky parser changes

Every PR touching decode, encode, object lifecycle, filters, tools, or plugin loading should answer:

Table 18 – Security-review checklist for parser-adjacent changes

Question	Required answer
Does this read file-derived bytes?	list buffer and bounds source
Does this allocate using file-derived values?	show checked multiplication
Does this copy memory using file-derived sizes?	show source and destination bounds
Does this traverse links, chunks, heaps, B-trees, or continuations?	show cycle and depth bounds
Does this construct native objects before validation?	explain why or refactor

Table 18 – Security-review checklist for parser-adjacent changes, continued

Question	Required answer
Does any file-derived validity predicate appear only in an assertion?	show the always-active check and matching -DNDEBUG negative test
Does this add or change cleanup paths?	show partial-init safety
Does this invoke plugins or filters?	show policy interaction
Does this alter error handling?	show malformed-input test

Add automated enforcement where possible:

```
fail PR if new decode function lacks size parameter
fail PR if malloc(count * size) appears outside checked wrapper
fail PR if memcpy appears in parser code without checked wrapper annotation
fail PR if new parser file lacks fuzz target registration
fail PR if new object traversal lacks max-depth or visited-set logic
fail PR if file-derived validity is assert-only
run malformed corpus in debug and -DNDEBUG builds
require the same controlled rejection in both
```

This will annoy contributors at first. Good. That friction is cheaper than post-release CVEs.

Recommendation CORE-S8: Separate compatibility from safety

HDF5's backward compatibility is valuable, but it creates a security trap: old, permissive parsing keeps malformed legacy structures alive.

Implement this policy:

Table 19 – Compatibility and safety profile policy

Case	Behavior
valid legacy file	accepted
ambiguous legacy construct	accepted only in <code>legacy</code> or <code>trusted_fast</code> ; warning in <code>forensic</code> ; rejected in <code>untrusted_strict</code>
structurally invalid file that old versions tolerated	rejected in all non-legacy profiles
parser repair needed	repair only through explicit tool, never silently during normal open
unknown message with unsafe flags	rejected unless profile explicitly allows preservation

The point is not to break legitimate archives. The point is to stop treating corrupted metadata as compatibility.

Recommendation CORE-S9: Harden tools as separate attack surfaces

Do not assume `h5dump`, `h5repack`, converters, and test utilities are less important. They are often the first thing users run on untrusted files.

Actions:

Table 20 – Tool hardening actions

Tool class	Change
inspection tools	use <code>H5_SECURITY_PROFILE_FORENSIC</code> by default
conversion tools	use <code>H5_SECURITY_PROFILE_UNTRUSTED_STRICT</code> by default
rendering paths	bounded strings only
legacy converters	disable by default; build behind explicit <code>HDF5_ENABLE_LEGACY_UNSAFE_TOOLS=ON</code>
plugin-dependent tools	warn or require allowlist

A dangerous file should not become more dangerous because the user tried to inspect it.

Recommendation CORE-S10: Plugin policy and signed extensions

Plugins are executable extension code requiring a distinct control domain; this statement is not a risk rating. They are loaded at runtime. HDF5's plugin system supports dynamically loaded filter plugins and lets users implement custom filters. That is useful, but it must not be invisible in untrusted mode.

Implement:

```
default profile:
  trusted_fast: existing plugin behavior
  untrusted_strict: plugins disabled unless allowlisted
  forensic: plugins disabled; report missing filter
```

Add a plugin manifest:

```
plugin_id: 32008
name: blosc
version: 1.0.0
sha256: ...
signer: ...
allowed_profiles:
  - trusted_fast
  - untrusted_strict
capabilities:
  - filter_decode
limits:
  max_expansion_ratio: 100
  max_alloc_bytes: 1073741824
```

For affected deployments, disable plugins or restrict loading to protected, explicitly approved paths and artifacts within the applicable response clock. Hash allowlists may be an interim control; supported signature verification and resource budgets should be configured and tested independently when applicable. Neither waits for a complete PKI nor extends a finding's response horizon, and a valid signature does not make a buggy plugin resource-safe.

Recommendation CORE-S11: Coordinate supply-chain controls with parser hardening

Supply-chain work can proceed in parallel with parser hardening. Assign it an owner and scope independently, and record any concrete cross-track dependency rather than inferring order from an unscored comparison of the risk areas.

Minimum controls:

Table 21 – Minimum supply-chain controls

Control	Target
OpenSSF Scorecard	weekly scheduled run, alerts in security tab
CodeQL	required for PRs touching C/C++ parser surfaces
pinned GitHub Actions	no floating third-party action refs
SBOM	release artifacts include source and binary SBOMs
signed release artifacts	signatures plus checksums
SLSA provenance	release builds produce verifiable provenance

OpenSSF Scorecard has an official GitHub Action, and public repositories can use it without cost. The SLSA GitHub generator provides tooling for SLSA provenance on GitHub Actions, including provenance that users can verify to understand how an artifact was built.

Recommendation CORE-S12: Metrics that force the right behavior

Use metrics that measure class elimination, not activity.

Table 22 – Security program metrics

Metric	Target
CVE corpus pass rate under ASan/UBSan/LSan	100%
parser decode functions with explicit buffer size or cursor	100%
parser allocations through checked wrappers	100%

Table 22 – Security program metrics, continued

Metric	Target
parser memcpy/memmove through checked wrappers or reviewed exemption	100%
structures in the versioned declared tranche with invariant files	100%
structures in the versioned declared tranche with registered fuzz targets	100%
cataloged file-derived assertions classified for dominance, reachability, and consequence	100%
declared file-derived invariants enforced in <code>-DNDEBUG</code> builds	100%
new CVEs that add invariant updates	100%
time from new crash to minimized corpus seed	under 48 hours
time from minimized seed to invariant classification	under 5 business days
sanitizer-clean nightly fuzz runs	sustained
untrusted profile test coverage	every release

Track the versioned tranche denominator and results in a public dashboard. The dashboard does not fix defects; it makes declared coverage and gaps auditable.

3.5.3. Safety: Durability and Recovery

The security track above answers “a hostile file arrived.” The two recommendations here answer “my file did not survive.” They address `CORE-SAFE-001` and `CORE-SAFE-002`, and neither has a counterpart elsewhere in this report. The `CORE-SAFE-S` identifiers name recommendations only; their numbers express no risk, urgency, effort, dependency, or implementation order.

Both findings are routed Near-term for *containment* and Planned or later for *structural treatment*. The containment obligations are operational and available today: capacity headroom with free-space monitoring, flush and checkpoint discipline, a rehearsed recovery procedure, and an independent copy of anything that cannot be regenerated. Nothing below is a prerequisite for those.

Recommendation CORE-SAFE-S1: Give HDF5 an ENOSPC story

Today an application cannot learn that the virtual file driver hit ENOSPC, cannot choose how to respond, and cannot mark the resulting file as untrustworthy. Four candidate mechanisms have already been proposed in the public design discussion; they are complementary rather than alternatives, and they differ sharply in cost [20].

Table 23 – Candidate ENOSPC mechanisms

Mechanism	What it provides	Cost and caveat
VFD-level error callback	The caller is told the driver hit ENOSPC and chooses retry, fail fast, or panic. This is the load-bearing item: it converts a silent unsafe state into an application decision.	Small to medium. Deliverable as a pass-through VFD wrapper first, without a public API commitment.
Poisoned-file state	A file that suffered a write failure is durably marked suspect, so a later open cannot silently treat it as sound.	Medium; needs a format-visible flag and a defined clearing procedure. Pairs with <code>h5clear</code> .
Space reservation (<code>posix_fallocate</code>)	Capacity is claimed before the write, converting a mid-write failure into an up-front one.	Medium; filesystem-dependent, and not available or meaningful on every backing store.
Pre-flush capacity check (<code>statvfs</code>)	A watchdog refuses to begin a flush that plainly cannot complete.	Small, but advisory only — it narrows the window and does not close it. A check that passes does not guarantee the flush succeeds.

Sequence them by that cost profile: the callback and the pre-flush check first, because they are small and independently useful; reservation and the poisoned state after, because they touch the format and the VFD contract. The public design discussion proposes a pass-through VFD wrapper as a low-friction proving ground before any public API change, and this report endorses that route.

Acceptance evidence. Fault injection returns `ENOSPC` at each of write, flush, and close, for each supported VFD and buffering profile. For every injected point: the application receives a distinguishable `ENOSPC`-class error rather than a generic failure or a success; the last durable state is recorded; a subsequent open either succeeds, fails with a diagnostic naming the condition, or reports the file as suspect — never silently returns wrong data; and the documented recovery procedure is executed and its result recorded. A run in which the library reports success and the file is later found inconsistent is a failure of this recommendation regardless of the error codes returned.

Recommendation CORE-SAFE-S2: State, then improve, the crash-consistency guarantee

The immediate deliverable is not a mechanism. It is a written statement of what HDF5 guarantees today when a writer dies mid-update — for each of the default single-file layout, SWMR, and the split and multi drivers — and, explicitly, what it does not. Practitioners currently infer this from forum threads and folklore, which is why the same questions recur across sixteen years [16, 11, 15]. A documented weak guarantee is more useful than an undocumented one, because it lets an application decide whether it needs checkpointing, an external journal, or a different store.

The structural work is a multi-release programme and is already under investigation: metadata journaling, write-ahead logging, mechanisms derived from SWMR, and checkpointing [9]. This report does not select among them. It records three constraints on whichever is chosen.

1. **Recovery tooling is part of the guarantee, not an afterthought.** Whether a file reopens after `h5clear` is the difference between I2 and I3 in

CORE-SAFE-002. Improving crash consistency without improving diagnosis and repair moves files from one failure class to another rather than reducing harm.

2. **Silent wrong data is the outcome to design against.** A file that fails to open is a bounded operational problem. A file that opens and returns data from an inconsistent state is a scientific integrity problem, and it is the one that will not be noticed.
3. **The guarantee must be testable by fault injection,** and its test must run in the release build configuration, not only under assertions — the same discipline CORE-S7 applies to parser invariants.

Acceptance evidence. Published documentation states the crash-consistency guarantee per layout and driver, including its limits. A fault-injection harness kills the writer at a matrix of points across metadata update, flush, and close, and for every point records whether the file opens, opens after repair, or does not open, and whether any data read back differs from what was acknowledged as written. The last case is reported separately and treated as a defect, never as expected behaviour. Results are published per supported release so that the guarantee can be seen to hold or to change.

3.5.4. Privacy

This section summarizes the Privacy Exposure Model's mechanisms and the bounded CORE-PRIV-001 finding; the other mechanisms are not separately rated. It then describes corresponding mitigation strategies.

Overview of Privacy Threat Categories

The HDF5 Privacy Exposure Model [89] identifies seven exposure families that describe mechanisms through which sensitive information may be unintentionally exposed in HDF5. These mechanisms can arise during normal scientific data cre-

ation, processing, sharing, and preservation; their frequency, affected population, and consequence remain deployment-specific.

Several threat categories involve direct disclosure of information stored within HDF5 files. Semantic Metadata Exposure (P1) occurs when descriptive information, such as object names, attributes, datatype member names, or other user-defined metadata, reveals sensitive contextual information even when the underlying scientific data are protected. Structural or On-disk Leakage (P2) arises from examination of HDF5's internal storage structures, including file layout, storage organization, and implementation metadata, which may disclose characteristics of the stored data without requiring HDF5 software. Raw-data Pattern Leakage (P3) addresses situations in which information about scientific data becomes observable through storage patterns, embedded textual descriptions, or other implementation artifacts rather than through intended data access.

Another group of threats results from routine scientific data processing. Unintended Inclusion During Processing (P4) occurs when software automatically generates, expands, or preserves additional information during data conversion or processing. Aggregation, Correlation, or Inference (P5) recognizes that individually non-sensitive datasets may collectively reveal sensitive information when combined or interpreted together. Persistence Beyond Intended Lifetime (P6) addresses privacy risks associated with temporary files, metadata snapshots, cached datasets, archived copies, and other artifacts that remain accessible after their intended purpose has ended.

The remaining threat category Misplaced Trust in Privacy by Format (P7) reflects broader characteristics of modern HDF5 ecosystems. Examples include:

- Misinterpretation of data privacy in HDF5 format leads to the possibility of sensitive information disclosure through metadata, structural information, or inference.
- Conversion to HDF5 highlights that transforming legacy scientific formats into HDF5 often increases accessibility and interoperability by making implicit information explicit, but this additional semantic information may itself introduce new privacy risks.

- Loss of Data Control describes situations in which data movement, replication, backup, cloud migration, or transfer across organizational boundaries weakens the original privacy protections and governance mechanisms applied to the data.

Although presented as distinct categories, these mechanisms can occur together. A single scientific workflow may enrich metadata during format conversion, aggregate datasets from multiple sources, generate temporary processing artifacts, and replicate results across multiple storage systems. Consequently, privacy risks should be evaluated across the complete scientific data lifecycle rather than by considering individual HDF5 files in isolation.

Recommendations

Because HDF5 was designed to maximize scientific data sharing, portability, interoperability, and long-term usability rather than enforce privacy, no single mechanism can eliminate all privacy risks. Effective privacy protection therefore requires a multi-facet approach that considers the entire scientific data lifecycle, including data creation, processing, storage, sharing, transfer, and archival. Privacy should be treated as a property of the complete HDF5 ecosystem including applications, plugins, workflows, storage systems, and organizational practices rather than solely of the HDF5 file format. The following `CORE-PRIV-S` identifiers identify recommendations in this track. Their numbers do not express risk, urgency, effort, dependency, or implementation order.

For a confirmed `CORE-PRIV-001` sharing boundary, the Near-term route applies: `RM-1D` first bounds the population and recipients and tests minimization and recipient controls; `RM-2G` is conditional when policy still requires whole-file or equivalent all-metadata protection. `CORE-PRIV-S2` and extension controls support the unrated `EXT-PRIV-001` evidence and assurance track and inherit no urgency from `CORE-PRIV-001`.

Recommendation CORE-PRIV-S1: *Privacy exposure awareness*

Scientists, software developers, data stewards, and infrastructure providers should recognize that HDF5's self-describing nature inherently exposes contextual information in many ways. Privacy reviews should therefore evaluate not only scientific datasets, but also metadata, structural organization, workflow artifacts, derived products, and information that may be inferred through aggregation or correlation. Reviews should also take into consideration possible privacy exposure during data lifecycle.

Awareness is not itself a control, so this recommendation is written to produce an artifact rather than a disposition of mind.

Acceptance evidence. A dated privacy review record exists for the defined data population and sharing boundary, and it contains, at minimum:

- (a) the named data owner and the intended recipient set for each disclosure boundary;
- (b) an enumeration of sensitive items found in each asset class identified in Section 3 — data, user-defined metadata, structural metadata, derived or reconstructed information, and workflow artifacts — or a positive statement that the class was examined and found empty;
- (c) the lifecycle stages examined, with the stages not examined named rather than omitted; and
- (d) a disposition for every enumerated item: retain, minimize, tokenize, restrict recipients, or accept with a recorded rationale.

A review that examines datasets only, or that records no negative findings for the metadata and artifact classes, does not satisfy this recommendation.

Recommendation CORE-PRIV-S2: *Privacy exposure through HDF5 tools and plugins awareness*

The HDF5 ecosystem extends beyond the core HDF5 software (i.e., file format and the library), and includes a diverse collection of command-line utilities, visualization tools, analysis libraries, VOL connectors, VFDs, filters, and third-

party applications. These components can generate additional outputs such as logs, diagnostic messages, temporary files, metadata snapshots, converted datasets, and cached intermediate results that may expose sensitive information beyond the contents of the original HDF5 file. Furthermore, plugins may introduce new metadata, or transform data in ways that alter their privacy characteristics. Consequently, privacy assessments should evaluate not only HDF5 files but also the behavior of the software ecosystem that processes them. Developers should document the privacy implications of their tools, minimize unnecessary generation of sensitive artifacts, and provide users with mechanisms to control logging, metadata generation, and retention of intermediate products.

Acceptance evidence. For each in-scope tool, connector, filter, and VFD:

- (a) published documentation lists the outputs it can generate — stdout and stderr, redirected files, logs, temporary files, caches, snapshots, converted or derived files — and the option, property, or environment control that suppresses or bounds each one;
- (b) the list is confirmed by a reproducible trace of a representative invocation rather than asserted from the source, and the trace is retained; and
- (c) any output observed in the trace but absent from the documentation is recorded as a defect against that component.

Table 9 is the starting inventory for the official command-line tools, not a completed trace. This recommendation supports the unrated EXT-PRIV-001 evidence track and inherits no response urgency from CORE-PRIV-001.

Recommendation CORE-PRIV-S3: *Privacy-aware system design*

HDF5 software systems should be built with data privacy in mind. Applications should minimize collection and retention of unnecessary metadata, avoid embedding sensitive information in object names or descriptive attributes, remove temporary processing artifacts when they are no longer needed, and carefully evaluate automatically generated metadata introduced during data conversion or interoperability. Workflow designers should review intermediate products, logs, cached datasets, and derived files before publication or long-term archival.

Acceptance evidence. The claim to be tested is that minimization actually removed the information, not that it was removed from the API view:

- (a) before-and-after inspection of a representative file through *both* the HDF5 API and a raw-byte search — at minimum `strings`-class extraction over the whole file, including the user block and any free or unreferenced space — shows that each item marked *minimize* or *tokenize* in the `CORE-PRIV-S1` review is absent from both views;
- (b) metadata introduced automatically during conversion is enumerated for the tested pipeline and each item is dispositioned; and
- (c) temporary artifacts are removed under a stated retention rule, and the rule is verified by inspecting the working area after both normal and abnormal termination.

An item that disappears from `h5dump` output but is still recoverable from the raw bytes has not been minimized. Where the file is rewritten to achieve this, record pre- and post-checksums and preserve provenance.

Recommendation CORE-PRIV-S4: *Data encryption as privacy control*

Encryption represents an important but limited privacy control.

Currently, HDF5 doesn't support data encryption that allows I/O on encrypted data, effectively disabling sensitive data exposure during data transfer and storage. Users may implement custom raw data encryption and partial metadata encryption (i.e., names of the objects) using existing HDF5 filtering mechanism. Such approach still leaves user-defined metadata (e.g., attributes) and structural metadata unencrypted. Information that remains unencrypted as application-specific workflow artifacts, temporary files, system logs, or derived datasets may continue to disclose sensitive information.

HDF5 encryption feature that properly implements encryption of all data stored in HDF5 while allowing partial I/O on encrypted data is currently under development. However, encryption does not eliminate all HDF5 privacy risks. Once encrypted data are decrypted for legitimate processing, applications, plugins, caches, diagnostic logs, and intermediate workflow products may again become

sources of unintended exposure. Encryption therefore reduces the attack surface but does not address inference, aggregation, excessive metadata generation, data retention, or loss of governance. Encryption also complicates workflows and may lead to data loss if on some stage of data life cycle encryption keys become unavailable.

Therefore, encryption should be viewed as one component of a comprehensive privacy strategy rather than as a complete solution. Effective protection requires combining encryption with careful metadata management, lifecycle-aware data governance, privacy-aware application design, access controls provided by the underlying computing environment, and education of users regarding the unique privacy characteristics of HDF5-based scientific data systems.

Acceptance evidence. Encryption is credited only for the boundaries actually tested, and only when recoverability is demonstrated alongside confidentiality:

- (a) ciphertext and metadata inspection shows that the items the CORE-PRIV-S1 review marked as requiring protection are not observable at the protected boundary, tested through both the HDF5 API and raw-byte search;
- (b) an unauthorized-open test fails closed, and a partial-I/O workflow test confirms the protected file remains usable for its intended access pattern;
- (c) a protected key inventory names a key custodian per key, and backup/restore evidence exists; and
- (d) a loss-and-recovery exercise is performed and passes.

Item (d) is not optional. Encryption without a tested recovery path converts a disclosure risk into a durable data-loss risk, which is a different harm and not an improvement. Until all four pass, the affected sharing boundary is avoided or a tested access restriction is maintained within the CORE-PRIV-001 Near-term clock. This recommendation is the conditional branch RM-2G, reached only where RM-1D minimization and recipient controls cannot meet the disclosure policy.

4. HDF5 Library Extensions

4.1. Overview

Supply chain-related issues are discussed in more detail in [Section 7](#).

Taxonomy and rating boundary. The labels P1–P7 in this chapter use the exposure families defined by the HDF5 Privacy Exposure Model and summarized in the core chapter: P1 semantic metadata exposure, P2 structural or on-disk leakage, P3 raw-data pattern leakage, P4 unintended inclusion during processing, P5 aggregation, correlation, or inference, P6 persistence beyond intended lifetime, and P7 misplaced trust in privacy by format. EXT, OPS, TCD, and SCD are separate SSP category tags for extensions, operations, toolchains, and distribution; they are not additional P-numbered families. These labels classify possible exposure mechanisms, not inherent risk or response urgency. The chapter-wide observation remains the unrated EXT-PRIV-001 evidence record unless a bounded deployment supports a formal finding.

4.2. Virtual Object Layer (VOL) Connectors

4.2.1. Privacy-Relevant Mechanisms for VOL Connectors

HDF5 VOL connectors can substantially broaden a deployment’s privacy-assessment boundary. When a VOL connector (or a stack of VOL connectors) is used, data privacy can no longer be evaluated by examining only the HDF5 file, its lifecycle, and the workflows in which it is used. Instead, the privacy boundary extends to the entire data ecosystem, including all underlying data sources, connector-generated data and metadata, caches, authentication credentials, network communications, and the dynamically loaded plugin code itself. Consequently, any comprehensive HDF5 privacy model must account for these components, their interactions, and the trust relationships they establish.

The following mechanisms can create deployment-specific privacy exposure when HDF5 VOL connectors are used.

External Data Sources (P7; SSP: EXT)

Users traditionally assume that an HDF5 file contains all of the data being accessed and that inspecting or protecting the file is sufficient to understand and secure the complete dataset. However, this assumption no longer holds when HDF5 VOL connectors are used. A VOL connector can transparently retrieve and integrate data from numerous external sources while presenting it as a single HDF5 file. These sources may include cloud object stores, databases, remote servers accessed through REST or other network services, live data streams, or files in non-HDF5 formats. In addition, connectors may generate derived data, metadata, or virtual objects that do not physically exist in the underlying storage. As a result, users may be unaware of the true origin, location, ownership, and privacy policies governing the data being accessed, making it more difficult to assess privacy exposure and apply appropriate protections.

Transparent Data Aggregation (P5; SSP: EXT)

A VOL connector can transparently aggregate data from multiple HDF5 files, databases, cloud object stores, external file formats, and live data streams into a single logical HDF5 dataset. For example, a connector could combine data from a geospatial database, near real-time weather and natural disaster feeds, and property assessment records to produce an HDF5 dataset for analyzing the potential impact of natural disasters on property values. From the application's perspective, all of these heterogeneous data sources appear as a single HDF5 dataset, potentially obscuring the number, nature, and privacy policies of the underlying data sources. Such transparent aggregation can create unintended disclosure because data originating from independent sources with different ownership, access controls, or privacy policies may be analyzed or shared as a single integrated dataset without the user's explicit awareness.

Access to non-HDF5 data (P7; SSP: EXT)

VOL connectors can access data in other binary formats, for example TIFF/GeoTIFF, BUFR, GRIB2, and CDF, and special storage systems, for example, DAOS. Therefore, the privacy implications of these formats and storage systems must be included in the deployment's HDF5 privacy assessment.

Generated metadata (P1, P4; SSP: EXT)

VOL connectors may generate their own metadata that in turn may reveal, for example, dataset relationships, storage locations, URLs, cloud bucket names, and provenance. VOL connectors may generate additional metadata; for example, BUFR and GRIB2 VOL connectors generate file inventories revealing file organization, create derived attributes and dimension scales even when raw data remain protected.

Network Workflow Artifacts (P4, P6; SSP: EXT, OPS)

HDF5 VOL connectors that access remote data sources generate network workflow artifacts as part of normal operation. These artifacts may include HTTP request and response headers, DNS queries and logs, proxy logs, browser or client history, authentication records, local caches, temporary downloads, and other network-related metadata retained on local or remote systems. Such artifacts can create a detailed record of the datasets accessed, potentially exposing sensitive information even when the underlying data remain protected. Consequently, HDF5 VOL connectors that rely on network communications inherit privacy-relevant characteristics of networked systems and should be evaluated within a broader network privacy and security model.

Caching (P4, P6; SSP: EXT)

HDF5 VOL connectors may cache all sorts of data (metadata, downloaded raw data, authentication information, etc.) that remain after execution and may be shared, copied, and archived. A user may not be even aware of VOL's artifacts and leaked data.

Provenance leakage (P1, P4; SSP: EXT)

VOL connectors may reveal provenance information, e.g., data origin, timestamps, organizational structure, modification history.

Policy Bypass (P7; SSP: EXT, OPS)

HDF5 applications are generally privacy-policy agnostic and have no inherent awareness of the privacy policies governing the data they access. When VOL connectors are used, the same application may transparently access data originating from multiple sources, each subject to different ownership, access controls, retention requirements, or privacy regulations. Because the application interacts only with the HDF5 API, it may unknowingly combine, process, or redistribute data under incompatible privacy policies, effectively bypassing policy boundaries that would otherwise apply to the individual data sources.

Plugin Trust (P4, P6; SSP: EXT, SCD)

HDF5 VOL connectors are dynamically loaded plugins that execute external code within the application's process. As a result, a connector has direct access to the data flowing through the HDF5 API and can inspect, copy, modify, cache, export, or share that data before returning it to the application. A malicious or untrusted VOL connector could therefore compromise the confidentiality, integrity, or privacy of the data without the application's knowledge. Consequently, VOL connectors should be obtained from trusted sources, verified before deployment, and included within the application's overall trust and privacy model.

Table 24 – Candidate privacy-exposure families for VOL connector mechanisms

Mechanism	Candidate exposure families / SSP tags
External Data Sources	P7; SSP: EXT
Transparent Data Aggregation	P5; SSP: EXT
Access to non-HDF5 Data	P7; SSP: EXT
Generated Metadata	P1, P4; SSP: EXT
Network Workflow Artifacts	P4, P6; SSP: EXT, OPS

Table 24 – Candidate privacy-exposure families for VOL connector mechanisms, continued

Mechanism	Candidate exposure families / SSP tags
Caching	P4, P6; SSP: EXT
Provenance Leakage	P1, P4; SSP: EXT
Policy Bypass	P7; SSP: EXT, OPS
Plugin Trust	P4, P6; SSP: EXT, SCD

4.3. Filter Plugins

4.3.1. Privacy-Relevant Mechanisms for Filter Plugins

Filter plugins operate on data immediately before storage and immediately after retrieval. As a result, the privacy boundary extends beyond the HDF5 file itself and the application that uses the data. Filter plugins execute external code, may create temporary plaintext copies of sensitive data, generate intermediate artifacts, maintain internal state, or transmit information outside the application. Consequently, evaluating privacy exposure requires considering not only the stored HDF5 file but also the behavior of the filter plugins, their execution environment, and any artifacts they produce during data processing.

The following mechanisms can create deployment-specific privacy exposure when HDF5 filters are used.

Metadata leakage via filter identifiers (P1; SSP: EXT)

The filter pipeline is stored in the file as HDF5 metadata in the object header. Filter identifier and its parameters may reveal security mechanisms, organizational standards, software ecosystem, application domain, etc.

Data characteristics leakage via filter parameters (P1, P3; SSP: EXT)

Filter parameters such as compression level, block sizes, scale-offset parameters,

etc. may reveal expected precision, data resolution, floating-point accuracy, acceptable information loss. Usage of lossy filters shows acceptable data approximation, expected error bounds, and precision requirements. Such data characteristics can give insights into assumptions under which data was collected and characteristics of experimental setup.

Sizes of compressed chunks reveal data characteristics (P2, P3; SSP: EXT)

Chunk sizes are metadata and can be examined without access to data. Different sizes of compressed chunks may reveal sparsity, repeated patterns, data randomness, and empty regions to name a few.

Filters operate on plaintext data buffers (P7; SSP: EXT)

Before data compression or encryption is applied filters have access to data. Malicious filter may have access to data before specialized compression or encryption are applied to protect data.

Filters may persist data outside HDF5 (P4, P6; SSP: EXT)

Filters may allocate memory buffers that may remain in memory, or paged to disk, appear in crash dumps. This sensitive data may persist outside the HDF5 file.

Plugin trust (P4, P6; SSP: EXT, SCD)

As HDF5 VOL connectors, HDF5 filters are dynamically loaded plugins that execute external code within the application's process. If the filter is malicious, it can do anything before passing data to application or to storage, for example, inspect memory, write logs, copy decoded data and send it elsewhere or retain it. Untrusted filter plugin could compromise data confidentiality, integrity and privacy without application's knowledge.

Plugin search path (P7; SSP: EXT, SCD)

Dynamic filters are loaded from directories specified by environment variables

while the HDF5 library is searching for the requested plugin. Such process may result in loading a malicious filter. On different systems application may load different plugins. Thus, it becomes a supply-chain problem.

Filters may generate derived information (P5; SSP: EXT). Since filters have access to data they can generate and store any type of derived information, e.g., statistics, patterns, sparsity characteristics, etc. The derived information may reveal sensitive information about data.

The table below provides primary privacy mechanism for built-in filters and several popular third-party filters.

Table 25 – Primary privacy mechanisms associated with HDF5 filter plugins

HDF5 Filter	Exposure Families / SSP Tags	Representative Privacy Considerations
Deflate (GZIP)	P1 Semantic Metadata Exposure P3 Raw-data Pattern Leakage	Compression method, compression level, and resulting chunk sizes may reveal data entropy, sparsity, and redundancy.
Shuffle	P3 Raw-data Pattern Leakage	Byte reordering itself is reversible, but its use together with compression may reveal characteristics of the underlying numeric representation and data organization.
Fletcher32	P1 Semantic Metadata Exposure	Checksum metadata reveals integrity protection is used but generally introduces minimal additional privacy risk.
N-bit	P1 Semantic Metadata Exposure P3 Raw-data Pattern Leakage	Bit-width reduction reveals effective numeric precision and expected value ranges, providing insight into experimental accuracy and measurement characteristics.
Scale-Offset	P1 Semantic Metadata Exposure P3 Raw-data Pattern Leakage	Scale factors and offsets reveal expected precision, acceptable quantization error, and data resolution.

Table 25 – Primary privacy mechanisms associated with HDF5 filter plugins, continued

HDF5 Filter	Exposure Families / SSP Tags	Representative Privacy Considerations
ZFP	P1 Semantic Metadata Exposure P3 Raw-data Pattern Leakage	Compression mode (fixed rate, fixed precision, or fixed accuracy) and associated parameters reveal acceptable information loss and numerical accuracy requirements.
SZ / SZ3	P1 Semantic Metadata Exposure P3 Raw-data Pattern Leakage	Error bounds expose tolerated approximation error and characteristics of scientific datasets.
Blosc	P1 Semantic Metadata Exposure P3 Raw-data Pattern Leakage	Compression algorithm, block size, and shuffle/bitshuffle options reveal storage optimization choices and data organization.
Zstandard (Zstd)	P1 Semantic Metadata Exposure P3 Raw-data Pattern Leakage	Compression level and resulting compressed sizes may reveal entropy and structural properties of datasets.
LZ4	P1 Semantic Metadata Exposure P3 Raw-data Pattern Leakage	Fast compression with modest ratios may reveal data compressibility through chunk sizes.
BZIP2	P1 Semantic Metadata Exposure P3 Raw-data Pattern Leakage	Compression characteristics and block sizes may expose redundancy and data structure.
All Dynamic Filters	P4 Unintended Inclusion P6 Persistence SSP: EXT, SCD	Filters operate on plaintext data before writing or after reading. Dynamically loaded plugins may inspect, cache, copy, or export sensitive data and may generate temporary buffers, logs, or crash artifacts.

4.4. Virtual File Drivers (VFD)

4.4.1. Privacy-Relevant Mechanisms for VFDs

HDF5 VFDs manage how actual bytes are stored and retrieved. While VFDs perform I/O and do not change the logical HDF5 file and data model, they can create privacy-relevant exposure through data movement, storage, and application I/O characteristics.

The following mechanisms can create deployment-specific privacy exposure when HDF5 VFDs are used.

External Data Sources (P7; SSP: EXT)

Unlike traditional file I/O, Virtual File Drivers may transparently store and retrieve HDF5 data from cloud object stores, distributed file systems, network storage, memory-resident files, or multiple physical storage devices. Consequently, users and system administrators may incorrectly assume where sensitive information resides and which security policies, regulatory requirements, or administrative controls apply. The privacy boundary therefore extends beyond the HDF5 file to the entire storage ecosystem.

Multiple Copies of Data (P6; SSP: EXT) Some VFDs create additional copies of HDF5 data without the application's explicit awareness. These copies may exist in replicated files, metadata journals, backing stores, memory dumps, swap files, or mirrored storage systems. Because these copies may persist beyond the lifetime of the primary HDF5 file, sensitive information can remain recoverable even after the original file has been deleted or relocated.

Storage Metadata Exposure (P2; SSP: EXT)

Although VFDs do not understand HDF5 objects, they manage physical storage and therefore may expose low-level storage metadata, including physical offsets, allocation sizes, storage layouts, and file organization. Such information can

reveal characteristics of datasets, storage architecture, and application behavior even without accessing the logical HDF5 objects.

Access Pattern Leakage (P3; SSP: EXT)

Storage activity observed by the operating system, distributed file system, storage servers, or cloud infrastructure may reveal application behavior. Access frequency, read/write ratios, sequential versus random I/O, update frequency, and accessed file regions can reveal properties of the underlying datasets, application algorithms, or user activity. These observations enable inference of information that is not explicitly stored within the HDF5 file.

Workflow Artifacts (P4, P6; SSP: EXT)

VFDs may generate artifacts during normal operation, including metadata journals, temporary files, swap files, memory images, crash dumps, replicated storage objects, and cloud access logs. These artifacts can persist outside the HDF5 file itself and may retain sensitive information or evidence of application activity.

Plugin Trust (P4, P6; SSP: EXT, SCD)

Dynamically loadable Virtual File Drivers execute external code within the application’s process. A malicious or compromised VFD can inspect, modify, retain, duplicate, or export data before writing it to storage or returning it to the application. Consequently, VFDs should be obtained only from trusted sources and considered part of the application’s overall trust and privacy model.

Table 26 – Candidate privacy-exposure families for HDF5 Virtual File Drivers

HDF5 VFD	Privacy Exposure families / SSP tags	Representative privacy considerations
SEC2	P6 Persistence	Disk blocks allocated for new HDF5 files may contain residual information if the underlying storage is not securely initialized or overwritten.
STDIO	P6 Persistence	Similar to the SEC2 driver, persistent storage may expose residual data remaining on reused disk blocks.

Table 26 – Candidate privacy-exposure families for HDF5 Virtual File Drivers, continued

HDF5 VFD	Privacy Exposure families / SSP tags	Representative privacy considerations
Core	P6 Persistence	Stores the entire HDF5 file in memory. Sensitive information may persist in process memory, swap space, or operating-system crash dumps. When the backing-store option is enabled, the in-memory file is written to persistent storage upon closing.
Family	P6 Persistence	Splits a logical HDF5 file into multiple physical files. Misconfigured permissions or incomplete deletion of family members may expose portions of the dataset.
Split	P2 Structural Leakage P6 Persistence P7 Misplaced Trust; SSP: EXT	Stores metadata and raw data in separate files, which may have different permissions or reside on different storage systems. The driver also creates multiple physical copies of file information.
Multi	P2 Structural Leakage P6 Persistence P7 Misplaced Trust; SSP: EXT	Stores different HDF5 memory types in separate files. Independent storage locations and permissions increase the risk of metadata and raw data disclosure.
Mirror	P6 Persistence P7 Misplaced Trust; SSP: EXT, SCD	Replicates HDF5 data to a remote mirror service. Privacy depends on secure communication, trusted mirror daemons, authentication, and protection of replicated copies.
Log SWMR) (VFD	P2 Structural Leakage P6 Persistence; SSP: EXT	Metadata journaling creates persistent journal files containing metadata snapshots for crash recovery, leaving additional workflow artifacts outside the HDF5 file.
MPI-I/O	P7 Misplaced Trust; SSP: EXT, OPS	Transfers metadata and raw data through the underlying HPC communication infrastructure. Unless protected by lower-layer mechanisms, network traffic may expose sensitive information.
Subfiling	P2 Structural Leakage P6 Persistence P7 Misplaced Trust; SSP: EXT	Divides a logical HDF5 file into multiple subfiles distributed across storage devices. Fragmentation and independently managed files increase the risk of unauthorized disclosure through misconfigured access controls.

Table 26 – Candidate privacy-exposure families for HDF5 Virtual File Drivers, continued

HDF5 VFD	Privacy Exposure families / SSP tags	Representative privacy considerations
Onion	P2 Structural Leakage	Maintains historical versions of HDF5 files, allowing recovery of deleted or overwritten data and reconstruction of meta-data evolution, provenance, and workflow history.
	P5 Inference	
	P6 Persistence	
ROS3	P4 Unintended Inclusion	Accesses HDF5 files stored in S3-compatible object stores. HTTP requests, DNS queries, cloud access logs, and HTTP Range requests create network artifacts and extend the privacy boundary to cloud infrastructure.
	P6 Persistence	
	P7 Misplaced Trust; SSP: EXT, OPS	

4.5. Architectural Observations and Mitigation Planning

The general privacy-boundary observation in this section is recorded as evidence-backlog item EXT-PRIV-001 in Appendix A. It is not a formal finding because the report has not defined a deployment-specific sensitive-data population, recipient, retention condition, impact, or exposure. The recommendations below remain valid control proposals; their presence does not silently convert the observation into a rated finding.

Priority reconciliation.

No report-wide risk urgency is assigned to EXT-PRIV-001. Phase 2 schedules RM-2E to establish a bounded deployment, data flow, recipient, retention condition, and consequence; that administrative assurance target is not a Low-risk conclusion or a risk-response clock. Documentation, introspection, and plugin-trust controls in RM-3D are proactive assurance and do not by themselves establish residual-risk reduction; privacy-specific treatment in RM-3D and RM-3F follows the resulting evidence. A deployment matching the separately rated plugin-search scenario APP-SEC-005 instead follows its Near-term application-treatment clock; that finding does not assign a High tier to extensions as a class.

4.5.1. Privacy

The architectural analysis identifies ways in which privacy boundaries may extend beyond the HDF5 file itself. VOL connectors, filter plugins, and Virtual File Drivers can add external data sources, dynamically loaded plugins, storage systems, cloud services, and network infrastructure. Depending on a concrete deployment, these mechanisms may expose raw data, metadata, derived information, workflow artifacts, or information about the surrounding software ecosystem, even when the HDF5 file itself is adequately protected. The following sections summarize the principal observations for each extension mechanism and present mitigation recommendations to improve privacy awareness, transparency, and trustworthiness within the HDF5 ecosystem.

Privacy – HDF5 VOL Connectors

Architectural observations

The analysis identifies VOL connectors as a particularly broad potential extension of the privacy boundary. Unlike the HDF5 core library, a VOL connector may reach external data sources, cloud services, databases, network infrastructure, authentication systems, caches, and dynamically loaded plugins. Because applications interact only with the HDF5 API, users may be unaware of the true origin of data, the privacy policies governing those data, or the workflow artifacts generated during processing. Consequently, deployment-specific privacy exposure can arise not only from the HDF5 file itself but also from the broader software ecosystem accessed by the connector; the resulting harm and risk depend on the data, recipients, and controls in that deployment.

Recommendations

The following VOL identifiers identify recommendations in this track. Their numbers do not express risk, urgency, effort, dependency, or implementation order:

Recommendation VOL-S1 - Document connector privacy characteristics

Each VOL connector should explicitly document any of applicable items listed below:

- External data sources VOL connector accesses
- Generated metadata
- Caching mechanism
- Network communications
- Authenticaion mechanism
- Workflow artifacts
- Privacy assumptions

Recommendation VOL-S2 - Improve connector transparency and introspection

HDF5 should provide a standardized mechanism through which applications and audit tools can identify the active VOL connector or connector stack and query privacy-relevant operational characteristics. Existing VOL introspection callbacks can report connector identity and selected capabilities, but they do not expose whether a connector accesses external resources, aggregates multiple sources, generates metadata, maintains caches, creates workflow artifacts, or performs network communication. The VOL introspection interface, connector capability flags, or standardized optional operations should therefore be extended to expose this information.

Possible standardized queries could include:

- Connector name, version, and stack composition
- Types and locations of external data sources
- Whether
 - network access is performed

- raw data, metadata, or credentials are cached
- the connector creates temporary or persistent artifacts
- the connector generates metadata or derived data
- the connector combines data governed by different policies
- privacy-relevant configuration information can be obtained

Recommendation VOL-S3 - Strengthen connector trust and deployment practices

VOL connectors are dynamically loaded shared libraries and therefore become part of the application's trusted computing base. Organizations should deploy only trusted connectors, restrict the directories searched for plugins, verify the provenance and integrity of connector libraries, and avoid loading untrusted plugins from user-writable or publicly accessible locations.

Future versions of HDF5 could further improve security by providing configurable policies that restrict connector loading based on connector identity, implemented transparency and introspection, or administrative policy. Where signed-plugin verification is used, users and application developers should deploy HDF5 2.2.0 or a later verified release, build it with `HDF5_REQUIRE_SIGNED_PLUGINS=ON`, configure and protect the trusted keystore, sign approved connector artifacts, and test rejection of unsigned or untrusted plugins; upgrading alone does not enable the control (Table 49).

Recommendation VOL-S4 - Follow privacy-aware connector development guidelines

Future connector implementations should minimize unnecessary metadata generation, avoid persistent caches unless explicitly requested, and clearly document connector's privacy characteristics.

Privacy – HDF5 Filter Plugins

Architectural observations

The analysis shows that HDF5 filters can alter a deployment's privacy characteristics because they execute immediately before storage and immediately after retrieval while operating directly on plaintext data. Filter parameters, compression characteristics, and generated metadata may reveal information about the underlying datasets even when the data themselves remain protected. Dynamically loaded filter plugins also introduce a security-relevant supply-chain mechanism because they execute arbitrary code within the application's process and may generate temporary artifacts outside the HDF5 file.

Recommendations

The following FILTER identifiers identify recommendations in this track. Their numbers do not express risk, urgency, effort, dependency, or implementation order:

Recommendation FILTER-S1 - Strengthen filter trust and deployment practices

Organizations should deploy only trusted connectors, restrict the directories searched for plugins, verify the provenance and integrity of connector libraries, and avoid loading untrusted plugins from user-writable or publicly accessible locations.

Future versions of HDF5 could further improve security by providing configurable policies that restrict connector loading based on connector identity, implemented transparency and introspection, or administrative policy. Where signed-plugin verification is used, users and application developers should deploy HDF5 2.2.0 or a later verified release, build it with `HDF5_REQUIRE_SIGNED_PLUGINS=ON`, configure and protect the trusted keystore, sign approved filter artifacts, and test rejection of unsigned or untrusted plugins; upgrading alone does not enable the control (Table 49).

Recommendation FILTER-S2 - Select and document filter configurations with privacy in mind

Filter identifiers and parameters are stored in HDF5 object headers and are required for correct interpretation of the stored data. Consequently, this metadata cannot generally be removed or obscured without affecting interoperability. Users should evaluate whether filter configurations reveal sensitive information about data precision, compression characteristics, or scientific workflows, and document these privacy implications when selecting filter pipelines.

Recommendation FILTER-S3 - Use encryption when filter metadata are privacy sensitive

When classification shows that filter metadata must be hidden from the file recipient, whole-file protection is a direct file-at-rest confidentiality control. A solution that protects both raw data and HDF5 metadata can prevent that recipient from inspecting filter identifiers, parameters, object headers, and structural metadata. Selection is deployment-specific, requires the key management and recovery evidence in RM-2G, and does not treat caches, logs, authorized misuse, retention, inference after decryption, or key-loss risk.

Recommendation FILTER-S4 - Minimize workflow artifacts

Filter implementations should securely erase temporary buffers when possible, avoid persistent temporary files, and clearly document any generated artifacts.

Recommendation FILTER-S5 - Standardize privacy documentation

Built-in and registered filters should document:

- Generated metadata
- Plaintext processing
- Derived information
- Temporary artifacts
- Privacy implications of filter parameters

Privacy – HDF5 Virtual File Drivers

Architectural observations

The analysis shows that Virtual File Drivers can affect a deployment's privacy boundary through physical storage management rather than the logical HDF5 data model. VFDs influence where data are stored, how they are replicated, transmitted, cached, or versioned, and consequently extend the HDF5 privacy boundary to operating systems, distributed storage systems, cloud infrastructure, and network services. Different VFDs emphasize different privacy concerns, including persistent copies of data, storage metadata exposure, historical version retention, network artifacts, and cloud-based workflow exposure.

Recommendations

The following VFD identifiers identify recommendations in this track. Their numbers do not express risk, urgency, effort, dependency, or implementation order:

Recommendation VFD-S1 - Document storage behavior

Every VFD should clearly describe:

- Where data are stored
- Whether additional copies are created
- Whether network communication occurs
- Generated workflow artifacts
- Storage assumptions

Recommendation VFD-S2 - Minimize unintended persistence

Drivers that create replicated files, journal files, historical versions, or backing stores should provide mechanisms for secure deletion, retention management, and lifecycle control.

Recommendation VFD-S3 - Strengthen cloud and distributed-storage guidance

Cloud-based and distributed-storage VFDs should document authentication requirements, network communications, logging behavior, and storage-provider privacy assumptions.

Recommendation VFD-S4 - Strengthen VFD trust and deployment practices

Organizations should deploy only trusted VFD plugins, restrict the directories searched for plugins, verify the provenance and integrity of connector libraries, and avoid loading untrusted plugins from user-writable or publicly accessible locations.

Future versions of HDF5 could further improve security by providing configurable policies that restrict VFD plugin loading based on VFD's identity, implemented transparency and introspection, or administrative policy. Where signed-plugin verification is used, users and application developers should deploy HDF5 2.2.0 or a later verified release, build it with `HDF5_REQUIRE_SIGNED_PLUGINS=ON`, configure and protect the trusted keystore, sign approved VFD artifacts, and test rejection of unsigned or untrusted plugins; upgrading alone does not enable the control (Table 49).

Summary

The analysis shows that HDF5 library extensions can alter a deployment's privacy characteristics. While the HDF5 core library primarily exposes information through the file format itself, VOL connectors, filter plugins, and Virtual File Drivers can extend the privacy boundary to external software, storage systems, network infrastructure, cloud services, and dynamically loaded plugins. The existence and magnitude of harm remain deployment-specific. Effective privacy assessment should therefore evaluate both HDF5 files and the execution environment in which HDF5 operates. This broader ecosystem perspective should guide the design, implementation, deployment, and review of future HDF5 extensions.

5. HDF5 Wrappers and Language Bindings

5.1. Overview

Vocabulary. This chapter uses the issue families defined by the SSP SIG’s HDF5 Python Wrapper (H5PW) model [44]. Although the model is written for Python, its families generalise to any binding that mediates between file content and a managed runtime: **W1** native parser or codec compromise, **W2** code in data, **W3** dynamic loader boundary failure, **W4** concurrency and lifecycle mismatch, **W5** resource exhaustion and amplification, **W6** ABI, dependency, or packaging drift, **W7** metadata, log, and artifact exposure, **W8** trust-boundary confusion, **W9** misleading safety or security signal, and **W10** external reference traversal. These labels classify mechanisms and carry no risk rating.

Bindings for C#, Java, Python, JavaScript, Rust, and other languages mediate between HDF5’s C interfaces and language-specific object, memory, exception, packaging, and deployment models. This chapter uses the Python-facing cases below only as boundary references. It does not claim exhaustive coverage of those binding families or establish that a condition observed in a downstream framework is caused by the binding through which HDF5 was reached.

5.2. Python

Four advisories in the assessment window reached users through a Python-facing workflow, and all four are useful precisely because none of them is a binding defect. In each case h5py or an equivalent layer performed as specified, and the failure occurred above it:

CVE-2025-9905 [68].

Keras legacy `.h5` loading executed a Lambda layer’s serialized code despite `safe_mode=True`. The unsafe step is application-level deserialization of a framework object, not HDF5 decode. Rated as APP-SEC-002. H5PW **W2** (code in data) compounded by **W9** (misleading safety signal): the model describes **W9** as a `safe_mode` or scanner label that “implies a boundary that

is not enforced for the active format or loader path,” which is this CVE stated as a family.

CVE-2026-1669 [70].

Model loading followed an HDF5 external dataset reference to a local path and disclosed its contents. External raw storage is a documented HDF5 capability; the missing control is a policy on whether the loader may follow it. Rated as APP-SEC-003. H5PW **W10**, external reference traversal.

CVE-2026-0897 [69].

Weight loading allocated from a file-declared shape without a budget. HDF5 metadata may legitimately declare a logical shape far larger than the useful payload, so the budget must exist in the consumer. Rated as APP-SEC-004. H5PW **W5**, resource exhaustion and amplification.

CVE-2026-2492 [59].

TensorFlow’s HDF5 integration searched an unsecured location for plugins. The mechanism is HDF5 dynamic plugin loading; the defect is the search-path policy of the embedding application. Rated as APP-SEC-005. H5PW **W3**, dynamic loader boundary failure.

The pattern is the chapter’s substantive point, and it is H5PW **W8**, trust-boundary confusion: a binding faithfully exposes HDF5 capability, and the security decision — whether this process, holding this authority, should honor that capability for this file — belongs to the layer above. That all five application-boundary findings in this report land on named W-families is worth noting in itself: the vocabulary anticipated them, and this report arrived at the same boundary independently rather than by applying it. It is also why APP-S1 asks bindings to surface these controls in the binding API rather than leaving them reachable only through environment variables.

5.3. Java

No Java-specific adverse-event scenario was sufficiently developed for a formal finding in this revision. Java remains within the ecosystem census and control-

exposure scope described below. The H5PW families apply to the JNI boundary as directly as to the CPython one — **W1**, **W3**, **W4**, and **W6** in particular — and a Java-facing review using that vocabulary is the obvious next increment for this chapter.

5.4. Audit Findings and Mitigation Planning

This chapter does not establish an additional binding-specific formal finding. The listed Keras and TensorFlow cases are rated at their application boundaries as APP-SEC-002 through APP-SEC-005 in Appendix A; their presence in a Python-facing workflow does not make them h5py or Python-binding findings without separate evidence.

Priority reconciliation.

Wrappers and bindings therefore have no separate report-wide urgency in this revision. They participate in the Phase 1 version, linkage, feature, and authority census and should expose the controls required by the applicable application package. When a binding is part of a deployment matching a formal finding, that finding's response clock controls; the chapter in which a CVE is mentioned does not create a second priority.

6. Applications using HDF5

Applications treat HDF5 as inert data, but HDF5 files can carry behavior-relevant metadata: paths, external references, huge declared shapes, filter/plugin requirements, serialized application objects, and model-layer definitions. That creates integration bugs even when the HDF5 library is behaving as designed.

The HDF Group's security policy [90] already attempts the right scope distinction: core libhdf5, official tools, Remote Code Execution (RCE) via parsing/API abuse, and supply-chain compromise are in scope; third-party plugins and application-level usage are usually triaged or referred to their maintainers.

Table 27 – Representative application-level HDF5 failure mechanisms

Family	Typical failure	Examples	Application-boundary mechanism
malformed file metadata reaches C memory code	buffer overflow, over-read, UAF, double-free, null dereference, divide-by-zero	many HDFGroup records, including 2024 and 2025 issues in object headers, datatypes, dataspaces, filters, heaps, free-space metadata, and cache images	HDF5 is a complex binary metadata graph, not a flat byte container
application deserialization layered on HDF5	arbitrary code execution	Keras CVE-2025-9905: crafted <code>.h5/.hdf5</code> model archives can execute code because legacy <code>.h5</code> loading did not honor <code>safe_mode=True</code> for Lambda-layer pickled code [68, 63].	ML frameworks often use HDF5 as a model container, then deserialize framework-level objects from it.
external references and file path indirection	arbitrary local file read, path traversal, data exfiltration	Keras CVE-2026-1669: crafted <code>.keras</code> model file using HDF5 external dataset references could read local files [70, 64].	HDF5 supports links and external storage mechanisms. External links store both target filename and object path; external storage can resolve raw-data files through prefixes and current working directory rules [88].
unbounded resource allocation	memory exhaustion / interpreter crash	Keras CVE-2026-0897: crafted <code>.keras</code> archive with <code>model.weights.h5</code> declaring an extremely large dataset shape could exhaust memory and crash Python [69, 62].	HDF5 metadata can declare logical shapes much larger than the actual useful payload; application code may allocate based on declared shape before policy checks.
dynamic plugin loading and search paths	local privilege escalation / arbitrary code loading	TensorFlow CVE-2026-2492: HDF5 plugin handling loaded plugins from an unsecured location, enabling local privilege escalation [59, 92, 79].	HDF5 supports dynamic plugin loading, and plugin paths can be affected by defaults or <code>HDF5_PLUGIN_PATH</code> ; dynamic loading can also be disabled via <code>H5PLset_loading_state()</code> or <code>HDF5_PLUGIN_PRELOAD=:</code> [81].

Table 27 – Representative application-level HDF5 failure mechanisms, continued

Family	Typical failure	Examples	Application-boundary mechanism
vulnerable or stale embedded HDF5 copies	inherited CVEs, unpatched parser behavior	CVE-2021-36977 involved a heap overflow related to use of HDF5 1.12.0; Apple CVE-2021-31009 says multiple issues were addressed by removing HDF5 [27, 6].	Downstream products vendor, statically link, or lag HDF5 releases; vulnerability scanners then attribute HDF5 defects to the application.
helper-tool exposure	crashes or code execution through inspection/conversion utilities	h5dump appears in older and newer issues; CVE-2026-34734 describes a heap-use-after-free in the h5dump helper utility triggered by a malicious .h5 file [28, 85].	Applications often call h5dump, h5repack, or converters as “safe inspection” tools on untrusted files.

6.1. Boundary Case Study: External Raw Storage and the Hugging Face Incident

The July 2026 Hugging Face incident is a concrete example of an application trust-boundary failure involving an intentional HDF5 capability. Hugging Face reports that its dataset service processed attacker-supplied, valid HDF5 files whose datasets declared local paths as external raw storage. The worker read `/proc/self/environ` and source files with its own filesystem permissions and returned the bytes as dataset rows. Hugging Face expressly distinguishes this file-disclosure vector, which executed no code, from a separate Jinja2 injection that supplied code execution [67].

The HDF Group likewise concludes that external raw storage was the mechanism, not an HDF5 parser vulnerability: the documented feature behaved as designed, while the authority-bearing service acted as a confused deputy. Updating `libhdf5` alone would not remove that service boundary; the relevant controls are fail-closed external-resource policy, preflight inspection, path confinement, least-privileged workers, and credential isolation [91].

The `h5py` community expressly considered the same boundary. Maintainers described per-dataset external-storage checks as possible but laborious, recommended sandboxing genuinely untrusted files, opened upstream issue #6618 because `h5py` had no direct enforcement mechanism, and discussed a simpler detection API. Issue #6618 in turn asks for unified controls at or before external-resource resolution across raw storage, external links, and virtual-dataset sources.

The discussion records policy and API proposals, including use of the external `h5policy` prototype as a proving ground; this report does not treat those proposals as delivered controls [25, 78]. Formal finding APP-SEC-001 in Appendix A rates the incident at the affected service boundary. The similar Keras external-dataset failure is separately registered as APP-SEC-003; recurrence of a mechanism does not erase the differences in product, version, and deployment scope.

The unifying root cause: The repeated mistake is *trust-boundary confusion*. The term is not coined here. It is already a named family in two SSP SIG models — **I9** in the independent-implementation model and **W8** in the Python wrapper model — each defined as users treating HDF5 as inert data when the layer they are using can in fact trigger complex parsing, codec execution, external access, or object reconstruction [42, 44]. This report reached the same conclusion from the incident evidence rather than by applying the models, which is worth recording as corroboration: two independent routes to the same root cause is a stronger result than either alone. HDF5 sits at the boundary between a file and the process. Downstream frameworks often add another boundary: model loading, dataset ingestion, scientific workflow execution, browser upload, cloud service processing, or plugin resolution. Many CVEs happen because those layers collapse into one assumption: “This is just an HDF5 file, so reading it is safe.” That assumption is false. An HDF5 file can influence:

- memory allocation size
- dataset rank and shape
- chunk count and cache pressure
- filter/plugin loading
- external file lookup
- link traversal
- datatype conversion
- application-level model reconstruction

- tool output rendering

The common issue is **unsafe interpretation of HDF5-controlled metadata at the application boundary**.

6.2. Audit Findings and Mitigation Planning

The formal application findings are APP-SEC-001 through APP-SEC-005 in Appendix A. Their ratings apply to the bounded versions and deployments stated there, not to Keras, TensorFlow, Hugging Face, or applications using HDF5 as undifferentiated projects.

Priority reconciliation.

Any equivalent deployment that still gives untrusted HDF5 input automatic access to file-selected external resources follows the Immediate route for APP-SEC-001: contain it before the next job and no later than 24 hours after identification. Confirmed affected scope for APP-SEC-002, APP-SEC-003, and APP-SEC-005 receives Near-term treatment; APP-SEC-004 receives Planned treatment. The historical Hugging Face rating is not a current residual-risk rating of that service. Application patches and compensating controls in Phase 1 must not wait for the reusable core APIs and integrations proposed for Phase 2.

Mitigation has to be layered. Patching `libhdf5` alone will not fix Keras model deserialization, TensorFlow plugin path handling, or application-level external-reference policy.

The APP-S identifiers below identify recommendations in the downstream- application track. Their numbers do not express risk, urgency, effort, dependency, or implementation order.

Recommendation APP-S1: Define an “untrusted HDF5” profile

HDF5 should offer, and downstreams should use, a strict profile for any file received from users, networks, archives, model hubs, object stores, email, web uploads, or third-party datasets.

Default behavior for that profile:

```
dynamic plugins disabled
external links disabled or sandboxed
external raw dataset storage disabled or sandboxed
VDS source traversal restricted
maximum rank enforced
maximum element count enforced
maximum allocation size enforced
maximum metadata bytes enforced
maximum chunk count enforced
maximum filter expansion ratio enforced
cycles and deep graph traversal rejected
unknown or legacy-dangerous messages rejected
    unless explicitly allowed
```

Today, pieces exist: plugin loading can be controlled, and all dynamic plugins can be disabled with `H5PLset_loading_state(0)` or `HDF5_PLUGIN_PRELOAD=: :`. But this needs to become a cohesive security profile, not a set of hidden knobs.

Recommendation APP-S2: Ship a preflight validator

Prior art: a working preflight validator already exists; see Table 10. The open work is the enforcement coupling described below, not the validator itself.

Create a validator that reads metadata before the application allocates large arrays or deserializes model objects. Preflight is not a stand-alone authorization: its result must be bound to the same open handle or verified immutable file identity

used by runtime loading, and runtime hooks must fail closed if the file, policy, or external-resource target changes.

Example output:

```
{
  "eligible_under_profile": false,
  "reasons": [
    "external dataset reference outside sandbox",
    "dataset declares 8.7 TB logical size",
    "requires dynamic filter plugin",
    "contains legacy Keras Lambda layer metadata"
  ]
}
```

For core HDF5, this validator should reason about file-format invariants. For downstreams, it should expose application hooks:

```
on_external_reference(path)
on_dataset_shape(path, dtype, shape, logical_bytes)
on_filter_required(filter_id)
on_link_traversal(source, target)
on_attribute(name, size)
```

Keras and TensorFlow could then enforce model-specific policy before creating Python objects.

Recommendation APP-S3: Disable dangerous HDF5 features by default in ML model loading

ML loaders should not behave like general HDF5 viewers.

For Keras/TensorFlow-style use:

```
do not allow Lambda or pickled code from legacy .h5
do not allow external dataset references
```

- do not allow dynamic HDF5 plugins
- do not eagerly allocate declared tensors before checking total byte budget
- do not resolve paths relative to current working directory
- do not honor HDF5_PLUGIN_PATH in model loading contexts

These loader defaults directly address APP-SEC-002 through APP-SEC-004. The plugin clauses provide defense in depth for APP-SEC-005; its primary linked controls are APP-S5 and CORE-S10.

Recommendation APP-S4: Make external references opt-in

External links and external raw dataset storage are legitimate HDF5 features, but they are unsafe defaults for hostile content. HDF5 documentation states that external links store a target filename and object path, and external dataset storage search behavior can involve the current working directory, absolute paths, relative paths, `${ORIGIN}`, and `HDF5_EXTFILE_PREFIX`.

Mitigation:

- reject absolute external paths
- reject path traversal
- reject current-working-directory lookup
- require explicit sandbox root
- require explicit allowlist by extension or path
- log every external resolution
- never follow external references during metadata preview

For services, the safest default is: `externalreferences=off`.

Recommendation APP-S5: Treat plugins as executable code

HDF5 filter plugins, VOL connectors, and VFDs are not data. They are code. The HDF5 docs state that plugins are shared libraries and that the default path can be overwritten by `HDF5_PLUGIN_PATH`. That is enough to create an application-level vulnerability if a privileged process uses an attacker-controlled path.

Mitigation:

```
clear HDF5_PLUGIN_PATH, HDF5_PLUGIN_PRELOAD,  
      HDF5_EXTFILE_PREFIX in services  
set plugin loading state explicitly at process start  
allow plugins only from root/admin-owned directories  
reject writable plugin directories  
require hashes or signatures for plugins  
never load plugins in setuid, service, notebook-hub,  
      web-worker, or privileged contexts unless explicitly  
      allowlisted
```

Where a required plugin cannot be disabled, run the file-processing boundary with least privilege and process isolation in addition to enforcing protected paths, explicit allowlisting, and resource limits.

Recommendation APP-S6: Require downstream SBOM and version reporting

Downstream projects should report:

```
HDF5 version  
linkage type: static / dynamic / vendored / system  
enabled filters  
enabled VOL/VFD connectors  
plugin search path policy  
external-reference policy  
safe-mode policy
```

This reduces a practical applicability ambiguity: a user seeing “HDF5 CVE” in a scanner often cannot tell whether they are affected through `libhdf5`, an official tool, a vendored copy, a Python wheel, a model loader, or a plugin. Reporting supplies census inputs; it does not by itself establish that a deployment is affected, compatible, or remediated.

Recommendation APP-S7: Split CVE classification into scope tags

For HDF5-related CVE triage, use explicit tags:

```
HDF5-core-parser  
HDF5-official-tool  
HDF5-legacy-tool  
HDF5-plugin-loading  
HDF5-external-reference  
HDF5-downstream-deserialization  
HDF5-downstream-resource-policy  
HDF5-vendored-library  
not-HDF5  
duplicate/rejected
```

This prevents scope collapse and supplies an input to triage. Each bounded scenario must still be rated and routed under the report-wide rubric; a tag does not assign priority. A heap overflow in `H5T__conv_struct_opt` is not the same class as Keras ignoring `safe_mode` for legacy `.h5` files, even though both mention HDF5.

A practical mitigation matrix for downstream maintainers

Table 28 – Baseline protective controls requiring deployment-specific validation

Downstream use case	Baseline protective controls
viewing unknown .h5 files	run in a sandbox; disable plugins; disable external references; cap metadata and output size
loading ML models	use a verified fixed loader or reject or contain affected legacy formats; disable executable reconstruction, external references, and plugins; enforce aggregate and per-object budgets before allocation
cloud upload service	enforcement-coupled preflight and runtime denial bound to the same file identity; no plugin loading or external file access; resource limits and process isolation
HPC workflow	separate trusted internal files from imported files; scrub environment; pin plugin directories; use module-level HDF5 version floors
desktop scientific app	default-deny external references; permit only explicit, path-confined opt-in after validation; ship applicable fixes; avoid writable global plugin dirs
package/distribution	publish SBOM; expose HDF5 version; confirm applicability and meet the finding-specific response horizon, retaining tested containment when a rebuild is unavailable
library binding	expose security controls in the binding API, not just through environment variables

Reusable controls The HDF Group can provide to reduce downstream exposure: For reusable downstream prevention, parser hardening should be paired with enforceable controls that applications can use easily. Publish recipes using controls available today in parallel, and update them as the Phase 2 APIs ship. This delivery sequence does not replace or defer Immediate or Near-term application treatment.

Recommended deliverables:

```
H5Pset_security_profile(fapl, H5_SECURITY_PROFILE_UNTRUSTED)
H5Pset_external_reference_policy(fapl, H5_EXTREF_DENY
    | H5_EXTREF_SANDBOX)
H5Pset_plugin_policy(fapl, H5_PLUGIN_DENY
    | H5_PLUGIN_ALLOWLIST)
H5Pset_max_logical_dataset_bytes(fapl, n)
H5Pset_max_metadata_bytes(fapl, n)
```

```
H5Pset_max_object_count(fapl, n)
H5Pset_max_link_traversal_depth(fapl, n)
h5validate --profile untrusted --json file.h5
```

Maintain downstream recipes for Keras, TensorFlow, h5py, netCDF, MATLAB, R, Java, and cloud services as current controls and standardized APIs evolve.

7. HDF5 in the Software Supply Chain

This chapter examines HDF5's position in the software supply chain from two directions: what an HDF5 build itself depends on, and what depends on HDF5. Both matter to the assessment for the same reason — a defect, a configuration difference, or a compromise at either end reaches far beyond the project that introduced it.

Two release-integrity mechanisms frame the discussion. Source-artifact integrity checking, applied to the HDF5 2.0.0 release artifacts [86], allows a consumer to establish that the source it builds is the source that was published. A software bill of materials extends the same idea to what a binary contains, and is the input the RM-1A census needs in order to answer whether a given deployment is affected by a given finding. Neither is currently a routine, verifiable property of every HDF5 release artifact; CORE-S11 and SUP-S1 propose making them so.

7.1. Upstream Dependencies

This subsection summarizes external software that can be required by an HDF5 build. It starts with the default serial C library and is organized by build option, because the default serial C library has a much smaller dependency set than feature-complete, testing, packaging, or developer builds. More details are provided in Appendix E.

HDF5's CMake build can either locate dependencies that are already installed or, for selected filter-related projects, add external source trees to the build with `FetchContent`. See table 44 for details.

The baseline HDF5 build has no mandatory external dependencies. The core library and C API tests can be built and run without any additional software. See table 45 for details.

The HDF5 build system has options that trigger additional dependencies when enabled. For example, enabling parallel HDF5 requires an MPI implementation,

while enabling Fortran support requires a Fortran compiler. See table 46 for details.

Enabling filter plugins or the plugin project can introduce additional filter-library dependencies. The HDF5 cache defaults include entries for bitgroom, bitround, bitshuffle, Blosc, Blosc2, bzip2, fpzip, JPEG, LZ4, LZF, SZ, ZFP, and Zstandard. Those libraries are controlled by the plugin project's own options when `hdf5_plugins` is built or found. See table 47 for details.

Enabling VFDs for ROS3, HDFS, or other storage systems can introduce dependencies on external libraries and credentials. The same applies to VOL connectors. These are not filter dependencies, but they can affect the security and robustness of HDF5 file access, especially for third-party data. See table 48 for details.

Additional dependencies are required for optional tools, tests, and security features. For example, the `hdf5_security` target depends on OpenSSL when built with `HDF5_ENABLE_SECURITY_FEATURES`. See table 49 for details.

Finally, developer builds and packaging scripts have their own dependencies. These include code linters, formatters, static analyzers, documentation tools, and packaging tools. These dependencies are not required for a default HDF5 build, but they are relevant for the supply-chain risk of development and release processes. For example, if a security vulnerability is introduced in a packager or source formatter, it could affect the integrity of HDF5 releases even if the vulnerability is not present in the HDF5 source code itself. See table 50 for details.

Several default upstream source locations are encoded in `config/CacheURLs.cmake` for dependencies that HDF5 can fetch or whose values are passed to external filter-plugin builds. See table 51 for details.

7.2. Downstream Dependencies

For this audit, downstream dependencies are projects that compile against, dynamically load, package, or define data conventions on top of HDF5. The HDF5 ecosystem is much larger than any static list, so representative downstreams were

selected for broad installation footprint, long-lived archival data, domain-standard file conventions, or use as an intermediate layer for other tools. These are sampling criteria, not formal risk ranks. The following categories are especially relevant to the assessment scope.

Scientific data models and exchange formats.

A prominent downstream is netCDF-4, which uses HDF5 as the storage layer for the enhanced netCDF data model and is central to climate, weather, ocean, atmospheric, and other Earth-system data workflows [93]. NASA HDF-EOS5, NeXus, and CGNS similarly define domain-specific conventions over HDF5 for Earth observation, neutron/x-ray/muon facilities, and CFD or aerospace simulation data [55, 74, 24]. For these formats, HDF5 compatibility is not just a library question: it affects the readability of long-lived archives and the ability of independent tools to interpret a shared schema.

Language bindings and analysis libraries.

h5py, PyTables, pandas HDFStore, Bioconductor's rhdf5, HDF5.jl, and MATLAB's HDF5 interfaces make HDF5 a routine dependency for Python, R, Julia, and MATLAB users [54, 77, 76, 21, 61, 71]. These packages are important because they often sit at the boundary between upstream HDF5 builds and end-user scripts. Their packaging choices determine whether users load a bundled HDF5, a system HDF5, or an environment-provided HDF5, and therefore whether ABI, plugin-path, thread-safety, MPI, and filter support match the files being opened.

Geospatial, visualization, and inspection tools.

GDAL's HDF5 driver, ParaView/VTK-family builds, ITK's HDF5 module, HDFView, and similar readers expose HDF5 files to interactive and automated analysis workflows [75, 65, 60]. This is an important security boundary: these tools are commonly used to inspect third-party data files, so robustness against malformed HDF5 objects, unusual filters, and unexpected VFD/plugin configuration has direct operational value.

HPC and parallel-I/O frameworks.

ADIOS2 includes an HDF5 engine, while netCDF-4, CGNS, h5py, and many simulation applications can be built against serial or parallel HDF5 variants [3].

These downstreams amplify configuration risk: MPI ABI compatibility, collective I/O semantics, file locking behavior, subfilings support, and optional VFDs must line up across the application, HDF5, MPI, and filesystem stack.

Machine-learning and model-artifact workflows.

TensorFlow/Keras still supports older HDF5 model and weight files even as newer native formats are preferred [80]. This keeps HDF5 in the model-exchange and training-checkpoint ecosystem, usually through h5py, and makes HDF5 parser behavior relevant for model files obtained from external sources.

Table 29 – Representative downstream projects that depend on HDF5.

Downstream	Role	Supply-chain relevance
netCDF-4	Common scientific data model and format layered on HDF5.	Large transitive footprint through climate, weather, ocean, remote-sensing, and HPC data stacks. Uses a constrained HDF5 subset, so compatibility depends on both HDF5 behavior and netCDF conventions.
HDF-EOS5	NASA Earth-observation format built on HDF5 concepts and libraries.	Long-lived public archives and many third-party readers make backward compatibility and robust metadata handling high-value.
NeXus	Facility data convention for neutron, x-ray, and muon science.	Uses HDF5 groups, datasets, and attributes as a standardized experimental-data container, so HDF5 schema interpretation affects reproducibility across facilities.
CGNS	CFD and aerospace data standard with an HDF5 mapping.	Important for mesh and simulation-result exchange; parallel-HDF5 behavior and large-file support affect production simulation workflows.
h5py, PyTables, and pandas HDFStore	Python interfaces and tabular/array storage layers.	Widely used bridge between binary HDF5 files and Python analysis. Wheel and conda packaging can hide which HDF5 build is actually loaded.

rhdf5, HDF5.jl, MATLAB HDF5 APIs	R, Julia, and MATLAB access layers.	Expose HDF5 to laboratory and scientific-computing workflows where files are often exchanged outside controlled build environments.
GDAL and geospatial readers	Raster and subdataset access for HDF5-based products.	Converts HDF5 issues into geospatial ingestion issues, especially for remote-sensing products with complex metadata and nested subdatasets.
ParaView/VTK, ITK, HDF5View, and viewers	Interactive inspection, visualization, and conversion.	May open untrusted or externally generated files; defensive parsing, filter availability, and plugin search paths matter.
ADIOS2 and simulation I/O layers	Alternative I/O stacks with HDF5 read/write engines or compatibility paths.	Parallel and high-throughput workflows depend on matching MPI, filesystem, and HDF5 feature assumptions across the whole stack.
TensorFlow/Keras artifacts	HDF5 Legacy model and weight serialization.	Keeps HDF5 in ML artifact exchange; malformed or incompatible model files surface through h5py and HDF5 library behavior.

7.3. Audit Findings and Mitigation Planning

The build-and-feature mismatch theme is recorded as evidence-backlog item SUP-SAFE-001 in Appendix A. It is not a formal finding pending a bounded affected population and evidence sufficient to assign exposure. The bullets below describe ecosystem conditions and potential failure mechanisms; they are not additional formally rated findings.

Priority reconciliation.

Ecosystem reach and transitive blast radius justify the census in RM-1A, but do not assign SUP-SAFE-001 a tier or response urgency. The roadmap schedules a bounded producer/consumer mismatch study in RM-2F; a supported-profile compatibility and provenance program follows in RM-3E and RM-4A. If the study substantiates a concrete affected population and harm, the resulting finding

is rated and routed under the report-wide rubric rather than inheriting a priority from this chapter's project list.

- HDF5 changes have a large transitive blast radius because many users reach HDF5 through netCDF, GDAL, h5py, MATLAB, or domain formats rather than through the HDF5 C API directly.
- ABI and packaging mismatches are a plausible downstream mismatch mechanism; this assessment did not establish their frequency or affected population. Python, R, Julia, MATLAB, MPI, and system-package environments may all load different HDF5 builds in the same organization.
- File parsing is a practical security boundary. Viewers, converters, notebooks, and data-ingestion jobs often process files received from outside the organization that built the HDF5 library.
- Optional features can become hidden interoperability requirements: filter plugins, SZIP/zlib variants, ROS3/HDFS/VFD support, parallel HDF5, and file-locking behavior may be assumed by downstream data even when they are not present in a default HDF5 build.
- Long-lived scientific archives make deprecation slower than ordinary application software. Even when a downstream project prefers a newer format, historic HDF5 files remain operationally relevant.

The following identifier names a recommendation only; it does not assign risk, urgency, effort, dependency, or implementation order.

Recommendation SUP-S1: Maintain a producer/consumer profile compatibility and migration matrix

For every supported producer, reader, binding, and packaging profile, record the HDF5 version and linkage, ABI, enabled filters and plugins, VOL/VFD features, parallel and MPI assumptions, file-locking policy, and format bounds. Preserve representative artifacts and state the expected accept or controlled-reject result for each supported pair. When a mismatch is discovered, inventory the affected artifacts and publish a deterministic repair, rebuild, migration, or documented

compatibility path rather than relying on the fact that one older environment happened to accept the file.

Acceptance requires a finite, versioned supported-profile catalog, matrix, and fixture set in CI; diagnostics that identify the mismatched capability; and a tested deterministic disposition for every incompatible supported pair: repair, rebuild, migration, controlled rejection with a supported alternative, or an accountable exception. Release SBOM and feature reporting under CORE-S11 and APP-S6 supply provenance inputs but do not, by themselves, demonstrate compatibility or treatment of legacy artifacts.

Maintaining the supported-profile catalog and CI matrix is a proactive assurance control; it does not by itself create or close a formal finding. Mismatch-specific repair, rebuild, migration, controlled rejection, or exception is selected only after RM-2F bounds the affected profile population and consequence.

8. Independent HDF5 Implementations

8.1. Overview

PureHDF Managed .NET read/write without native binaries.

jHDF JVM HDF5 reading.

netCDF-Java JVM netCDF/HDF-EOS/HDF5 scientific-data reading.

pyfive Pure Python read-only inspection.

jsfive Browser-side direct HDF5 reading.

scigolib/hdf5 Pure Go read/write.

hdf5-pure and rustyhdf5-format/rustyhdf5 Pure Rust read/write.

async-hdf5 or h5coro Cloud object-store chunk mapping.

JLD2.jl Julia-native persistence in HDF5-compatible files.

hdf5-pickles / H5Lens Machine-readable format definitions with an independent reader, validator, and preflight oracle. Distinct from the others in that its interpretation of the format is *inspectable as data* rather than inferable from behaviour.

Vocabulary. This chapter uses the issue families defined by the SSP SIG's HDF5 Independent Implementation (H5II) model [42]: **I1** parser/spec divergence, **I2** fail-open compatibility behaviour, **I3** writer correctness and interoperability failure, **I4** codec, filter, or external-reference boundary failure, **I5** numeric, structural, or resource amplification, **I6** concurrency, durability, or lifecycle mismatch, **I7** metadata and artifact exposure, **I8** dependency, provenance, or compatibility-claim drift, and **I9** trust-boundary confusion. As elsewhere, these labels classify mechanisms; they are not risk ratings, and they do not carry the SSP SIG's numeric scores into Section 1.7.

The model's central observation frames this chapter: an independent implementation removes a dependency on the reference library but not the hard problem,

because the trusted computing base becomes the implementation's own parser, validator, writer, and support libraries. Its working assumption that *partial support is normal, but unsupported or unknown features must fail closed rather than being silently ignored or guessed* is the specific standard against which the two 2026 cases below are best read.

Why the count matters. The number of independent implementations is not a curiosity; it changes the character of specification risk. With a single implementation, an ambiguity in the format is invisible, because that implementation's behaviour is operationally indistinguishable from the format itself — there is nothing for it to disagree with. With five or nine, **every ambiguity is resolved independently once per implementation**, and each unresolved ambiguity becomes a latent divergence that surfaces only when files cross implementation boundaries. That is precisely when the data is least recoverable and the diagnosis hardest, because the failure appears as a reader defect rather than as the specification defect it is.

This is the mechanism behind FMT-SAF-001. Both documented instances — issues #6409 and #6616 — were found by an independent implementation exercising a construct and diverging from the reference library, not by reading the specification. Independent implementations are therefore currently the ecosystem's only conformance test, and what they are detecting is not confined to their own defects: they are finding errors in the reference library and in the specification. That is a strength of the ecosystem and an indictment of the normative artifact at the same time.

Independent readers and writers are valuable checks on both the prose specification and the reference implementation. They also expose a three-way distinction essential to interoperability analysis: a file can be accepted by one reader, conform to or violate the normative format independently of that acceptance, and remain safe or unsafe for a particular deployment. Reader acceptance alone is not conformance evidence.

8.2. Two Distinct 2026 Chunk-Layout Cases

The two cases below produced superficially similar upgrade failures, but they must not be combined. One involved a reader imposing a stronger condition than the format required; the other involved contradictory writer output that older readers had tolerated.

8.2.1. Sufficient but Nonminimal Encoding

HDF5 issue #6409 concerns version-4 chunk-layout dimension widths. The format required an encoded width sufficient to hold the dimension value, but did not require the smallest possible width. The exact-width check present in HDF5 2.0.0, 2.1.0, and 2.1.1 could therefore reject a specification-permitted, nonminimal encoding produced by an independent writer. Stated plainly, and this is the part the interoperability framing tends to soften: **the reference library was non-conforming**, the specification was correct, and it took an independent implementation to establish that. The merged HDF5-side change accepts any sufficient width and was tracked against 2.2.0 [4, 29]. PureHDF separately changed new output to match the reference reader's expectation [22, 66] — a rational local decision with a corrosive global effect, since converging on the reference implementation rather than on the specification is how a specification quietly stops being normative. FMT-SAF-001 records that dynamic. Formal finding IND-SAF-001 records the historical reader-side interoperability risk; issue closure is not, by itself, verification of every release artifact or backport. In H5II terms this is **I1**, parser/spec divergence — and note that it is divergence by the *reference* reader, which is the direction the model's framing makes it easy to miss.

8.2.2. Contradictory PureHDF Metadata

HDF5 issue #6534 began as a suspected HDF5 2.0 regression because the same production-derived file opened with HDF5 1.10.10 and 1.14.6 but failed with HDF5 2.0. Maintainer byte-level analysis identified a different condition in the

affected PureHDF 2.1.2 output: float64 datatype metadata declared an eight-byte element while the version-4 chunk-layout message recorded a four-byte stored element. The file was internally inconsistent; the older readers had been permissive, and the HDF5 2.0 rejection was correct [2, 1].

The reporter described a continuously written production acquisition pipeline and a reader-version pin as the contemporaneous workaround. A technical metadata-repair path was proposed and older readers remained a workaround, although the reporter said rewriting the files was not operationally feasible in that pipeline. The pin restored availability but was neither a conformance repair nor the report's Immediate urgency category. The evidence therefore supports bounded operational and archival harm, not irreversible data loss. Formal finding IND-SAF-002 records this writer-side condition. Existing affected files remain a migration and inventory question even after corrected writer output is deployed. In H5II terms it is **I3**, writer correctness and interoperability failure, and the older readers' acceptance of the file is **I2**, fail-open compatibility behaviour — the pair is the model's canonical chain, where a writer emits bytes that are only self-consistent to itself and a permissive reader conceals it until a stricter one does not.

8.3. When the Specification Itself Is the Defect

The two chunk-layout cases above are divergences with a determinable answer: once examined, it was clear which party was wrong. A third case shows that this is not guaranteed.

HDF5 issue #6616 records that the file-format specification documents the group info message as *optional*, while the C library raises a “message type not found” error when it is absent and consequently cannot modify a file written without one. The reporter — again an independent-implementation maintainer, again working from a PureHDF change — sets out the two available resolutions: the library should fall back to default group-info values, or the specification should be corrected to state that the message is required. At the evidence cutoff the issue is open, labelled a documentation bug, assigned, and milestone, with no ruling recorded [5].

Until that ruling exists, **it is not determinable whether a file without a group info message conforms**. This matters beyond the individual case for two reasons. First, it is a limitation of this report’s own method: the conformance analysis in Section 1.6 distinguishes the normative requirement from writer behaviour and reader validation, and here the normative requirement is exactly what is in dispute. Second, it shows that the missing control is not only a better artifact but a *procedure* — who rules, on what timeline, and where the ruling is recorded. A machine-readable specification would have forced the question earlier by making the optionality explicit and testable; it would not, by itself, have answered it.

The condition is registered as FMT-SAF-001 in Appendix A and treated by FMT-S1 below.

8.4. Audit Findings and Mitigation Planning

The main interoperability challenge across direct-format implementations is not opening simple files; it is coverage of the long tail: dense groups, fractal heaps, shared object-header messages, chunk-index variants, references, VLEN, compound and nested datatypes, virtual datasets, external storage, SWMR flags, user blocks, non-core filters, unusual library-version bounds, and files produced by different HDF5 releases. This architectural observation is not assigned a risk tier without a concrete affected workflow. The H5II model’s II-## review register is the natural instrument for working through that long tail feature by feature, and IND-S1 below supplies the fixtures it would need. The two bounded formal findings are IND-SAF-001 and IND-SAF-002 in Appendix A; the condition that produces them is registered separately as FMT-SAF-001.

Priority reconciliation.

FMT-SAF-001 is routed Near-term, and its Near-term obligation is narrow: rule on the disputes already identified, record each ruling, and change whichever artifact was wrong. That is a small, bounded action with a named owner in at least one case, and it is not contingent on FMT-S1. IND-SAF-001 and IND-SAF-002 are Moderate and follow the Planned route; neither is ordered ahead of the other

by issue chronology or recommendation numbering. Phase 2 therefore verifies the exact supported reader artifacts, preserves positive and negative fixtures, inventories the affected legacy PureHDF population, and provides a deterministic repair, migration, or compatibility disposition. Invariant and sustainment work continues in Phases 3–4. A production or archival population discovered to be actively interrupted may be expedited, but that scoped decision does not revise the historical inherent ratings.

The FMT-S and IND-S identifiers below identify recommendations only; their numbers do not express risk, urgency, effort, dependency, or implementation order.

Recommendation FMT-S1: Make the specification machine-readable and adjudicable

Two separate deliverables are conflated whenever this is discussed, and they fail in different ways if either is attempted alone.

The artifact. Maintain the normative file-format description in a machine-readable form from which validators and fixtures can be generated, rather than only as prose. The argument is not tooling convenience. **Prose can be silently ambiguous; an executable description cannot.** Writing “the group info message is optional” in a form that must generate a valid file and validate one forces the question that issue #6616 is currently suspended on — optional to write, optional to read, or optional only when some other condition holds. The existing `hdf5-pickles` work is the obvious foundation, and it is already an independent implementation in its own right rather than only a fixture generator [30].

Two constraints on that foundation. It is GPLv3 while HDF5 is BSD-style, so the relationship must be consumption of identifiers, boundaries, and fixtures as data rather than incorporation of source; see Section C.3. And a machine-readable description is normative only if the project says it is — otherwise it becomes a fourth artifact to disagree with, alongside the prose, the library, and the implementations.

The procedure. Adopt a standing rule for specification-versus-implementation disputes, covering who rules, within what period, what the possible dispositions are (the specification is corrected, the library is corrected, or the construct is declared implementation-defined and documented as such), and where the ruling is recorded so the next implementer finds it. Issue #6616 is the evidence that the artifact alone is insufficient: the contradiction has been identified, labelled, assigned, and milestone, and it is still not adjudicated. An ambiguity that is documented but unruled is not resolved.

The two halves reinforce each other. Without the procedure, the machine-readable artifact records disagreements it cannot settle. Without the artifact, the procedure has nothing precise to rule on and every dispute is re-litigated from prose.

Acceptance evidence. For a declared subset of the format — not the whole format, which is a multi-release scope — the machine-readable description generates valid fixtures that the reference library accepts and invalid fixtures it rejects, and any divergence is either an identified library defect or an identified description defect, with no third category. Every dispute raised in the tracking period has a recorded disposition naming the authoritative artifact and the change made, and the elapsed time from report to ruling is published. A dispute closed by an implementation changing its output to match the reference library, without a ruling on which artifact was authoritative, is recorded as *unresolved* rather than closed.

Recommendation IND-S1: Maintain a differential conformance and compatibility corpus

Independent writers should validate generated files against normative on-disk invariants and a matrix of current and supported older reference readers. Reference-library maintainers should preserve independent-writer fixtures that exercise specification-permitted alternatives, not only files emitted by libhdf5 itself. The shared corpus should include both positive fixtures, which must be accepted, and negative fixtures, which contain a single intentional inconsistency and must be rejected predictably.

At minimum, the program should:

- round-trip generated files through `h5dump`, `h5debug`, `h5py/libhdf5`, and the independent implementation;
- test every supported chunk-layout and index variant, datatype width, filter pipeline, external-resource form, and library-version bound;
- preserve two separate regression cases: #6409 for sufficient but nonminimal widths and #6534 for contradictory element sizes;
- carry a third fixture class for *contested* readings — a construct whose conformance is genuinely undetermined, such as a group written without a group info message under #6616 — recorded with the competing interpretations and the expected result under each, so that a dispute has a test rather than an argument and so that the eventual ruling has something to close;
- report the writer, writer version, format bounds, reader, reader version, and expected accept/reject result for every fixture; and
- inventory already-produced files when a writer defect is discovered, provide a deterministic repair, regeneration, migration, or supported compatibility disposition, and preserve checksums and provenance when bytes change.

Acceptance requires the current supported-reader matrix to accept every valid fixture, reject every intentionally invalid fixture with a controlled and documented diagnostic, and demonstrate a reproducible repair, regeneration, migration, or accountable supported-compatibility disposition for the complete defined legacy PureHDF population, preserving checksums and provenance when bytes change.

The machine-readable specification project `hdf5-pickles` is a natural source for generating and checking these fixtures [30], and under FMT-S1 it is also the candidate normative artifact rather than only a generator of test material.

9. Long-term Data Accessibility and Interpretability

Long-term accessibility depends on more than preservation of the primary file bytes. It can also depend on format bounds, required filters and extensions, external resources, application schema and provenance, and the continued availability of readers that implement the relevant interpretation. A file whose bytes are perfectly preserved and whose required filter no longer exists has been archived in the same sense that a sealed envelope with no known language inside has been archived.

9.1. Why Extensions Are the Sharp Edge

HDF5's extension mechanisms are the part of the format that most readily converts a self-describing file into a dependent one, and the three classes degrade differently. The distinction matters because it changes both the severity and what a curator can do about it.

Filter plugins.

The best case, and the only one with short-horizon evidence. A filtered dataset records its filter by numeric identifier, so the dependency is at least *discoverable* from the file, and a determined reader can in principle re-implement the codec. It is nonetheless a real and current failure mode: users already report being unable to read data because a required filter was missing from their build or unavailable on their platform, across a fifteen-year span and several filter families [19, 13, 10, 14]. Registered filter identifiers help; an unregistered or private identifier removes even discoverability.

Virtual file drivers.

A VFD determines whether the bytes can be reached at all. If a file was written through a driver with a non-trivial storage mapping, preserving “the file” may not preserve a readable object, and the dependency is less visible in the file than a filter identifier is.

VOL connectors.

The most severe and the least evidenced. A VOL connector can service the

HDF5 API from a store that is not an HDF5 file at all. In that case there may be no HDF5 file to archive, and an institution that believes it has preserved HDF5 data may have preserved a client library pointed at a system that no longer exists. This report identifies the exposure and does not quantify it.

Community discussion has long recorded the operational importance of preserving plugin and reader knowledge alongside archived data [7]. This assessment did not inventory an archive or test a representative preservation horizon, and the archival extrapolation — that extension availability decreases over a decade rather than increasing — is reasoned inference rather than measurement.

9.2. Audit Findings and Mitigation Planning

The scenario is registered as LONG-SAF-001 in Appendix A, rated **I2–I3/E2, Moderate–High, Provisional, Low confidence** at a stated assessment horizon of ten years or more. Because that horizon is material, decision rule 7 of Section 1.7 forbids ordering this rating directly against ratings that state no horizon.

The bounded reader/writer compatibility cases in Section 8, IND-SAF-001 and IND-SAF-002, remain separately registered. They are worth reading as the short-horizon, fully evidenced instance of the same family: in both, a file that one reader accepted became unreadable to another. The only difference between those cases and LONG-SAF-001 is how long the gap took to open.

Priority reconciliation.

LONG-SAF-001 is routed *Planned* by recorded departure from the default map, for the rationale given in Appendix A. The operative point is that the effective treatment window is at *write and deposit* time, not at read time: once an archive holds files whose extension is gone, the remaining options are recovery archaeology or loss. A 30-day clock on a decadal harm would measure nothing, while a producer-side practice adopted now removes the harm prospectively. If an archive is identified whose readers are already failing, that scoped instance is expedited and rated without a horizon qualifier.

The following identifier names a recommendation only; its number expresses no risk, urgency, effort, dependency, or implementation order.

Recommendation LONG-S1: Make extension dependencies discoverable, then removable

Treat an extension dependency as something a file must declare, a depositor must justify, and an archive must be able to satisfy without the original environment.

1. **Declare.** Provide a supported way to enumerate every non-core dependency of a file — filter identifiers and versions, VFD, VOL connector, external resources, and format bounds — as a single machine-readable report. A depositor cannot manage a dependency that cannot be listed. This reuses the reporting proposed in APP-S6 and CORE-S11, applied to a file rather than to a build.
2. **Prefer removable dependencies at deposit.** Where an archival copy is being produced, rewriting to core-only features — or to a filter that is registered, widely implemented, and specified — is the single most effective control, and `h5repack` already performs the mechanical part. Make this an explicit, documented archival profile rather than folklore.
3. **Preserve the specification, not the binary.** Where a dependency cannot be removed, archive what would let someone re-implement it: the codec specification, test vectors with known inputs and outputs, and the identifier registration. A preserved shared library becomes unbuildable long before a preserved specification becomes unreadable.
4. **Say what a VOL-backed dataset actually is.** Require that a deposit record state whether the archived artifact is a self-contained HDF5 file or a view onto an external system, because those are different preservation objects with different custodians.

Acceptance evidence. For a defined corpus: every file has a generated dependency report, and the report is confirmed complete by opening the file in an environment where each declared dependency is deliberately absent and checking that no

undeclared dependency is discovered. Files in the archival profile open and read correctly in a reference build with *all* non-core plugins disabled — that is the test that proves the dependency was actually removed rather than merely satisfied locally. For each retained dependency, the archived specification and test vectors are sufficient for an independent party to reproduce the codec output, demonstrated at least once rather than assumed.

10. Conclusion

10.1. Overall Assessment Conclusion

Within the engagement scope and the August 27, 2026 evidence cutoff, this assessment supports a bounded conclusion: HDF5 input is not necessarily inert data. File-controlled metadata can reach memory, allocation, traversal, external-resource, plugin, deserialization, storage, and disclosure boundaries. Risk arises at the component that supplies and controls the relevant authority, and not every appearance of HDF5 in an adverse event is therefore a defect in the core library or file format.

The evidence supports four ecosystem-level judgments. First, historically affected core and official-tool paths exhibit recurring failures to validate file-derived values before trusted C operations; local fixes can treat a known variant, but class elimination requires invariant-driven prevention and continuing regression evidence. Second, applications can turn documented HDF5 capabilities into security mechanisms when they process untrusted files with ambient filesystem, plugin-loading, deserialization, or resource authority. Third, reader acceptance, format conformance, and safe deployment are separate decisions: permissive historical acceptance is not proof that a file is consistent, and stricter rejection is not automatically a regression — and in at least one documented case the reference library, not the independent writer, was the non-conforming party, while in another it is still undetermined which artifact is wrong. Fourth, privacy harm can arise during correct and authorized operation because payload protection does not necessarily protect metadata, structure, derived information, caches, logs, or other workflow artifacts. Fifth, and distinct from the preceding four in kind: a substantial part of HDF5's risk is not about input at all. Storage exhaustion, abrupt termination, and extension-dependent loss of interpretability threaten data that no adversary ever touched, and the library provides materially weaker guarantees against them than against malformed input.

The registered scenarios have a determinate, conditional mitigation-priority order, and confirmed applicable scope follows it. No exception permits a known, uncontained Critical-equivalent external-resource path to process untrusted input; any other variance requires a scoped, accountable, expiring record and tested interim control. The rubric and response routing control—not chapter order, recommendation number, CVE count, publicity, effort, or dependency. This is neither a universal HDF5 rating nor a certification or conclusion about an unassessed release or deployment.

The authoritative register contains fifteen formal findings: seven Rated and eight Provisional. Their tiers comprise one Critical, four High, six Moderate–High ranges, and four Moderate. Three additional observations remain in the evidence-and-scoping backlog. These counts describe the bounded scenarios in [Appendix A](#); they are not prevalence estimates and must not be aggregated into a score for HDF5 as a technology.

10.2. Reconciled Mitigation Priorities

[Table 30](#) summarizes the only report-wide ordering. The deployment census and assert-site source catalog are due within 30 days of roadmap adoption; a finding-response clock starts when an affected deployment or equivalent exposure is identified. Treatment removes or controls the concrete scenario; prevention addresses recurrence and may continue after the scoped finding is reassessed.

Table 30 – Conclusion-level reconciliation of mitigation priorities

Decision lane	Records	Required outcome	Boundary
Immediate—before the next untrusted job and within 24 hours of identification	APP-SEC-001; a separately confirmed Critical-equivalent service exposure under APP-SEC-003	Stop or sandbox attacker-selected external-resource resolution; remove unnecessary worker authority, preserve evidence, and rotate exposed secrets where indicated.	Applies to an equivalent still-exposed deployment, not automatically to every HDF5 service or to Hugging Face’s present state.
Near-term—treatment within 30 days of affected-scope identification	CORE-SEC-001, CORE-TOOL-001, CORE-PRIV-001, CORE-SAFE-001, CORE-SAFE-002, FMT-SAF-001, APP-SEC-002, APP-SEC-003, APP-SEC-005	Use the program census to identify scope; patch, upgrade, isolate, or deploy a tested compensating control for each affected deployment; minimize sensitive plaintext metadata and verify the enforcing trust boundary. For the two write-side safety findings the obligation is tested operational containment — capacity headroom and monitoring, flush and checkpoint discipline, a rehearsed recovery procedure, and an independent copy of unregenerable data — not the library change.	Census: 30 days after roadmap adoption. Treatment: 30 days after affected-scope identification. Ranges route at their upper bound; dependencies do not extend treatment.
Planned—within 90 days or the next supported release or migration window, whichever is sooner	CORE-SEC-002, APP-SEC-004, IND-SAF-001, IND-SAF-002, LONG-SAF-001 (recorded departure)	Bound file-controlled work and allocation; verify exact reader/writer behavior; inventory affected artifacts; and provide tested repair, migration, or controlled compatibility handling.	Expedite a scoped item when an affected production pipeline, archive, or service is identified.

Table 30 – Conclusion-level reconciliation of mitigation priorities, continued

Decision lane	Records	Required outcome	Boundary
Evidence action—no assigned risk urgency	CORE-SEC-003, EXT-PRIV-001, SUP-SAFE-001	Perform bounded, reproducible studies sufficient to rate and route a supported scenario or to document a negative result over the tested scope; catalog and compare assertion-enabled and -DNDEBUG behavior for file-derived assertions.	NE and backlog status do not mean Low risk. The 30-day Phase 1 source-catalog target and Phase 2 administrative targets are assurance schedules, not risk-response categories.
Continuing prevention and sustainment	Relevant formal findings and conditional evidence records	Deliver strict processing and preflight controls, invariant-driven parsing, always-active release-build checks, checked arithmetic, differential conformance tests, governed extension loading, privacy-lifecycle controls, provenance, release gates, and reassessment.	Phases 2–4 may run in parallel. Structural work does not defer earlier treatment or acquire an inherent-risk tier of its own.

This ordering also reconciles the chapters. The application chapter owns the only Immediate route. The core and tool chapter combines Near-term treatment of confirmed affected High and upper-bound-High scope with Planned resource controls and longer-term class prevention; the assert-masking observation adds a 30-day source-catalog action but no risk tier of its own; its one reproduced instance is routed under CORE-SEC-002 by that instance's consequence. The core chapter additionally owns two Near-term write-side safety containments and their longer structural treatments. The independent-implementation chapter owns two Planned interoperability treatments, and the long-term-access chapter owns one Planned, horizon-qualified archival finding that an earlier revision left unregistered. Extension privacy and supply-chain mismatch themes remain evidence actions until a concrete affected population and harm are established. The

wrappers chapter creates no additional formal priority in this revision. Appendix C then translates these lanes into accountable work packages without changing their inherent risk.

10.3. Significance of the 2026 Developments

The Hugging Face incident and HDF5 issue #6618 sharpen the application-boundary conclusion. The incident demonstrates that a valid, documented external-storage capability can become a repeatable disclosure mechanism when an automatic service supplies filesystem authority on behalf of an untrusted file. That evidence supports Immediate containment of equivalent deployments and the need for unified, enforcement-coupled external-resource policy. It does not support reclassifying the mechanism as a core parser vulnerability, treating a policy proposal as an implemented control, or assigning the incident's Critical tier to every HDF5 consumer [67, 91, 25, 78].

The PureHDF case sharpens the safety and interoperability conclusion in a different way. Older-reader acceptance of contradictory chunk metadata did not make the file conforming, while the separate sufficient-but-nonminimal case shows that stricter validation can itself reject a permitted encoding. Together the cases require a normative invariant decision, paired positive and negative fixtures, exact supported-reader verification, an inventory of affected legacy files, and deterministic repair or migration. The correct response is neither permissiveness by default nor strictness without compatibility evidence [4, 29, 2, 1].

HDF5 Pickles issue #87 sharpens the core-parser conclusion. A validity predicate derived from in-file bytes but enforced only inside `assert(...)` is not a release control when `-DNDEBUG` removes it. The issue's seven component families and cursory 50–100 estimate define an urgent audit perimeter, not 50–100 proven vulnerabilities. This changes sequence without manufacturing a rating: complete the catalog and paired release/debug tests within 30 days of roadmap adoption, route and locally repair a supported adverse path as soon as it is confirmed, and use the results to order independently releasable Phase 3 migration tranches [57].

The perimeter is no longer entirely hypothetical. One instance has been carried to reproduction and is promoted into CORE-SEC-002: a version-2 B-tree header declaring a zero record size reaches an integer division whose only guard is an assertion, producing SIGFPE on a shipped tool in a release build, at both 32- and 64-bit widths [33]. That single case settles the question of kind while leaving the question of extent open, and it does so in the direction the rubric anticipated: the mechanism is real, the consequence for this instance is termination rather than corruption, and the affected version range is still unestablished. It is evidence that the 30-day catalog is warranted, not evidence of its result. The promotion also demonstrates the routing rule working as designed — a confirmed path leaves the backlog and acquires its consequence's tier, without the backlog record itself ever having been assigned one.

The developments therefore reinforce the rubric rather than reorder it by recency or publicity. The Hugging Face evidence supports a Critical application-boundary scenario with an Immediate conditional response; the two independent-implementation cases support Moderate safety scenarios with Planned treatment; and the assert-masking observation remains an evidence action until exact paths and consequences are verified. Together they show why the enforcing actor, affected scope, consequence, and acceptance evidence must be explicit.

10.4. Assurance, Closure, and Residual Risk

This report does not establish that any proposed mitigation has been adopted or that current residual risk has been reduced. Unless a record expressly states otherwise, the assessment relied on public document and artifact examination rather than independent reproduction. Current installed populations, exact fix coverage across supported releases, and deployment-specific control effectiveness remain incomplete. Every roadmap package is therefore *Proposed / implementation not verified* at publication, and no residual-risk acceptance is recorded.

Priority becomes an auditable commitment only after the roadmap authority adopts or modifies the plan, defines the census boundary, assigns named owners and due dates, and preserves finding-by-scope response records. A concrete

finding may close after verified local treatment removes its scoped scenario and the accountable authority records the residual-risk disposition; the separate recurrence-prevention program remains open until its own acceptance evidence passes. Every package requires independent verification and a residual-risk disposition. For High and Critical scope, the verifier must also be separate from the package owner, rollout approver, and risk acceptor.

The practical conclusion is consequently twofold. Known affected populations should be treated on their response clocks without waiting for architectural work, and HDF5's recurring classes should be addressed structurally without pretending that local closure proves class elimination. Applied together, those tracks convert the report from a catalog of concerns into a testable mitigation program: constrain file-selected capabilities at authority boundaries, validate file-derived state before trusted object construction, decide compatibility against normative evidence, and govern privacy across the full data lifecycle.

References

- [1] Abbo93. *Compatibility Warning: HDF5 2.0.0 / h5py 3.16.0 Regression Breaks Reading PureHDF Files*. 2026. [URL](#). (accessed: 27.08.2026).
- [2] Abbo93. *Regression in 2.0.0 Chunk Layout Decoder Fails on Valid Files from Third-Party Writers (PureHDF)*. 2026. [URL](#). (accessed: 27.08.2026).
- [3] ADIOS2 Developers. *ADIOS2 Supported Engines: HDF5*. 2026. [URL](#). (accessed: 29.05.2026).
- [4] Apollo3zehn. *HDF5 lib is more restrictive than file format spec*. 2026. [URL](#). (accessed: 27.08.2026).
- [5] Apollo3zehn (PureHDF maintainer). *Group info message is documented as optional but the C library requires it*. 2026. [URL](#). (reported: 16.08.2026; open, labelled documentation bug, milestone 2.3.0; accessed: 30.08.2026).
- [6] Apple. *CVE-2021-31009*. 2021. [URL](#). (accessed: 04.06.2026).
- [7] Multiple authors. *223 results for 'plugin'*. 2007-2026. [URL](#). (accessed: 21.05.2026).
- [8] Multiple authors. *Avoiding a corrupted HDF5 file or being able to recover it*. 2019. [URL](#). (accessed: 30.08.2026).
- [9] Multiple authors. *Crashproofing HDF5*. 2024. [URL](#). (accessed: 30.08.2026).
- [10] Multiple authors. *Dynamic filters on Windows*. 2017. [URL](#). (accessed: 30.08.2026).
- [11] Multiple authors. *File state after flush and crash*. 2015. [URL](#). (accessed: 30.08.2026).
- [12] Multiple authors. *Flushing files sometimes corrupts them*. 2018. [URL](#). (accessed: 30.08.2026).
- [13] Multiple authors. *H5Dump dataset retrieve error (compressed datasets)*. 2015. [URL](#). (accessed: 30.08.2026).
- [14] Multiple authors. *h5dump error: unable to print data (missing LZO filter)*. 2017. [URL](#). (accessed: 30.08.2026).

- [15] Multiple authors. *HDF5 data corruption issues after a crash?* 2020. [URL](#). (accessed: 30.08.2026).
- [16] Multiple authors. *HDF5 file state in case of crash*. 2010. [URL](#). (accessed: 30.08.2026).
- [17] Multiple authors. *Preventing file corruption on crash*. 2012. [URL](#). (accessed: 30.08.2026).
- [18] Multiple authors. *Recover a corrupt HDF5 file*. 2008. [URL](#). (accessed: 30.08.2026).
- [19] Multiple authors. *Troubles with compressed data*. 2012. [URL](#). (accessed: 30.08.2026).
- [20] Andreas Beckmann and Gerd Heber. *How to properly close HDF5 files when disk is full? — design discussion*. 2023–2026. [URL](#). (accessed: 30.08.2026).
- [21] Bioconductor Project. *rhdf5*. 2026. [URL](#). (accessed: 29.05.2026).
- [22] Blackclaws. *Chunked Datasets Are Unreadable by HDF5 2.x: Chunk Dimension Size Encoded Length Is Hardcoded*. 2026. [URL](#). (accessed: 27.08.2026).
- [23] Sean Brooks et al. *An Introduction to Privacy Engineering and Risk Management in Federal Systems*. NISTIR 8062. Gaithersburg, MD: National Institute of Standards and Technology, Jan. 2017. DOI: [10.6028/NIST.IR.8062](#). [URL](#).
- [24] CGNS Steering Committee. *CGNS/HDF5 – An HPC implementation*. 2026. [URL](#). (accessed: 29.05.2026).
- [25] chinhnguyen007. *Local File Access Through Untrusted HDF5 External Dataset References*. 2026. [URL](#). (accessed: 27.08.2026).
- [26] Cisco Talos. *HDF5 Group libhdf5 H5T_ARRAY Code Execution Vulnerability*. 2016. [URL](#). (accessed: 27.08.2026).
- [27] CVE Project. *CVE-2021-36977*. 2021. [URL](#). (accessed: 04.06.2026).
- [28] CVE Project. *CVE-2026-34734*. 2026. [URL](#). (accessed: 04.06.2026).

- [29] Neil Fortner. *Allow Reading of Files with Chunk Dimensions Encoded Using More Bytes than Necessary*. 2026. [URL](#). (accessed: 27.08.2026; merged pull request).
- [30] The HDF Group. *A machine-readable specification of the HDF5 file format*. 2026. [URL](#). (accessed: 21.05.2026).
- [31] The HDF Group. *Case Record: Cache-Image LRU Rank Guarded Only by an Assertion*. 2026. [URL](#). (accessed: 30.08.2026).
- [32] The HDF Group. *Case Record: External Data File Opened Before Segment Validation*. 2026. [URL](#). (accessed: 30.08.2026).
- [33] The HDF Group. *Case Record: v2 B-tree Record Size Guarded Only by an Assertion*. 2026. [URL](#). (measured: 15.08.2026; re-measured: 29.08.2026; accessed: 30.08.2026).
- [34] The HDF Group. *Commit 3914bb7*. Sep 25, 2025. [URL](#). (accessed: 21.05.2026).
- [35] The HDF Group. *Commit 804f3bae*. Aug 13, 2025. [URL](#). (accessed: 21.05.2026).
- [36] The HDF Group. *Commit ce53bc0*. Mar 31, 2024. [URL](#). (accessed: 21.05.2026).
- [37] The HDF Group. *H5Patch: Evidence-Gated Repair Planning, Application, and Audit Logging*. 2026. [URL](#). (accessed: 30.08.2026).
- [38] The HDF Group. *H5Policy Invariant Registry, Validation Coverage, and libhdf5 Evidence*. 2026. [URL](#). (accessed: 30.08.2026).
- [39] The HDF Group. *H5Policy: Metadata Preflight, Security Profiles, Regression Corpus, and Differential Testing*. 2026. [URL](#). (accessed: 30.08.2026).
- [40] The HDF Group. *HDF5 CVE Test Suite*. 2026. [URL](#). (accessed: 21.05.2026).
- [41] The HDF Group. *HDF5 extension skeletons, literally*. 2026. [URL](#). (accessed: 21.05.2026).
- [42] The HDF Group. *HDF5 Independent Implementation (H5II) Model*. 2026. [URL](#). (accessed: 30.08.2026).
- [43] The HDF Group. *HDF5 Plugins*. 2026. [URL](#). (accessed: 21.05.2026).
- [44] The HDF Group. *HDF5 Python Wrapper (H5PW) Model*. 2026. [URL](#). (accessed: 30.08.2026).

- [45] The HDF Group. *HDF5 Safety Hazard Model (H5CS)*. 2026. [URL](#). (accessed: 30.08.2026).
- [46] The HDF Group. *HDF5 Safety, Security & Privacy (SSP) Special Interest Group's governance, processes, and artifacts*. 2026. [URL](#). (accessed: 21.05.2026).
- [47] The HDF Group. *HDF5 Security Threat Model*. 2026. [URL](#). (accessed: 30.08.2026).
- [48] The HDF Group. *HDF5 SSP SIG Policy Manual and HDF5-SHINES- * Control Catalog*. 2026. [URL](#). (accessed: 30.08.2026).
- [49] The HDF Group. *HDF5 SSP SIG Privacy Exposure Registry (EXP-001-EXP-010)*. 2026. [URL](#). (accessed: 30.08.2026).
- [50] The HDF Group. *HDF5 SSP SIG Safety Hazard Registry (HAZ-001-HAZ-016)*. 2026. [URL](#). (accessed: 30.08.2026).
- [51] The HDF Group. *HDF5 SSP SIG Security Threat Registry (ATK-001-ATK-013)*. 2026. [URL](#). (accessed: 30.08.2026).
- [52] The HDF Group. *OSPS → SHINES Coverage Matrix*. 2026. [URL](#). (accessed: 30.08.2026).
- [53] The HDF Group. *Top Weaknesses (CWE) that Become Vulnerabilities in HDF5 Contexts*. 2026. [URL](#). (accessed: 30.08.2026).
- [54] h5py Developers. *HDF5 for Python*. 2026. [URL](#). (accessed: 29.05.2026).
- [55] HDF-EOS Tools and Information Center. *HDF-EOS Libraries*. 2026. [URL](#). (accessed: 29.05.2026).
- [56] Gerd Heber. *A CVE Strategy for the HDF5 Library*. 2026. [URL](#). (accessed: 30.08.2026).
- [57] Gerd Heber. *Catalog assert-masked de-serializer invariants in the HDF5 library source*. 2026. [URL](#). (created: 24.08.2026; accessed: 28.08.2026; open issue).
- [58] Gerd Heber. *What is Bounded Raw Decode?* 2026. [URL](#). (accessed: 30.08.2026).
- [59] Zero Day Initiative. *CVE-2026-2492*. 2026. [URL](#). (accessed: 21.05.2026).

- [60] Insight Software Consortium. *ITK Module: ITKHDF5*. 2026. [URL](#). (accessed: 29.05.2026).
- [61] JuliaIO. *HDF5.jl*. 2026. [URL](#). (accessed: 29.05.2026).
- [62] Keras Team. *Denial of Service via Malicious HDF5 Shape Metadata in a Keras Model*. 2026. [URL](#). (accessed: 27.08.2026).
- [63] Keras Team. *Legacy HDF5 Model Loading Ignores safe_mode=True*. 2025. [URL](#). (accessed: 27.08.2026).
- [64] Keras Team. *Local File Disclosure via HDF5 External Storage During Keras Weight Loading*. 2026. [URL](#). (accessed: 27.08.2026).
- [65] Kitware, Inc. *Building ParaView*. 2026. [URL](#). (accessed: 29.05.2026).
- [66] Mark Lambert. *Make Tests Runnable on Windows and with Newer h5dump, with Related Fixes*. 2026. [URL](#). (accessed: 27.08.2026; merged pull request).
- [67] Hugo Larcher et al. *Anatomy of a Frontier Lab Agent Intrusion: A Technical Timeline of the July 2026 Incident*. 2026. [URL](#). (accessed: 27.08.2026).
- [68] Google LLC. *CVE-2025-9905*. 2025. [URL](#). (accessed: 21.05.2026).
- [69] Google LLC. *CVE-2026-0897*. 2026. [URL](#). (accessed: 21.05.2026).
- [70] Google LLC. *CVE-2026-1669*. 2026. [URL](#). (accessed: 21.05.2026).
- [71] MathWorks. *HDF5 Files*. 2026. [URL](#). (accessed: 29.05.2026).
- [72] MITRE. *HDF5 CVE: Common Vulnerabilities and Exposures*. 2026. [URL](#). (accessed: 21.05.2026).
- [73] National Institute of Standards and Technology. *NIST Privacy Framework: A Tool for Improving Privacy through Enterprise Risk Management, Version 1.0*. Framework. Gaithersburg, MD: National Institute of Standards and Technology, Jan. 2020. [URL](#) (visited on 06/25/2026).
- [74] NeXus International Advisory Committee. *NeXus Data Format Manual*. 2026. [URL](#). (accessed: 29.05.2026).
- [75] Open Source Geospatial Foundation. *HDF5 – Hierarchical Data Format Release 5 (HDF5)*. 2026. [URL](#). (accessed: 29.05.2026).

- [76] pandas Developers. *IO tools: HDF5 / PyTables*. 2026. [URL](#). (accessed: 29.05.2026).
- [77] PyTables Developers. *PyTables Documentation*. 2026. [URL](#). (accessed: 29.05.2026).
- [78] takluyver. *Option(s) to Disable Various Kinds of External File Access*. 2026. [URL](#). (accessed: 27.08.2026).
- [79] TensorFlow Authors. *Commit 46e7f7f: Disable Loading HDF5 Plugins*. 2025. [URL](#). (accessed: 27.08.2026).
- [80] TensorFlow Authors. *Save, serialize, and export models*. 2026. [URL](#). (accessed: 29.05.2026).
- [81] The HDF Group. *Dynamically-loaded Plugins (H5PL)*. 2026. [URL](#). (accessed: 04.06.2026).
- [82] The HDF Group. *H5D__b12_filt_decode() at d8ec639*. 2026. [URL](#). (source snapshot: 25.08.2026; accessed: 28.08.2026).
- [83] The HDF Group. *H5EA__hdr_init() at d8ec639*. 2026. [URL](#). (source snapshot: 25.08.2026; accessed: 28.08.2026).
- [84] The HDF Group. *H5HF cache direct-block deserialization at d8ec639*. 2026. [URL](#). (source snapshot: 25.08.2026; accessed: 28.08.2026).
- [85] The HDF Group. *H5T__conv_struct Use After Free*. 2026. [URL](#). (accessed: 27.08.2026).
- [86] The HDF Group. *HDF5 2.0.0 source-artifact integrity check*. [URL](#). (accessed: 03.05.2026).
- [87] The HDF Group. *HDF5 assertion configuration in HDFCompilerFlags.cmake at d8ec639*. 2026. [URL](#). (source snapshot: 25.08.2026; accessed: 28.08.2026).
- [88] The HDF Group. *HDF5 Data Model and File Structure*. 2026. [URL](#). (accessed: 04.06.2026).
- [89] The HDF Group. *HDF5 Privacy Exposure (H5PE) Model*. 2026. [URL](#). (accessed: 14.07.2026).
- [90] The HDF Group. *HDF5 Security Policy*. 2026. [URL](#). (accessed: 29.05.2026).

- [91] The HDF Group. *HDF5 Was the Mechanism, Not the Vulnerability*. 2026. [URL](#). (accessed: 27.08.2026).
- [92] Trend Micro Zero Day Initiative. *TensorFlow HDF5 Library Uncontrolled Search Path Element Local Privilege Escalation Vulnerability*. 2026. [URL](#). (accessed: 27.08.2026).
- [93] Unidata Program Center. *NetCDF-4 File Format*. 2026. [URL](#). (accessed: 29.05.2026).

A. Findings Register

This appendix is the authoritative register of formal findings for this revision and maintains a separate evidence-and-scoping backlog. It applies the method in Section 1.7 to public evidence available through August 27, 2026. A formal finding is a concrete adverse-event scenario at a stated boundary; it is not a score assigned to a technology, feature, CVE count, recommendation, or project. Broad mechanisms that do not yet define an adverse consequence and affected population appear only in Table 33; they are not formal findings. Unless a record expressly says otherwise, the assessment did not independently reproduce the reported condition and did not assess the current installed population.

Table 31 records inherent-risk axes and tiers where evaluated, together with every finding's rating status. It does not imply that every version or deployment is affected today. A public fix, issue closure, or reporter statement is noted where relevant, but it is not credited as current residual-risk reduction without sufficient evidence of the implemented control and its coverage. An *NE* entry is an evidence or scoping gap, not a Low-risk conclusion.

Linked recommendation identifiers show intended control coverage only. Response routing is shown separately in Table 34. Implementation dependencies, effort, delivery sequence, ownership, acceptance evidence, and residual-risk decisions do not alter the inherent ratings below and are not populated in this findings register. The proposed work packages, dependencies, accountable roles, effort, and acceptance gates are maintained separately in Appendix C; the linked identifiers must not be read as completed planning records.

Table 31 – Formal findings and report-wide rating status

ID	Scenario shorthand	Dim.	I/E	Tier	Status	Conf.
CORE-SEC-001	Malformed on-disk metadata reaches memory-unsafe core-library paths in historically affected releases.	Security	I3/E2-E3	High	Provisional	Moderate
CORE-SEC-002	File-controlled work, allocation, traversal, or error paths terminate or exhaust an affected reader.	Security	I2/E2-E3	Moderate	Provisional	Moderate
CORE-TOOL-001	A malicious file triggers the reported h5dump use-after-free in HDF5 1.14.1-2 and earlier.	Security	I2-I3/E2	Moderate-High	Provisional	Moderate

Table 31 – Formal findings and report-wide rating status, continued

ID	Scenario shorthand	Dim.	I/E	Tier	Status	Conf.
CORE-PRIV-001	Sensitive plaintext metadata is disclosed when a file is shared under payload-only protection.	Privacy	I2-I3/E2	Moderate-High	Provisional	Moderate
CORE-SAFE-001	Storage exhaustion during write or close leaves an incomplete or corrupted file, with no library mechanism to detect or recover.	Safety	I2-I3/E3	Moderate-High	Provisional	Low
CORE-SAFE-002	Abrupt process or host termination during write leaves a file that is corrupted or not reopenable.	Safety	I2-I3/E3	Moderate-High	Provisional	Low
LONG-SAF-001	An archived file becomes unreadable or uninterpretable because a required filter, VFD, or VOL connector is no longer available.	Safety	I2-I3/E2	Moderate-High	Provisional	Low
APP-SEC-001	The affected Hugging Face dataset service followed attacker-selected external raw-storage paths with worker authority and returned local files.	Security	I3/E4	Critical	Rated	High
APP-SEC-002	Affected Keras legacy-HDF5 model loading executes embedded code despite <code>safe_mode=True</code> .	Security	I3/E2	High	Rated	High
APP-SEC-003	Affected Keras model loading follows attacker-controlled HDF5 external dataset references and discloses local files.	Security	I3/E3	High	Rated	High
APP-SEC-004	Affected Keras weight loading allocates from an extreme file-declared shape and exhausts memory.	Security	I2/E3	Moderate	Rated	High
APP-SEC-005	TensorFlow 2.17.0 loads HDF5 plugins from an uncontrolled search location, enabling local privilege escalation.	Security	I3/E2	High	Rated	Moderate
FMT-SAF-001	The normative specification is incomplete, ambiguous, or contradicted by the reference implementation, so conformance is not determinable.	Safety	I2-I3/E2-E3	Moderate-High	Provisional	Moderate
IND-SAF-001	HDF5 2.0/2.1 exact-width validation rejects a sufficient but nonminimal chunk encoding permitted by the specification.	Safety	I2/E2	Moderate	Rated	High
IND-SAF-002	Older permissive readers accept contradictory PureHDF 2.1.2 chunk metadata that HDF5 2.0 rejects after upgrade.	Safety	I2/E2	Moderate	Rated	High

A.1. Correspondence with the SSP SIG Registries

Table 32 relates each record above to the SSP SIG threat, hazard, and privacy registries. Three cautions govern its use, and all three are load-bearing.

First, this is an **editorial correspondence established by this report**. Neither project has adopted it, and a row asserts only that the two records describe a related mechanism. Second, **it is not a rating comparison**: the registries score Severity $\times$ Likelihood on a 1–25 scale with five bands, which Section 1.9 establishes is not convertible to or from the tiers used here. Third, **“no direct counterpart” runs in both directions and means neither project is wrong**. The registries model the library and format; this report additionally rates application, service, and independent-implementation boundaries that the registries do not cover, and the registries enumerate operational safety hazards this report did not examine.

Table 32 – Editorial correspondence between this register and the SSP SIG registries

This report	SSP SIG entry	Note
CORE-SEC-001	ATK-001-ATK-008	The registry decomposes the memory-unsafe parser class into eight mechanism entries; this report treats them as one shared adverse event with a common scope and evidence base. The decomposition is the better basis for the CORE-S1 invariant tranches.
CORE-SEC-002	ATK-009, ATK-010, ATK-012	Resource exhaustion, null and uninitialized dereference, and I/O amplification from pathological chunk layouts. The promoted assert-only instance sits under ATK-010.
CORE-TOOL-001	ATK-007 (mechanism only)	The registry rates lifetime defects as a library class and does not separate official-tool exposure. This report rates the tool boundary distinctly.
CORE-PRIV-001	EXP-002; also HAZ-004	The one pairing where both projects rate the same mechanism and land in comparable bands.
CORE-SAFE-001	HAZ-005	Same scenario. The registry supplies a hazard analysis and rating; this report declines to rate pending RM-2D measurement. See Table 6.
CORE-SEC-003	ATK-010 (partial)	No registry entry addresses build-mode-dependent enforcement directly.
CORE-SAFE-002	HAZ-001; also HAZ-002, HAZ-006, HAZ-012, HAZ-015	HAZ-001 is the registry’s highest-rated safety entry. This report registers one finding where the registry has five; the durability cluster around flush semantics, concurrency, and recovery tooling is not separately rated here.

Table 32 – Editorial correspondence between this register and the SSP SIG registries, continued

This report	SSP SIG entry	Note
LONG-SAF-001	HAZ-014 (partial)	The registry covers missing filter plugins. Neither registry covers VFD or VOL archival dependency; that scope originates here.
APP-SEC-001	No direct counterpart	The registries contain no confused-deputy or authority-boundary entry. EXP-008 and EXP-009 cover external-link disclosure as a privacy mechanism, which is related but not the same scenario.
APP-SEC-002	No direct counterpart	Application-level deserialization layered on HDF5 is outside the registries' scope.
APP-SEC-003	EXP-009 (partial)	External references revealing or reaching unintended targets; the registry frames it as disclosure, this report as an authority-boundary failure.
APP-SEC-004	ATK-009	Same mechanism, rated here at the application boundary that performs the allocation.
APP-SEC-005	ATK-011	The clearest methodological disagreement in the pair of documents; see Table 5.
FMT-SAF-001	No direct counterpart	The hazard model has an H6 parser-ambiguity family and H5II has II parser/spec divergence, but neither registry rates defects in the normative artifact itself.
IND-SAF-001, IND-SAF-002	No direct counterpart	The registries have a parser-ambiguity hazard family but no independent-implementation or reader/writer-conformance entries. This is material this report contributes.
EXT-PRIV-001	EXP-004-EXP-007; also HAZ-003	The registry has already enumerated four bounded extension exposures where this report has one unrated theme. RM-2E should adopt them as its enumeration.
SUP-SAFE-001	ATK-013, HAZ-014	Distribution compromise and missing-filter portability. ATK-013 is a security scenario this report never developed; RM-2F should adopt it rather than re-derive one.

A.2. Evidence and Scoping Backlog

The observations below identify work needed to determine whether additional formal findings exist. They have no impact or exposure axes, risk tier, response urgency, or implied Low-risk status.

Table 33 – Observations requiring a concrete adverse-event scenario

ID	Observation	Evidence needed for a finding
CORE-SEC-003	Candidate deserializer invariants derived from in-file bytes are masked inside <code>assert(...)</code> expressions and therefore may not be enforced in <code>-DNDEBUG</code> release builds. Issue #87 identifies seven component families and estimates 50–100 candidates, but does not provide a verified site catalog or consequence analysis; a fixed develop-source sample corroborates the build mechanism and representative sites, not their runtime consequences [57, 87, 82, 83, 84]. One instance has been promoted into CORE-SEC-002 on reported reproduction: a v2 B-tree <code>record_size</code> of zero reaching an integer division, SIGFPE on a release build [33]. A second candidate is reported for the cache-image LRU rank relation without a measured release-build consequence [31]. The remaining population is unquantified, and this record stays open.	For each supported source line and build profile, record the source site, file-byte provenance, non-asserting dominating guard if any, release reachability, first security-relevant sink, paired debug and <code>-DNDEBUG</code> fixture result, affected versions, and consequence. Promote each supported adverse-event scenario into CORE-SEC-001, CORE-SEC-002, or a new formal finding as the evidence requires; a redundant or unreachable assertion needs a documented negative disposition. Begin from the promoted instance’s own sibling audit, which names further unaudited assertions in the same B-tree header initializer and in fractal-heap initialization.

Table 33 – Observations requiring a concrete adverse-event scenario, continued

ID	Observation	Evidence needed for a finding
EXT-PRIV-001	Extensions can move data and metadata across storage, cache, logging, network, and executable-plugin boundaries.	Identify a deployment, sensitive-data population, unauthorized recipient or retention event, affected scope, and resulting harm.
SUP-SAFE-001	Downstream builds and work-flows can disagree about ABI, linkage, filters, plugins, VFDs, and recurrence evidence. parallel features, or file locking.	Identify a concrete producer/consumer profile pair, affected population, failure mode, recoverability, and recurrence evidence.

A.3. Response Routing

The routing below follows the default rule in Section 1.7. Where current residual risk was not evaluated, it uses inherent risk as the conservative provisional baseline; a tier range is routed at its upper bound. Routing is conditional on confirming the stated affected scope and is not a claim that every present deployment needs the same action. The target horizons begin when an affected deployment or equivalent exposure is identified. They are assessment recommendations, not stakeholder-approved delivery commitments, and do not incorporate effort or dependencies. In particular, Hugging Face reports that it stopped wrongly processing HDF5 external references, but this assessment did not validate the implemented control or assign a current residual tier. Phases 0–2 of Appendix C operationalize these clocks; later structural work neither replaces nor extends them.

Table 34 – Provisional response urgency and target horizons

Urgency and horizon	Findings	Decision rule for this register
Immediate—before the next untrusted processing job and no later than 24 hours after identification	APP-SEC-001	Immediately contain any equivalent service that still automatically follows file-selected external resources with access to sensitive local files. The historical incident rating is not a current rating of Hugging Face.

Table 34 – Provisional response urgency and target horizons, continued

Urgency and horizon	Findings	Decision rule for this register
Near-term—within 30 days	CORE-SEC-001, CORE-TOOL-001, CORE-PRIV-001, CORE-SAFE-001, CORE-SAFE-002, FMT-SAF-001, APP-SEC-002, APP-SEC-003, APP-SEC-005	Identify affected releases and exposed deployments, apply available fixes or containment, minimize disclosed metadata, and verify the trust boundary. For the two write-side safety findings the Near-term obligation is <i>tested operational containment</i> — capacity headroom and monitoring, flush and checkpoint discipline, a rehearsed recovery procedure, and an independent copy of data that cannot be regenerated. The library changes in CORE-SAFE-S1 and CORE-SAFE-S2 are structural and follow the Planned and Phase 3 horizons. For FMT-SAF-001 the Near-term obligation is to <i>adjudicate the already-identified disputes</i> — rule on each, record the ruling, and change whichever artifact was wrong — not to build the machine-readable specification, which is the Phase 3 programme in FMT-S1.
Planned—within 90 days or the next supported release or migration window, whichever is sooner	CORE-SEC-002, APP-SEC-004, IND-SAF-001, IND-SAF-002, LONG-SAF-001 (by recorded departure)	Plan treatment and acceptance testing; expedite where an affected production pipeline, archive, sensitive-data population, or service is identified. LONG-SAF-001 routes here rather than at its Moderate-High upper bound; the departure and its rationale are recorded below.

A.4. Evidence Actions Without Assigned Urgency

Items CORE-SEC-003, EXT-PRIV-001, and SUP-SAFE-001 require the scenario evidence listed in Table 33. These are evidence actions, not a fifth response-urgency category.

CORE-SAFE-001 appeared in this category in an earlier revision and no longer does. The change is explained in its record below: the evidence gap was misiden-

tified as an absence of knowledge about the library's behaviour when it is in fact a documented absence of a mechanism, which bounds the exposure axis rather than leaving it open.

Recorded departure from the default routing map for LONG-SAF-001.

Section 1.7 maps a Moderate-High range, routed at its upper bound, to *Near-term*. LONG-SAF-001 is routed *Planned* instead. Rationale: the finding states a decadal assessment horizon, and decision rule 7 forbids directly ordering ratings that use materially different horizons against those that do not. The harm accrues gradually, is not acute in any identified present deployment, and is prevented by decisions taken at write and archive time rather than by an urgent change to a running system. A 30-day treatment clock would therefore measure nothing. This departure requires an accountable decision at adoption; if an archive is identified whose readers are already failing, that scoped instance is expedited and rated without a horizon qualifier.

A.5. Core Library, Tool, and Format Findings

CORE-SEC-001 — Memory-unsafe processing of malformed metadata

Scenario. An untrusted originator supplies a crafted HDF5 file to a process using a historically affected HDF5 C library. Insufficiently validated file-derived sizes, counts, offsets, message state, layout state, filter parameters, or datatype state reach allocation, copy, object-construction, or cleanup paths and cause memory corruption, including credible process compromise for the variants whose primary records support that consequence.

Scope and evidence. The assessed population consists only of deployments matching an affected version-and-path pair represented in Appendix D and processing untrusted files. The H5FS, H5C, H5O, H5T, H5Z, heap, dataspace/layout, and vector-copy variants are evidence for the shared adverse event, not separately scored objects. A detailed primary report for HDF5 1.8.16 traces file-controlled array rank to an out-of-bounds heap write and possible code execution in the

consuming application [26]; the cited later fixes demonstrate early sanity checks, bounds and overflow tracking, and controlled rejection replacing unsafe behavior [40, 35, 34, 36]. This record excludes termination-only and resource-consumption variants assigned to CORE-SEC-002 and the tool-specific UAF assigned to CORE-TOOL-001. It does not assert that every CVE had code-execution impact or that a current HDF5 2.x release contains every historical condition.

Rating basis. I3 reflects credible process compromise or major loss of process integrity in the evidenced variants. E2 applies to interactive or otherwise constrained file processing; E3 applies where an affected deployment automatically ingests untrusted files through a common workflow. Both produce a High tier, while exact version and deployment prevalence remain incomplete. The rating is therefore **I3/E2-E3, High, Provisional, Moderate confidence**. Current release and installed-population residual risk were not evaluated.

Linked recommendations. CORE-S0-CORE-S7 and CORE-S12; the primary structural controls are CORE-S1-CORE-S3.

CORE-SEC-002 — Uncontrolled work and resource consumption

Scenario. A crafted, truncated, or inconsistent file reaches affected decode, traversal, arithmetic, allocation, recursion, or lifecycle paths and causes an assertion, null dereference, divide-by-zero, leak, stack exhaustion, or uncontrolled resource consumption, interrupting the consuming process or workflow.

Scope and evidence. The assessed population consists only of deployments matching an affected version-and-path pair in Appendix D and processing the triggering file. This record includes termination and resource-consumption outcomes without evidenced memory corruption; it excludes the memory-unsafe variants assigned to CORE-SEC-001 and the tool-specific UAF assigned to CORE-TOOL-001. Exact fixed versions across the represented paths and the present installed population are not established. The file-space repair is one direct example of converting an assertion reached from corrupted metadata into a controlled error [35, 40].

Confirmed instance promoted from CORE-SEC-003. One assert-masked deserializer invariant has been carried to the standard that Section 1.7 requires for a rated scenario, and is recorded here rather than left in the evidence backlog. A version-2 B-tree header declaring `record_size==0` reaches an integer division while the leaf record capacity is derived; the only guard is `assert (cparam->rrec_size>0)` in `H5B2hdr.c`, which is absent from release builds compiled with `-DNDEBUG`. The reported measurement records SIGFPE on a shipped `h5dump-H` against HDF5 2.3.0, on the public API path `H5Fopen → H5Gopen2 → H5Gget_info`, on both x86-64 and i386 builds, with the condition re-checked against a later source state [33].

This satisfies the three conditions CORE-SEC-003 had left open for its population: the assertion is the sole guard, it is release-reachable in a supported version, and the adverse outcome is observed rather than inferred. Under RM-1E, a confirmed adverse path is routed by its consequence without waiting for the catalog; the consequence here is controlled-process termination, which is this record's scenario rather than CORE-SEC-001's. The instance is therefore promoted into CORE-SEC-002.

Two qualifications are part of the record. The measurement is *source-reported*: this assessment did not independently reproduce it, and the reporting project states the affected version range as *not established*, having measured one release without version archaeology. Second, a related variant — a leaf capacity of exactly zero rather than a division by zero — is reported to produce intermittent heap corruption under an older h5py-linked library, which if characterized would belong to CORE-SEC-001 instead. That variant is expressly recorded by its reporters as observed but not characterized, and this report does not rate it.

Rating basis. I2 reflects a meaningful but ordinarily recoverable process or workflow interruption; a mission-level outage requires a separate deployment-specific record. E2 applies to interactive or otherwise constrained file processing; E3 applies where an affected deployment automatically ingests untrusted files through a common workflow. Both produce a Moderate tier, while incomplete version prevalence makes the assessment provisional. The rating is **I2/E2-E3**,

Moderate, Provisional, Moderate confidence. Current release and installed-population residual risk were not evaluated.

The promoted instance does not change the tier: a confirmed termination path is the I2 consequence already assigned. It does not raise confidence to High either, because the affected version boundary — a material rating assumption under Section 1.7 — remains unestablished. What it does change is the *status of the mechanism*: release-build absence of a file-derived check is no longer a hypothesis for this record, and the RM-1B negative test for it is a concrete, available deliverable rather than one contingent on the RM-1E catalog.

Linked recommendations. CORE-S0-CORE-S8, CORE-S12, APP-S1, and APP-S2.

SSP SIG correspondence. ATK-009; the promoted instance also corresponds to ATK-010. See Section 1.9 for why the ratings differ.

CORE-TOOL-001 — h5dump use-after-free

Scenario. An attacker supplies a malicious HDF5 file to a user or automated workflow that invokes an affected h5dump; the file triggers the reported H5T__conv_struct heap use-after-free and may compromise or terminate the inspection process.

Scope and evidence. The HDF Group advisory identifies HDF5 1.14.1-2 and earlier as affected and reproduces AddressSanitizer-detected termination when the supplied file is processed by the utility [28, 85]. It states that practical code-execution exploitability is unknown. This is a tool finding, not evidence that every libhdf5 consumer reaches the same path.

Rating basis. The demonstrated termination supports I2; credible but unverified process compromise from the UAF supports I3 as the upper bound. E2 reflects the affected-version, crafted-file, and explicit inspection preconditions. The rating is **I2-I3/E2, Moderate-High, Provisional, Moderate confidence**. Current installed-population residual risk was not evaluated.

Linked recommendations. CORE-S6, CORE-S9, CORE-S12, and APP-S2.

CORE-PRIV-001 — Plaintext metadata disclosure

Scenario. An HDF5 file containing sensitive object names, attributes, member labels, comments, user-block content, provenance, or structural relationships is shared with a recipient who is authorized for a protected payload but not for that context. Direct HDF5 or byte-level inspection reveals the unintended metadata.

Scope and evidence. This finding is limited to files that actually contain sensitive plaintext metadata and a sharing boundary that protects only the nominal payload. The format's self-describing structure establishes the mechanism; the Privacy Exposure Model identifies metadata and structural exposure as privacy threat classes [88, 89].

Rating basis. I2 reflects bounded sensitive-context disclosure; I3 is plausible where the same mechanism reveals high-sensitivity context or substantially affects a defined population. E2 reflects the need for both sensitive metadata and a mis-scoped disclosure boundary. Data sensitivity, affected population, and workflow prevalence were not measured, so the rating is **I2–I3/E2, Moderate–High, Provisional, Moderate confidence**. Deployment-specific findings should replace this range when those facts are known. Current installed-population residual risk was not evaluated.

Linked recommendations.

CORE-PRIV-S1 and CORE-PRIV-S3–CORE-PRIV-S4.

CORE-SAFE-001 — Storage-exhaustion write failure

Scenario. An application writes an HDF5 file to storage that fills during the write, flush, or close sequence. The write fails with `ENOSPC`. The library provides no mechanism by which the application can be notified at the point of failure, no defined recovery procedure, and no way to mark the resulting file as suspect. The file is left incomplete or internally inconsistent, and data that has not been independently preserved may be lost.

Scope and evidence. The assessed population is any deployment that writes HDF5 files to storage which can reach capacity — which is not a narrow population. The controlling evidence is architectural rather than statistical, and it is a maintainer statement in the cited design discussion: *there is no HDF5 story for ENOSPC, and no callback for the condition that the virtual file driver has just encountered it*. The same discussion records that the current practice is a workaround — configuring the metadata cache to discard on evict — rather than a supported control, and enumerates four candidate treatments: space reservation via `posix_fallocate`, an explicit poisoned-file state, a pre-flush capacity check, and a virtual-file-driver error callback that lets the caller choose to retry, fail fast, or panic [20]. The condition also appears in general file-corruption reports across the community forum [18, 8].

Why this is no longer NE. An earlier revision recorded this finding as NE on the ground that the recovery path was not bounded. That was a category error, and correcting it is the substantive change in this record. The recovery path *is* bounded: it does not exist. That is a verifiable property of the published API surface, not an unmeasured quantity. It also determines the exposure axis, because a control that is absent cannot intermittently succeed — every occurrence of the triggering condition during a write reaches the unsafe state. Exposure therefore reduces to how often storage fills during a write, which is an ordinary operational fact rather than an open question about HDF5. Treating the documented absence of a control as an absence of evidence understated this finding.

Rating basis. I2 reflects a recoverable loss where the workload can be rerun or an independent copy exists. I3 reflects major loss or corruption where it cannot; I4 is not assigned, because irreversible loss of unique or mission-critical records is credible but is not established by the cited evidence, and decision rule 1 requires the greatest *supported* impact rather than a worst case. E3 reflects a common workflow with few material preconditions. The rating is **I2–I3/E3, Moderate–High, Provisional, Low confidence**. Confidence is Low because no failure sequence, durable-state result, recoverability outcome, or population has been measured; RM–2D remains required and would move confidence rather than the tier. Current residual risk was not evaluated.

Response. Routed Near-term at the upper bound. The Near-term obligation is tested operational containment — capacity headroom, free-space monitoring and alerting, the documented cache workaround where applicable, and an independent copy of data that cannot be regenerated — not the library change. The library treatment in CORE-SAFE-S1 is structural and follows the Planned and Phase 3 horizons; under Section 1.7 that split is normal and does not extend the containment clock.

Linked recommendations. CORE-SAFE-S1; CORE-S12 for the durability metrics.

SSP SIG correspondence. HAZ-005, whose accident chain and safety constraints SC-1 to SC-3 should be adopted as the RM-2D hypotheses.

CORE-SAFE-002 — Corruption from abrupt termination

Scenario. A writer process is killed, crashes, or loses power while an HDF5 file is open and metadata updates are in flight. Because in-memory metadata-cache state and the durable on-disk state can disagree, and because the format has no journaling or write-ahead log, the file may be left internally inconsistent: not reopenable, reopenable only after `h5clear`, or reopenable but missing or misrepresenting recently written data. Data written since the last consistent state may be lost, and the inconsistency may not be detected until a later read.

Scope and evidence. The assessed population is any deployment holding an HDF5 file open for write across a window in which the process or host can fail — long acquisitions, simulation checkpointing, and continuous logging in particular. The evidence is a recurrence pattern in the community forum spanning sixteen years, from requests to recover corrupted files and questions about file state after a crash, through reports that flushing itself sometimes corrupts files, to a 2024 design discussion in which The HDF Group states that a crash or failure “can lead to data loss or corruption of the file” and describes restarting work on crash-proofing through metadata journaling, write-ahead logging, SWMR-derived mechanisms, and checkpointing [18, 16, 17, 11, 12, 8, 15, 9]. That discussion also records a practitioner report that trapping signals and closing the library prevents

the worst outcomes and usually leaves files intact, with `h5clear` still needed at times — which is compensating-control evidence, not a supported guarantee.

Rating basis. I2 reflects a recoverable interruption where the file reopens after `h5clear` or the workload can be rerun. I3 reflects major loss or durable corruption where neither holds. E3 reflects a common workflow — any long-running write — with few material preconditions; abrupt termination requires no adversary and no unusual configuration. The rating is **I2–I3/E3, Moderate–High, Provisional, Low confidence**. Confidence is Low because the assessment measured no failure injection, established no version boundaries, and quantified no population: the evidence establishes recurrence and maintainer acknowledgement, not distribution of outcomes.

Relationship to CORE–SAFE–001. The two are deliberately separate records, because the trigger, the available containment, and the structural treatment all differ. Storage exhaustion is foreseeable and can be preflighted; abrupt termination cannot. Rating them together would obscure both.

Response. Routed Near-term at the upper bound, with the same containment-versus-structure split as CORE–SAFE–001: the Near-term obligation is flush and checkpoint discipline, a rehearsed and tested recovery procedure including `h5clear`, and independent preservation of data that cannot be regenerated. The structural treatment in CORE–SAFE–S2 is a multi-release programme.

Linked recommendations. CORE–SAFE–S2; CORE–S12 for the durability metrics.

SSP SIG correspondence. HAZ–001, the highest-rated entry in that registry, together with HAZ–002, HAZ–006, HAZ–012, and HAZ–015, which this report does not separately register.

A.6. Application-Boundary Findings

APP-SEC-001 — External raw storage confused deputy

Scenario. In the affected July 2026 Hugging Face deployment, a remote party supplied a valid HDF5 file whose dataset declared local files as external raw storage. The dataset-processing worker opened those paths with its own filesystem authority and returned their bytes to the requester, disclosing its environment, credentials, and source code.

Scope and evidence. Hugging Face reports that this vector performed file disclosure without executing code; the separate Jinja2 injection supplied code execution. The HDF Group identifies external raw storage as an intentional documented capability and the service as the authority-bearing confused deputy, not an HDF5 parser failure [67, 91]. The rating is confined to the affected service at incident time and to equivalent deployments with the stated preconditions; it is not an ecosystem-prevalence claim.

Rating basis. I3 reflects actual unauthorized resource access and substantial disclosure of secrets, credentials, and implementation details. E4 reflects normal automatic processing of an ordinary attacker upload, transparent reference following, and a direct, repeatable disclosure path within that deployment. The inherent rating is **Critical, Rated, High confidence**.

Current residual and response. Hugging Face states that its renderer no longer wrongly processes HDF5 external references. Public evidence is insufficient to validate control design and coverage, so current residual risk for that service was not evaluated and no residual tier is assigned. The h5py discussion records laborious application-level detection, a sandbox recommendation, and the decision to seek upstream control; HDF5 issue #6618 records the resulting request for unified controls over external data, external links, and virtual sources. Proposals discussed there are not treated as shipped controls [25, 78]. Equivalent still-exposed deployments require Immediate containment.

Linked recommendations. APP-S1, APP-S2, APP-S4, and CORE-S4.

APP-SEC-002 — Keras legacy-HDF5 code deserialization

Scenario and scope. An attacker supplies a crafted legacy `.h5/.hdf5` model to Keras 3.0.0 through 3.11.2. A victim loads it expecting `safe_mode=True` to prevent unsafe reconstruction, but a Lambda layer's serialized code executes in the loader process [68, 63].

Rating basis. I3 reflects arbitrary code execution in the loader's authority. E2 reflects the legacy format, crafted Lambda content, affected version, and passive load preconditions. The rating is **High, Rated, High confidence**.

Linked recommendations. APP-S3. This is application deserialization layered on HDF5, not a core HDF5 parser finding.

APP-SEC-003 — Keras external-dataset local-file read

Scenario and scope. A remote attacker supplies a crafted `.keras` model to Keras $\geq 3.0.0$, $< 3.12.1$ or $\geq 3.13.0$, $< 3.13.2$. During model loading, an embedded HDF5 external dataset reference selects a local path and discloses a file readable by the loader process [70, 64].

Rating basis. I3 reflects unauthorized local-resource access and potentially substantial sensitive-information disclosure. E3 reflects a remotely supplied artifact, few technical preconditions, and a normal model-load action, while remaining bounded to the affected versions and model-loading surface. The rating is **High, Rated, High confidence**.

Linked recommendations. APP-S1-APP-S4 and CORE-S4.

APP-SEC-004 — Keras shape-driven memory exhaustion

Scenario and scope. A remote attacker supplies a crafted `.keras` archive to Keras $\geq 3.0.0$, $\leq 3.12.0$ or $\geq 3.13.0$, $< 3.13.2$. Its valid `model.weights.h5` declares an extremely large dataset shape; weight loading allo-

cates without an adequate budget, exhausts memory, and crashes the Python interpreter [69, 62].

Rating basis. I2 reflects the evidenced process and workflow interruption; no broader outage or durable data loss is assumed. E3 reflects low-complexity, remotely supplied content crossing a normal model-loading boundary, subject to the victim or service loading the artifact. The rating is **Moderate, Rated, High confidence**.

Linked recommendations. APP-S1-APP-S3, CORE-S3, and CORE-S4.

APP-SEC-005 — TensorFlow plugin search-path escalation

Scenario and scope. A low-privileged local attacker on a TensorFlow 2.17.0 installation places a plugin in an unsecured location searched by the HDF5 integration. TensorFlow loads the plugin and executes its code with the target user's authority [59, 92, 79].

Rating basis. I3 reflects privilege escalation and arbitrary code execution. E2 reflects the required local foothold, affected version, writable searched location, and plugin-load path. The condition is supported by the coordinated advisory and vendor fix, but the report did not reproduce it. The rating is **High, Rated, Moderate confidence**.

Linked recommendations. APP-S5 and CORE-S10.

A.7. Independent-Implementation and Interoperability Findings

IND-SAF-001 — Sufficient nonminimal chunk encoding rejected

Scenario. A workflow upgrades its reader to an HDF5 2.0/2.1 release containing the exact-width check—confirmed in 2.0.0, 2.1.0, and 2.1.1—and opens a version-4 chunk layout whose encoded dimension width is sufficient but nonminimal. Validation rejects the file even though the format did not require the smallest possible width, interrupting access to data that earlier readers accepted.

Scope and evidence. HDF5 issue #6409 distinguishes the specification-permitted encoding from an undersized invalid encoding and records effects on independent writers. The merged reader-side change accepts any sufficient width and was tracked against the HDF5 2.2.0 milestone; this report does not claim that a specific release artifact or backport contains it without release-level verification [4, 29, 22, 66].

Rating basis. I2 reflects a meaningful, bounded interoperability and data-access failure. E2 reflects the specific layout version, nonminimal encoding, and reader-upgrade preconditions. The rating is **Moderate, Rated, High confidence** for the historical affected scope. Current residual risk was not evaluated, and no residual tier is assigned.

Linked recommendations. CORE-S1, CORE-S6, CORE-S8, and IND-S1.

IND-SAF-002 — Contradictory PureHDF chunk metadata

Scenario. A production or archival workflow upgrades from a tolerant HDF5 1.x reader to HDF5 2.0 and reads an affected compressed file produced by PureHDF 2.1.2. The file's version-4 chunk layout contradicts its datatype metadata; the stricter reader correctly rejects the affected datasets, interrupting processing and stranding existing data until the reader is pinned or the files are repaired or regenerated.

Scope and evidence. In the supplied production-derived file, the HDF5 maintainer analysis found float64 datatype element size 8 while the chunk layout recorded stored element size 4. Older HDF5 1.10.10 and 1.14.6 readers accepted the bytes; HDF5 2.0 rejected them. The issue was closed as malformed writer output, and the PureHDF discussion records the writer-side response [2, 1]. This condition is distinct from the sufficient nonminimal width in IND-SAF-001; permissive acceptance is not evidence of conformance.

Rating basis. I2 reflects actual production workflow interruption and a persistent but repairable compatibility problem; no irreversible data loss is evidenced. E2 reflects the confirmed writer version, affected layout, compressed dataset, and reader-upgrade preconditions. The rating is **Moderate, Rated, High confidence**.

A technical metadata-repair path was proposed and older readers remained a workaround, although the reporter said rewriting affected production files was not operationally feasible in that pipeline. Later PureHDF output restored production operation, but the inventory and repair status of existing files are unknown. Current residual risk for the legacy corpus was not evaluated, and no residual tier is assigned.

Linked recommendations. CORE-S1, CORE-S6, CORE-S8, and IND-S1.

A.8. Format and Specification Findings

FMT-SAF-001 — Conformance is not determinable

Scenario. An independent implementation writes or reads an HDF5 structure according to the published file-format specification. The reference library behaves differently. Because the specification and the reference implementation disagree — and because no procedure exists to rule on which is authoritative — neither party can establish whether the file conforms. Files are rejected, silently reinterpreted, or accepted by one implementation and not another, and the dispute is resolved by whoever concedes rather than by reference to a normative artifact.

Scope and evidence. Two instances are documented, and they fail in different directions, which is what makes the class rather than either case the finding.

The library was non-conforming.

Issue #6409. The format required an encoded chunk-dimension width sufficient to hold the value and did not require the smallest possible width; HDF5 2.0.0, 2.1.0, and 2.1.1 rejected a sufficient but nonminimal encoding anyway. The specification was correct and the reference reader was wrong, and the reader was changed [4, 29].

Which artifact is wrong is undetermined.

Issue #6616. The specification documents the group info message as optional; the C library raises a “message type not found” error when it is absent, so it cannot modify a file written without one. The reporter states the two possible resolutions explicitly — the library should fall back to default group-info

values, or the specification should be corrected to say the message is required — and at the evidence cutoff the issue is open, labelled a documentation bug, assigned, and milestone, with no ruling recorded [5].

Both were discovered the same way: an independent implementation exercised a construct and diverged from the reference library. Neither was found by inspecting the specification, and neither would have been visible at all if libhdf5 were the only implementation, because a sole implementation's behaviour is indistinguishable from the format.

Why this is a distinct finding. IND-SAF-001 and IND-SAF-002 rate the *consequences* of two particular divergences — data that became unreadable after a reader upgrade. This record rates the *condition that produces them*: a normative artifact that cannot settle the question. It is registered separately because its treatment is different in kind. Repairing a file or fixing a reader does not make the next ambiguity determinable, and the report's own method assumes determinability: Section 1.6 states that where implementations disagree the analysis distinguishes the normative requirement from writer behaviour and reader validation. Issue #6616 is a case where that distinction cannot currently be drawn, which is a limitation of the method as well as a finding about the format.

Rating basis. I2 reflects a bounded, repairable interoperability or data-access failure — the evidenced outcome in both instances. I3 reflects consequential loss where an archive or a scientific result depends on the contested interpretation and the dispute is not resolved in time; this is credible but not evidenced, so the range is stated rather than the upper bound asserted. E2 reflects the preconditions for any single ambiguity: a construct must be exercised, an implementation must diverge, and files must cross implementations. E3 reflects that the condition now recurs — two instances surfaced within months of each other in 2026 — and that exposure rises with the number of independent implementations, since each ambiguity is resolved independently by every one of them. The rating is **I2–I3/E2–E3, Moderate–High, Provisional, Moderate confidence**. Confidence is Moderate because both instances rest on primary evidence but no systematic survey of the specification has been performed; the extent of the defect population is unmeasured.

A second-order effect worth recording. In #6409 the independent writer changed its output to match the reference reader, even though its original output was specification-permitted. That is the rational local decision and the corrosive global one: under-specification makes implementations converge on the reference library's behaviour rather than on the specification, so the specification loses normativity exactly as the number of implementations that should be reinforcing it grows. A dispute resolved by concession leaves the ambiguity in place for the next implementation to rediscover.

Response. Routed Near-term at the upper bound. The Near-term obligation is *adjudication of the identified disputes*: for each open case, rule which artifact is authoritative, record the ruling and its rationale, and change the losing artifact. #6616 is the immediate instance and already has an owner and a milestone. The systematic treatment — a machine-readable normative specification and a standing procedure for future disputes — is FMT-S1 and runs on the Phase 3 horizon. Adjudicating two open cases does not require the programme, and the programme does not excuse leaving them open.

Linked recommendations. FMT-S1; IND-S1 for the contested-reading fixtures; CORE-S1, whose invariant registry must record which invariants are specification-derived and which are reference-implementation-derived, because the two carry different authority.

SSP SIG correspondence. No direct counterpart. The hazard model has a parser-ambiguity hazard family (H6) and the independent-implementation model has **I1** parser/spec divergence, but neither registry contains an entry for defects in the normative artifact itself. This scope originates here.

A.9. Long-Term Accessibility Findings

LONG-SAF-001 — Extension-dependent loss of interpretability

Scenario. An HDF5 file is retained in an archive. Reading or interpreting it requires a filter plugin, virtual file driver, or VOL connector that is not part of the core library. At some later point that extension is unavailable for the reader's

platform, unbuildable against a current toolchain, unmaintained, or simply unidentifiable from the file itself. The bytes survive; the data does not. Depending on the extension class, the outcome ranges from a dataset that cannot be decompressed to a file whose contents were never fully self-contained in the first place.

Assessment horizon. This finding states an assessment horizon of **ten years or more** from the date of writing. The horizon is material: at a one-year horizon the scenario is largely hypothetical, and at a multi-decade horizon it approaches certainty for any file depending on a single-maintainer extension. Under decision rule 7 of Section 1.7, this rating must not be directly ordered against ratings that state no horizon.

Scope and evidence. The assessed population is archived files that depend on a non-core extension for read or interpretation. The immediate mechanism is already observable at short horizons: community reports record users unable to read data because a required compression filter was absent from their build or platform, across a fifteen-year span and several filter families [19, 13, 10, 14]. The archival extrapolation — that availability decreases rather than increases over a decade — is reasoned inference, not measurement, and is the reason confidence is Low.

The three extension classes are not equally exposed, and the report does not claim they are. A missing *filter* is the best-evidenced and most bounded case: the file identifies the filter by numeric identifier, so the dependency is at least discoverable, and re-implementation is possible in principle. A missing *VFD* affects whether the bytes can be reached at all. A missing *VOL connector* is the most severe and the least evidenced, because the data may never have been in an HDF5 file in the first place — the connector may have been mapping to an entirely different store, in which case “preserving the HDF5 file” preserves nothing.

Rating basis. I2 reflects a bounded interoperability failure where the extension can be recovered, rebuilt, or re-implemented and the data is then readable. I3 reflects consequential loss where it cannot and the records are unique or scientifically significant. E2 reflects the preconditions — the file must depend on a non-core extension, and the extension must have become unavailable within the horizon — which is realistic but not universal, since a large share of HDF5

data uses only core features or `gzip`. The rating is **I2–I3/E2, Moderate–High, Provisional, Low confidence**, at the stated horizon.

Response. Routed *Planned* by recorded departure from the default map; see the rationale in Section A.4. The controlling point is that the effective treatment window is at *write and deposit* time, not at read time: once an archive holds files whose extension is gone, the options are recovery archaeology or loss. Treatment is therefore a producer-side and curator-side practice, delivered through LONG–S1.

Linked recommendations. LONG–S1; IND–S1 for the fixture and reader-matrix machinery it reuses; CORE–S11 and APP–S6 for the provenance inputs that make a dependency discoverable.

SSP SIG correspondence. HAZ–014 covers the filter-plugin portability slice at Moderate. **Neither registry covers VFD or VOL archival dependency**, and this report contributes that scope.

A.10. Evidence-Backlog Record Details

EXT-PRIV-001 — Extension-mediated privacy boundary observation

Observation and evidence gap. VOL connectors, filters, and VFDs can move data or metadata into remote stores, caches, temporary artifacts, logs, networks, or dynamically loaded code and thereby change the privacy boundary described in Section 4. The report does not define a concrete deployment, sensitive-data population, unauthorized recipient, retention period, or harm magnitude for this general scenario [89].

Register treatment. This mechanism is not a formal finding. Impact, exposure, tier, confidence, and response urgency are not assigned; this does not dispute the documented mechanism.

Linked recommendations. CORE–PRIV–S2, CORE–PRIV–S3, VOL–S1–VOL–S4, FILTER–S1–FILTER–S5, and VFD–S1–VFD–S4.

SUP-SAFE-001 — Downstream build and feature mismatch observation

Observation and evidence gap. A downstream environment loads an HDF5 build whose ABI, linkage, filters, plugins, VFDs, parallel features, or file-locking behavior differs from the profile assumed by a file or workflow, causing a read failure or operational interruption. Section 7 establishes the breadth and configuration mechanisms but does not establish an affected population, frequency, bounded version set, or realized impact.

Register treatment. This theme is not a formal finding. Impact, exposure, tier, confidence, and response urgency are not assigned; a material workflow interruption remains a plausible hypothesis, not a report-wide rating.

Linked recommendations. CORE-S11, APP-S6, and SUP-S1.

B. Recommendation Register

Section 1.7 requires every recommendation record to identify the findings addressed, the enforcing actor or owner, response urgency, dependencies, effort, sequence and target horizon, acceptance evidence, status, and residual-risk decision. This appendix is that register. It consolidates the forty-three recommendations issued across Sections 3–7 into a single reverse index from recommendation to finding, owner, work package, and observable acceptance evidence.

The register does not create, revise, or imply a risk rating. **Recommendation identifiers name items only.** Their numbers express no risk, urgency, effort, dependency, or delivery order, and a recommendation addressing several findings does not thereby acquire a tier. Where a *route* is shown, it is the maximum inherited response route of the findings the recommendation addresses, reproduced from Table 34 for convenience; it is a property of those findings, not of the recommendation. A recommendation whose only linked record is an unrated evidence record inherits no urgency and is marked *Evidence-conditional*.

Dependencies, delivery sequence, and target horizons are maintained in Appendix C through the work-package column; they are not duplicated here. Effort uses the definitions in Table 4 and describes work breadth, not elapsed time or risk.

Effort revised against existing work. Six recommendations — CORE-S1, CORE-S2, CORE-S4, CORE-S5, CORE-S6, and APP-S2 — describe controls that are already designed, and in several cases built and measured, in the H5Lens / hdf5-pickles project. Their effort below is the *revised* estimate from Table 10, which assumes reuse of existing identifiers, boundaries, fixtures, and measurements as data, subject to the licensing boundary in Section C.3. Each is marked (**re-scoped**). Effort is still not subtracted from risk, and a smaller estimate never shortens a response clock.

Status and residual-risk decision. At publication every recommendation in this register has status *Proposed*, implementation status *not verified*, and **no**

recorded residual-risk decision. These two fields are deliberately not tabulated per row in this revision, because populating them with a uniform value would misrepresent an unadopted plan as a tracked program. On adoption, the roadmap authority should add the per-row status vocabulary and residual-risk disposition required by Appendix C.

Table 35 – Recommendation register: findings addressed, owner, package, effort, and acceptance evidence

ID	Recommendation / owner	Findings addressed (inherited route)	Package / effort	Acceptance evidence
Core library, tools, and format — security and safety track				
CORE-S0	Separate defect closure from recurring-class closure. CORE	CORE-SEC-001, CORE-SEC-002 (Near-term; Planned)	RM-1B, RM-3C, RM-4A / Medium	Every closed defect record carries the seven artifacts in Table 11; the paired class-prevention record remains open and separately identified until the invariant framework covers sibling cases.
CORE-S1	Build an invariant registry for the file format. CORE	CORE-SEC-001, CORE-SEC-002, IND-SAF-001, IND-SAF-002 (Near-term; Planned)	RM-3A / Medium (re-scoped)	Versioned invariant files exist for every structure in the declared tranche; each invariant maps to an always-active enforcement point; single-invariant violation fixtures are rejected predictably in assertion-enabled and -DNDEBUG builds; no declared invariant relies solely on an assertion.
CORE-S2	Introduce a two-phase parser boundary. CORE	CORE-SEC-001, CORE-SEC-002 (Near-term; Planned)	RM-3B / Large (re-scoped; unchanged)	Each migrated component exposes raw-decode, then record-local, aggregate-object and reference-graph validation, then construction, with materialization separated from activation; no native object is constructed from file metadata before its validator passes; malformed and valid compatibility corpora pass with sanitizer and fuzz evidence. Closure names migrated components only, never the class.
CORE-S3	Centralize checked arithmetic and allocation. CORE	CORE-SEC-001, CORE-SEC-002, APP-SEC-004 (Near-term; Planned)	RM-2A, RM-3A / Large	Parser allocations and file-derived copies route through checked wrappers or a reviewed, recorded exemption; overflow-boundary unit tests cover each wrapper; the corresponding CORE-S12 metrics report 100% over the declared tranche denominator.
CORE-S4	Create security profiles for file opening. CORE	APP-SEC-001, APP-SEC-003, APP-SEC-004, CORE-SEC-001, CORE-SEC-002 (Immediate; Near-term)	RM-2A / Medium (re-scoped)	Profile API and defaults documented and default-state tested; hostile-file integration corpus passes; external-resource, plugin, traversal, metadata, allocation, and filter-expansion budgets are enforced and tested; a file-replacement test proves preflight and runtime enforcement cannot diverge.

Table 35 – Recommendation register, continued

ID	Recommendation / owner	Findings addressed (inherited route)	Package / effort	Acceptance evidence
CORE-S5	Fuzz by structure, not just by whole-file crashes. CORE	CORE-SEC-001, CORE-SEC-002 (Near-term; Planned)	RM-3C / Medium (re-scoped)	Each declared parser has a registered harness, and the two named gaps — OSS-Fuzz integration and per-family targets — are closed; the PR, nightly, weekly, release-candidate, and CVE-closure gates in Table 17 run at their stated cadence; coverage is reported by component, not only as total line coverage.
CORE-S6	Make malformed-input regression cheap. CORE	CORE-SEC-001, CORE-SEC-002, CORE-TOOL-001, IND-SAF-001, IND-SAF-002 (Near-term; Planned)	RM-1B, RM-2C, RM-3C / Small (re-scoped)	The test/security corpus layout exists; every closed finding has a minimized seed and expected.yml; the full corpus runs green in assertion-enabled and -DNDEBUG builds with identical controlled rejection.
CORE-S7	Add a security-review checklist that blocks risky parser changes. CORE	CORE-SEC-001, CORE-SEC-002; CORE-SEC-003 evidence (Near-term; Planned)	RM-1E, RM-3C / Medium	The checklist is recorded on every parser-adjacent PR; the automated rules fail non-conforming PRs; a new or declared file-derived validity predicate enforced only by an assertion demonstrably fails the gate.
CORE-S8	Separate compatibility from safety. CORE	CORE-SEC-002, IND-SAF-001, IND-SAF-002 (Planned)	RM-2C, RM-4A / Medium	Each case class in Table 19 has a documented per-profile disposition; positive and negative fixtures produce exactly the documented result across the supported-reader matrix.
CORE-S9	Harden tools as separate attack surfaces. CORE	CORE-TOOL-001, CORE-SEC-001, CORE-SEC-002 (Near-term; Planned)	RM-1B, RM-2A, RM-3C / Medium	Tool default-profile tests pass; rendering paths are length-bounded under adversarial fixtures; legacy converters do not build without HDF5_ENABLE_LEGACY_UNSAFE_TOOLS=ON; tool-specific sanitizer regression is green.
CORE-S10	Plugin policy and signed extensions. EXT	APP-SEC-005 (Near-term); EXT-PRIV-001 conditionally	RM-3D / Large	Plugin manifest schema and protected allowlist or keystore are deployed; tests reject unsigned, untrusted, writable-path, over-budget, and wrong-profile plugins. A valid signature is not accepted as evidence of resource safety.
CORE-S11	Coordinate supply-chain controls with parser hardening. REL	SUP-SAFE-001 (Evidence-conditional); provenance support for all records	RM-3E, RM-4A / Large	Scorecard runs weekly with alerting; CodeQL is required on C/C++ parser PRs; third-party actions are pinned; releases carry source and binary SBOMs, signatures, checksums, and verifiable SLSA provenance.
CORE-S12	Metrics that force the right behavior. CORE	CORE-SEC-001, CORE-SEC-002, CORE-TOOL-001; CORE-SEC-003 evidence (Near-term; Planned)	RM-1B, RM-3C, RM-4A / Medium	A public dashboard publishes every metric in Table 22 against a versioned tranche denominator; each target is met or carries a named, expiring exception. The dashboard is coverage evidence, not defect closure.

Table 35 – Recommendation register, continued

ID	Recommendation / owner	Findings addressed (inherited route)	Package / effort	Acceptance evidence
CORE-SAFE-001	Give HDF5 an ENOSPC story. CORE	CORE-SAFE-001 (Near-term containment; structural treatment Planned)	RM-2D, RM-3G / Medium	Fault injection returns ENOSPC at write, flush, and close for each supported VFD and buffering profile; the application receives a distinguishable error at every injected point; last durable state and recovery result are recorded; a later open never silently returns wrong data. Reported success followed by a file later found inconsistent is a failure.
CORE-SAFE-002	State, then improve, the crash-consistency guarantee. CORE	CORE-SAFE-002 (Near-term containment; structural treatment Phase 3)	RM-2D, RM-3G / Large	Published per-layout and per-driver guarantee including its limits; a fault-injection harness kills the writer across metadata update, flush, and close and records for each point whether the file opens, opens after repair, or does not, and whether data read back differs from what was acknowledged written. The last case is reported separately and treated as a defect. Results published per supported release.
Core library and ecosystem — privacy track				
CORE-PRIV-001	Privacy exposure awareness. PRIV	CORE-PRIV-001 (Near-term)	RM-1D / Small	A dated privacy review exists for the defined population, covering datasets, metadata, structure, workflow artifacts, derived products, and aggregation/inference, and naming a data owner and intended recipients for each disclosure boundary.
CORE-PRIV-001	Privacy exposure through HDF5 tools and plugins awareness. PRIV	EXT-PRIV-001 (Evidence-conditional)	RM-3D / Medium	Each in-scope tool, connector, filter, and VFD has published documentation of the outputs it generates and the controls available to suppress or bound them; the artifact inventory is reproduced by test rather than asserted.
CORE-PRIV-001	Privacy-aware system design. PRIV	CORE-PRIV-001 (Near-term); EXT-PRIV-001	RM-1D, RM-3F / Medium	Before/after inspection through both the HDF5 API and raw-byte search shows the effect of minimization; temporary artifacts are removed on a tested retention rule; generated metadata introduced by conversion is enumerated and dispositioned.
CORE-PRIV-001	Data encryption as one privacy control. PRIV	CORE-PRIV-001 conditionally (Near-term)	RM-2G / Large	Ciphertext and metadata inspection, unauthorized-open test, partial-I/O workflow test, protected key inventory, backup/restore evidence, and a successful key-loss-and-recovery exercise. Encryption is credited only for the boundaries actually tested.
Downstream applications and services				
APP-S1	Define an “untrusted HDF5” profile. APP	APP-SEC-001, APP-SEC-003, APP-SEC-004, CORE-SEC-002 (Immediate; Near-term)	RM-0A, RM-1C, RM-2A, RM-2B / Medium	The profile’s stated defaults are enforced and individually tested at each governed boundary; a deployment that cannot reach the core API demonstrates an equivalent tested wrapper rather than documentation alone.

Table 35 – Recommendation register, continued

ID	Recommendation / owner	Findings addressed (inherited route)	Package / effort	Acceptance evidence
APP-S2	Ship a preflight validator. APP, CORE	APP-SEC-001, APP-SEC-003, CORE-SEC-002, CORE-TOOL-001 (Immediate; Near-term)	RM-2A, RM-2B / Medium (re-scoped)	Validator output is machine-checkable against the profile; a file-replacement test proves the preflight result is bound to the same open handle or verified immutable identity used at load time; runtime hooks fail closed on file, policy, or target change.
APP-S3	Disable dangerous HDF5 features by default in ML model loading. APP	APP-SEC-002, APP-SEC-003, APP-SEC-004 (Near-term)	RM-1C / Medium	Negative tests prove rejection of executable reconstruction from legacy containers, external dataset references, dynamic plugins, and CWD-relative resolution; allocation budgets are enforced before materialization.
APP-S4	Make external references opt-in. APP	APP-SEC-001, APP-SEC-003 (Immediate; Near-term)	RM-0A, RM-1C, RM-2B / Medium	Negative fixtures exist for every supported external-resource form — raw storage, external links, virtual sources — including absolute paths, traversal, and CWD lookup; a sandbox-escape test passes; every resolution is logged.
APP-S5	Treat plugins as executable code. APP	APP-SEC-005 (Near-term)	RM-1C, RM-3D / Medium	Startup evidence records the effective plugin-loading state; canary plugins planted in the current directory, an environment-selected path, and a user-writable searched path do not load or execute.
APP-S6	Require downstream SBOM and version reporting. REL, APP	SUP-SAFE-001 (Evidence-conditional); census support for all records	RM-1A, RM-3E, RM-4A / Medium	Each downstream publishes the listed fields in a consumer-readable form and they are ingested by the RM-1A census. Reporting supplies census input only; it does not establish that a deployment is affected or remediated.
APP-S7	Split CVE classification into scope tags. GOV	<i>None</i> . Triage-process control supporting the RM-1A census and RM-4A reassessment; addresses no single finding (Evidence-conditional)	RM-1A, RM-4A / Small	Every HDF5-related advisory carries exactly one scope tag from the published list, and the tag is recorded in the census. A tag assigns no priority: each bounded scenario is still rated and routed under Section 1.7.
Extensions — VOL connectors, filter plugins, and virtual file drivers				
VOL-S1	Document connector privacy characteristics. EXT	EXT-PRIV-001 (Evidence-conditional)	RM-3D / Small	Each in-scope connector publishes the listed disclosure characteristics, and a reproducible data-flow test confirms the documented external endpoints, caches, and generated metadata are complete for the tested configuration.
VOL-S2	Improve connector transparency and introspection. EXT	EXT-PRIV-001 (Evidence-conditional)	RM-3D / Medium	An application can query, at runtime, the actual data origin, endpoints, and artifacts a connector will use; introspection output is tested against the observed data flow.

Table 35 – Recommendation register, continued

ID	Recommendation / owner	Findings addressed (inherited route)	Package / effort	Acceptance evidence
VOL-S3	Strengthen connector trust and deployment practices. EXT	EXT-PRIV-001 (Evidence-conditional); mechanism shared with APP-SEC-005	RM-3D / Medium	Connector search paths are protected and non-writable by unprivileged users; provenance or signature verification is enforced and its rejection path tested.
VOL-S4	Follow privacy-aware connector development guidelines. EXT, PRIV	EXT-PRIV-001 (Evidence-conditional)	RM-3F / Small	Published guidelines exist and a reference connector demonstrably satisfies them under the RM-2E data-flow test.
FILTER-S1	Strengthen filter trust and deployment practices. EXT	EXT-PRIV-001 (Evidence-conditional); mechanism shared with APP-SEC-005	RM-3D / Medium	Filter plugin paths are protected; unsigned, untrusted, and writable-path filters are rejected by test; expansion-ratio and allocation budgets are enforced.
FILTER-S2	Select and document filter configurations with privacy in mind. EXT	EXT-PRIV-001 (Evidence-conditional)	RM-3D / Small	Filter identifiers, parameters, and chunk-size characteristics that remain observable are enumerated for the tested configuration and dispositioned by the data owner.
FILTER-S3	Use encryption when filter metadata are privacy sensitive. PRIV	EXT-PRIV-001 conditionally (Evidence-conditional)	RM-3F / Medium	Ciphertext inspection confirms the identified filter metadata are not observable, and key-loss and recovery testing passes under RM-2G custody controls.
FILTER-S4	Minimize workflow artifacts. PRIV	EXT-PRIV-001 (Evidence-conditional)	RM-3F / Medium	Temporary buffers and files created by the filter pipeline are enumerated, securely handled, and removed on a tested retention rule.
FILTER-S5	Standardize privacy documentation. EXT	EXT-PRIV-001 (Evidence-conditional)	RM-3D / Small	A common documentation template is published and applied to the in-scope filter set.
VFD-S1	Document storage behavior. EXT	EXT-PRIV-001 (Evidence-conditional)	RM-3D / Small	Each in-scope VFD publishes where bytes and metadata are placed, replicated, cached, and logged; the documentation is confirmed by a reproducible storage trace.
VFD-S2	Minimize unintended persistence. PRIV	EXT-PRIV-001 (Evidence-conditional)	RM-3F / Medium	Snapshots, caches, and temporary copies are enumerated and removed on a tested retention rule; residual artifacts after normal and abnormal termination are inspected.
VFD-S3	Strengthen cloud and distributed-storage guidance. EXT	EXT-PRIV-001 (Evidence-conditional)	RM-3D / Small	Published guidance covers credentials, endpoint trust, request logging, and access-pattern observability for the supported cloud VFDs.
VFD-S4	Strengthen VFD trust and deployment practices. EXT	EXT-PRIV-001 (Evidence-conditional); mechanism shared with APP-SEC-005	RM-3D / Medium	VFD plugin paths are protected; where signed-plugin verification is used, the keystore is protected, approved artifacts are signed, and rejection of unsigned or untrusted drivers is tested. Upgrading alone is not accepted as enabling the control.
Interoperability and supply chain				

Table 35 – Recommendation register, continued

ID	Recommendation / owner	Findings addressed (inherited route)	Package / effort	Acceptance evidence
FMT-S1	Make the specification machine-readable and adjudicable. INTEROP	FMT-SAF-001 (Near-term adjudication; Phase 3 artifact)	RM-2C, RM-3I / Large	For a declared subset, the machine-readable description generates fixtures the reference library accepts and rejects as predicted, with every divergence classified as either a library defect or a description defect and no third category. Every dispute raised in the tracking period has a recorded disposition naming the authoritative artifact and the change made, with elapsed time from report to ruling published. A dispute closed by an implementation changing to match the reference library, without a ruling, is recorded <i>unresolved</i> .
IND-S1	Maintain a differential conformance and compatibility corpus. INTEROP	IND-SAF-001, IND-SAF-002 (Planned)	RM-2C, RM-4A / Large	The supported-reader matrix accepts every valid fixture and rejects every intentionally invalid fixture with a controlled, documented diagnostic; the #6409 and #6534 cases are preserved as separate regressions; the defined legacy population has a reproducible repair, regeneration, migration, or accountable compatibility disposition preserving checksums and provenance.
LONG-S1	Make extension dependencies discoverable, then removable. INTEROP, EXT	LONG-SAF-001 (Planned, recorded departure)	RM-2C, RM-3E / Medium	Every file in a defined corpus has a generated non-core dependency report, confirmed complete by opening the file with each declared dependency deliberately absent and finding no undeclared one. Archival-profile files read correctly in a reference build with <i>all</i> non-core plugins disabled. For each retained dependency, archived specification and test vectors are demonstrated sufficient for an independent party to reproduce the codec output.
SUP-S1	Maintain a producer/consumer profile compatibility and migration matrix. REL	SUP-SAFE-001 (Evidence-conditional)	RM-3E, RM-4A / Large	A finite, versioned supported-profile catalog, matrix, and fixture set run in CI; diagnostics identify the mismatched capability; every incompatible supported pair has a tested deterministic repair, rebuild, migration, controlled rejection with a supported alternative, or accountable exception.

B.1. Correspondence with SSP SIG Policy Controls

The SSP SIG maintains a catalog of HDF5-SHINES-* policy controls [48], which is a *different namespace* from the recommendation identifiers in this appendix (Section 1.8). The correspondence below is recorded so that the two

documents can be read together, and so that a reader of the OpenSSF coverage matrix [52] can see which of this report's recommendations bear on a control family that matrix scores.

The mapping is stated at family granularity, not control granularity, because the control catalog was consulted through its published references rather than enumerated in full for this revision; a control-level mapping is an adoption-time task for the GOV role. **A correspondence is not a claim that the control is satisfied.** A recommendation may bear on a control and, if implemented, still leave it unmet — and the SSP SIG, not this report, renders that judgment.

Table 36 – Recommendation families mapped to SSP SIG policy-control families

Recommendations	Control family	Note
CORE-S0, APP-S7	HDF5-SHINES-VULN-*	Vulnerability triage, classification, and disclosure process.
CORE-S1, CORE-S2	HDF5-SHINES-TM-*	Threat modeling and security architecture. The OpenSSF matrix records TM as its only COVERED entry at Level 3 security assessment, so this is the family where existing policy is strongest.
CORE-S3, CORE-S7	HDF5-SHINES-SDLC-*	Secure SDLC and code review, including review-blocking gates.
CORE-S4, CORE-S9, APP-S1-APP-S4	HDF5-SHINES-HARD-*	Secure configuration and hardening; the profile work is the largest single contribution this report makes to that family.
CORE-S5, CORE-S6, IND-S1	HDF5-SHINES-TEST-*	Security testing and fuzzing.
CORE-S8, SUP-S1, LONG-S1, FMT-S1	HDF5-SHINES-COMP-*	Compatibility and lifecycle support. LONG-S1 extends this family to a decadal archival horizon that the control catalog does not currently address.

Table 36 – Recommendation families mapped to SSP SIG policy-control families, continued

Recommendations	Control family	Note
CORE-SAFE-S1, CORE-SAFE-S2	No control family	A gap in the control catalog, not in this report. Neither project's policy controls cover write-side durability, crash consistency, or recovery tooling. The SSP SIG hazard registry rates these conditions but no HDF5-SHINES-* control governs them.
CORE-S10, APP-S5, VOL-S1-VOL-S4, FILTER-S1-FILTER-S5, VFD-S1-VFD-S4	HDF5-SHINES-PLUG-*	Plugin and extension governance. Signature and keystore aspects of CORE-S10 and FILTER-S3 also touch HDF5-SHINES-CRYPTO-*
CORE-S11, APP-S6	HDF5-SHINES-BUILD-*, HDF5-SHINES-DEP-*, HDF5-SHINES-REL-*	Secure build and provenance, dependency policy, release and change control.
CORE-S12	HDF5-SHINES-GOV-*	Governance and program metrics.
CORE-PRIV-S1- CORE-PRIV-S3	HDF5-SHINES-PRIV-*	Data handling and privacy policy.
CORE-PRIV-S4	HDF5-SHINES-CRYPTO-*	Cryptography and key management, including the key-custody and recovery requirements this report makes non-optional.

Conversely, two families in the control catalog have no recommendation in this report: HDF5-SHINES-IR-* (incident response and evidence retention) and the CI/CD least-privilege and input-sanitization controls that the OpenSSF matrix flags as GAP. This report's RM-0A touches evidence preservation in passing but develops no incident-response recommendation, and build-pipeline security is outside its stated scope.

B.2. Recommendations Not Tied to a Formal Finding

Three recommendation groups deliberately address no rated finding, and this is recorded rather than hidden:

APP-S7 is a triage-process control. It improves the applicability signal that the RM-1A census and RM-4A reassessment consume, but it treats no adverse-event scenario and therefore inherits no urgency.

The extension track (VOL-S1-VOL-S4, FILTER-S1-FILTER-S5, VFD-S1-VFD-S4) and CORE-PRIV-S2 address the unrated evidence record EXT-PRIV-001. They are proactive assurance proposals. Their existence does not convert that observation into a rated finding, and their delivery does not by itself establish residual-risk reduction.

CORE-S11, APP-S6, and SUP-S1 address the unrated evidence record SUP-SAFE-001. They also supply provenance inputs used across the register, but provenance is not treatment.

In an earlier revision one finding had no recommendation at all: CORE-SAFE-001 was recorded as a treatment gap. That gap is closed — CORE-SAFE-S1 now addresses it, drawing on treatment options that had already been proposed publicly by maintainers but had not been carried into this report. **Every registered finding now has at least one linked recommendation.**

C. Phased Mitigation Roadmap

This appendix converts the response routing in Table 34 into a proposed delivery plan. It is an assessment recommendation, not evidence that an organization has accepted the dates, assigned the work, or implemented the controls. The response clocks in Phases 0–2 begin when an affected deployment or equivalent exposure is identified, as specified in Table 34; the scope census in RM–1A, assert-site catalog in RM–1E, and the Phase 3 program horizon begin when the roadmap is adopted. The assessment sponsor should record a dated decision to adopt, modify, or reject the proposal; adoption must set due dates for both inventories so a response clock cannot remain latent through unbounded non-discovery. A pre-adoption exposure still enters the response route and is not held for the program decision.

The roadmap does not revise inherent risk. Phase, dependency, owner, effort, and implementation order are treatment-planning fields under Table 4; none is added to or subtracted from a finding’s impact, exposure, tier, or confidence. A dependency that prevents a final control from shipping does not extend a response horizon: the accountable owner must maintain an interim containment, record the variance, and obtain an explicit residual-risk decision.

At publication, every work package below has planning status *Proposed* and implementation status *not verified*. The role labels are proposed accountability boundaries, not assertions about present HDF Group or downstream assignments. No residual-risk acceptance is recorded by this roadmap. Within ten business days of adoption, the roadmap authority should replace each role label with a named accountable person, issue or program record, dated target or release, and implementation status. For a triggered Phase 0 record, the APP owner and GOV risk authority are assigned at T0 and containment begins immediately; the ten-day allowance applies only to untriggered program packages.

C.1. Accountability Roles

Table 37 – Proposed roadmap accountability roles

Code	Role	Accountability boundary
GOV	Security and roadmap authority	Owns scope, named assignments, exceptions, phase-gate decisions, and formal residual-risk acceptance or escalation.
CORE	Core library and tools engineering	Owns parser, arithmetic, resource-policy, utility, regression, fuzzing, and core API changes.
APP	Downstream application and service owner	Owns model or dataset loading policy, deployment containment, supported-version rollout, sandboxing, and application acceptance tests.
PRIV	Privacy and data-governance owner	Owns data classification, metadata minimization, disclosure boundaries, retention, encryption requirements, and privacy acceptance evidence.
INTEROP	Format and interoperability owner	Coordinates the reference library, specification, independent implementations, compatibility fixtures, and repair or migration policy.
EXT	Extension and plugin ecosystem owner	Owns plugin, VOL, VFD, and filter policy, manifests, trust controls, introspection, and extension documentation.
REL	Release and supply-chain owner	Owns build provenance, dependency policy, SBOMs, release gates, and downstream-consumable configuration reporting.
ASSURE	Independent verifier	Does not implement the control being reviewed; reproduces the acceptance tests, checks the evidence boundary, and signs the verification result used for closure and residual-risk reassessment.

One role is accountable for each work package even when several roles perform the work. The named accountable person may delegate tasks but remains responsible for the evidence package and readiness attestation; only GOV approves a program gate, exception, or residual-risk disposition. A package that spans independently governed projects or deployments must be instantiated once per accountable owner and scope; the generic APP package, for example, does not make one downstream responsible for another. Every instance also names its responsible implementer, an ASSURE verifier independent of that implementer, and a separate release or deployment approver that authorizes rollout after verification. For a

High or Critical scope, the verifier may not also be the package owner, rollout approver, or risk acceptor.

C.2. Phase Gates and Clocks

Phases are delivery waves, not severity levels. Work may proceed in parallel, and a later-phase structural control may begin before an earlier phase closes. The earlier response clock still controls. Phase 2 also contains evidence work for unrated records; its placement is an assurance target and does not assign those records a response urgency.

Table 38 – Roadmap phases and exit gates

Phase	Clock	Required outcome	Exit gate
0—Contain	Before the next untrusted job and no later than 24 hours after equivalent exposure is identified	Remove the demonstrated external-resource path from attacker reach while preserving evidence and rotating exposed secrets where indicated.	No confirmed Critical-equivalent path remains in service without tested containment; an accountable owner, affected scope, and decision record exist.
1—Verify and remediate known scope	Initial deployment census and assert-site catalog complete within 30 days of adoption; Near-term treatment completes within 30 days of identifying an affected deployment	Establish the current population, apply supported fixes or tested compensating controls for known High and upper-bound-High scenarios, and triage release-disabled file-derived checks.	Every applicable Phase 1 finding has version and exposure evidence, a negative test, an implementation record, and a current residual-risk disposition; the assert-site perimeter is cataloged and each confirmed adverse path is routed without waiting for completion of structural migration.

Table 38 – Roadmap phases and exit gates, continued

Phase	Clock	Required outcome	Exit gate
2—Establish release safeguards	For Planned findings, within 90 days of identification or the next supported release or migration window, whichever is sooner; evidence actions have an administrative assurance target of 90 days after adoption	Deliver strict-processing guardrails, interoperability coverage, and bounded evidence packages for the unrated observations.	The safeguards pass integration tests; compatibility cases have deterministic expected results; evidence gaps are either promoted to findings and routed or remain explicitly bounded and unrated.
3—Eliminate recurring classes	Target 3–12 months after roadmap adoption, through staged supported releases	Replace local repairs with invariant-driven parsing, structural validation, fuzz and review gates, and governed extension loading.	Each declared component or release tranche satisfies its acceptance evidence; unfinished components remain visible with containment and dated follow-on work, rather than being treated as class closure.
4—Sustain and reassess	Every pull request, nightly run, release candidate, supported release, and at least annual program review, as applicable	Keep controls effective, detect regression and ecosystem mismatch, and make residual-risk decisions from verified evidence.	Required gates run at their defined cadence; failures block or create a named, expiring exception; the findings and roadmap registers are updated.

When a package covers findings with different response routes, each finding retains its own maximum horizon. Co-delivering a Planned item during Phase 1 is an implementation convenience, not an increase in its risk or urgency; a Planned item in the same package cannot be used to defer a Near-term item to the later clock.

Each finding-and-scope instance therefore has an SLA child record separate from the work package's final-control or structural due date. The child records the finding ID, defined census boundary and completeness attestation, deployment or artifact scope, T0, urgency and maximum horizon, required interim action, due

date, performed date, status, and evidence. A 24-hour containment can close its SLA child without falsely marking a shared architectural package complete; a large shared package cannot hide an overdue child behind its later program date.

C.3. Licensing Constraint on Reuse of Existing Work

Several packages below — RM-1E, RM-2A, RM-2C, RM-3A, RM-3B, and RM-3C — can draw on work that already exists in the H5Lens / hdf5-pickles project, which supplies a bounded decoder, a machine-readable invariant registry, an untrusted-input preflight oracle, a malformed-input corpus, and differential measurements against a selected libhdf5 build [30, 39, 38]. Section 3.5.2 describes what exists and how each recommendation is re-scoped as a result.

That reuse has a hard boundary that must be planned for rather than discovered during implementation. **H5Lens is distributed under the GNU General Public License, version 3 or later; the HDF5 library is distributed under a permissive BSD-style license.** Source cannot move from the former into the latter without changing HDF5's licensing terms, which is a governance decision far outside this report's scope and is not proposed here.

The reuse that *is* available is the data and the evidence, and the producing project has been designed for exactly that separation: invariant identifiers, structural boundaries, finding codes, expected outcomes, fixture specimens, and measurement results are data, and a differently-licensed native implementation may consume them without importing implementation source. Accordingly:

Consume as data, re-implement independently.

RM-3A may adopt the existing invariant identifiers, record-family decomposition, and structural boundaries as its registry vocabulary, and must write its own enforcement code in the library. The registry is the specification; it is not a source of patches.

Consume as an oracle, not as a component.

RM-2A and RM-3C may run the existing preflight oracle as an external differential reference in CI — comparing what libhdf5 does against what

the oracle decides — without linking or vendoring it. Running a GPL tool alongside a BSD library in a test harness does not affect the library’s terms; embedding it would.

Fixtures and expectations are freely usable evidence, subject to their own provenance.

RM-1B, RM-2C, and RM-3C may adopt malformed specimens and expected-outcome records as corpus inputs. Each specimen still needs its own provenance check: some are generator-produced and reproducible, and others are supplied corpus finds whose redistribution terms must be confirmed before they enter a public test suite.

Case records are triage input, not findings.

RM-1E may take the existing case records as its starting inventory. They are third-party analyses; each still requires this programme’s own reachability, version-boundary, and consequence determination before it is promoted and routed.

The accountable GOV role should record a dated decision on this boundary at adoption, because it changes effort estimates in both directions: it removes design and enumeration work from RM-3A, and it adds independent re-implementation work that a naive “reuse the existing code” plan would have assumed away. Any proposal to relicense, dual-license, or upstream any part of the existing work is a governance action requiring its own decision record and is not assumed by this roadmap.

C.4. Work-Package Register

Dependencies below are concrete enabling relationships, not reasons to defer containment. *None* means no prerequisite work package is known; *external* means that a supported vendor release, deployment sponsor, or test fixture must be available. Effort uses the definitions in Table 4 and describes work breadth, not risk or elapsed time.

Table 39 – Proposed mitigation work packages

ID	Phase / owner	Records	Scope and recommendation coverage	Dependencies	Effort
RM-0A	0 / APP	APP-SEC-001; equivalent exposure under APP-SEC-003	Disable or sandbox external raw storage, external links, and virtual sources in untrusted processing; remove worker access to unnecessary local files and credentials; preserve logs and rotate secrets where exposure is indicated. Provides application containment consistent with APP-S1 and APP-S4 while compensating for the absent or unverified CORE-S4 API; it does not implement that core recommendation.	None; where a library control is unavailable, isolate or stop the affected job.	Medium
RM-1A	1 / GOV	All formal findings; scoping support for evidence records	Create a deployment, supported-version, feature, provenance, and authority census; assign the applicable HDF5/downstream scope tag; identify applicability, a named owner, production build mode and assertion state, and current control evidence for each record. Supports APP-S6, APP-S7, and CORE-S11.	None; affected projects and deployment owners must supply evidence.	Medium
RM-1B	1 / CORE	CORE-SEC-001, CORE-SEC-002, CORE-TOOL-001; confirmed cases promoted from CORE-SEC-003	Move affected core and tool populations to a supported fixed version or isolate them; replace any confirmed sole assertion guard on an adverse path with an always-active check; add minimized negative tests in assertion-enabled and -DNDEBUG builds and tool-specific sanitizer regression. Covers the near-term portions of CORE-S0, CORE-S6, CORE-S9, and CORE-S12.	RM-1A; RM-1E supplies promoted cases, but catalog completion is not a prerequisite to an urgent local repair; external supported fix or a documented containment.	Medium
RM-1C	1 / APP	APP-SEC-001-APP-SEC-005	Patch affected model and dataset loaders or deploy a tested local compensating control; verify safe deserialization, external-reference rejection, allocation budgets, and plugin-path ownership. Covers APP-S3-APP-S5 and the deployable portions of APP-S1; retains RM-0A until final controls pass.	RM-0A when triggered; RM-1A; an external supported application release or tested local control. Release unavailability does not extend the response clock.	Medium
RM-1D	1 / PRIV	CORE-PRIV-001	Identify sensitive plaintext metadata and sharing boundaries; minimize or tokenize metadata, restrict recipients, and determine whether those controls meet the disclosure policy. Covers CORE-PRIV-S1 and CORE-PRIV-S3.	RM-1A; a data owner must classify the affected population and intended recipients.	Medium

Table 39 – Proposed mitigation work packages, continued

ID	Phase / owner	Records	Scope and recommendation coverage	Dependencies	Effort
RM-1E	1 / CORE	CORE-SEC-003	Within 30 days of roadmap adoption, catalog file-byte-dependent assertions in supported deserializer source lines, beginning with the seven component families in issue #87. For each site, record whether an always-active check dominates it, release reachability, first post-assert sink, paired debug and -DNDEBUG fixture behavior, affected versions, and consequence. Replace a confirmed sole guard or create and route a formal finding and SLA child without waiting for the catalog or Phase 3 migration to finish. Seeds CORE-S1, CORE-S7, and CORE-S12.	None; supported-branch owners and reproducible release configurations must bound the catalog.	Medium
RM-2A	2 / CORE	CORE-SEC-001, CORE-SEC-002, CORE-TOOL-001, enabling APP-SEC-001, APP-SEC-003, APP-SEC-004	Ship an enforceable untrusted-input profile and preflight interface with external-resource, plugin, traversal, metadata, allocation, and expansion budgets; bind any preflight decision to the same file identity and runtime open so validation cannot be bypassed by replacement or policy drift; harden inspection tools. Covers CORE-S3, CORE-S4, CORE-S9, and the core portions of APP-S1 and APP-S2.	RM-1A; fixtures from RM-1B are a coordination input, not a hard prerequisite, and downstream work need not wait for the final core API.	Large
RM-2B	2 / APP	APP-SEC-001, APP-SEC-003, APP-SEC-004	Integrate strict runtime denial and enforcement-coupled preflight into each downstream boundary; impose overflow-safe per-object and aggregate allocation budgets before materialization. Covers the downstream portions of APP-S1-APP-S4. Application allocation controls are independently required and neither wait for nor follow automatically from core checked arithmetic.	RM-1A, RM-1C; RM-2A is a coordination dependency, and an interim application policy wrapper is acceptable.	Medium

Table 39 – Proposed mitigation work packages, continued

ID	Phase / owner	Records	Scope and recommendation coverage	Dependencies	Effort
RM-2C	2 / INTEROP	compatibility policy for CORE-SEC-002; IND-SAF-001, IND-SAF-002; Near-term adjudication for FMT-SAF-001	Maintain positive, intentionally invalid, and <i>contested-reading</i> fixtures across supported readers and writers; inventory affected legacy files and provide deterministic repair, migration, or documented compatibility handling. Separately and on the Near-term clock, rule on each identified specification-versus-library dispute, record the ruling, and change the losing artifact — this is a bounded action and is not contingent on RM-3I. Covers CORE-S6, CORE-S8, and IND-S1; per-case normative decisions feed, but do not duplicate, the CORE-S1 registry in RM-3A.	RM-1A; external writer, reader, and production-derived fixtures.	Large
RM-2D	2 / CORE	CORE-SAFE-001, CORE-SAFE-002	Write-path fault injection (widened). One harness, two triggers: storage exhaustion <i>and</i> abrupt process or host termination, injected across bounded write, flush, and close sequences, HDF5 versions, and VFD/filesystem/buffering profiles. For each injection point record the error surfaced to the caller, the last durable state, whether a later open succeeds, succeeds after repair, or fails, whether data read back differs from what was acknowledged written, repairability, and data uniqueness. Also covers the HAZ-006, HAZ-012, and HAZ-015 durability cluster, which shares this apparatus. Supports CORE-SAFE-S1 and CORE-SAFE-S2. Unlike the other Phase 2 studies this is <i>not</i> evidence acquisition for an unrated record: both findings are rated, and this package produces their acceptance evidence.	Storage and process fault injection, representative workloads, and archival/data-owner input.	Large
RM-2E	2 / PRIV	EXT-PRIV-001	Select a representative extension deployment and trace classified data, metadata, credentials, caches, logs, network flows, artifacts, recipients, and retention to a bounded consequence or negative result. This is evidence acquisition, not assigned urgency.	Deployment sponsor, EXT technical support, data classification, and observable test environment.	Medium
RM-2F	2 / REL	SUP-SAFE-001	Reproduce a concrete producer/consumer mismatch with exact build and runtime profiles, affected artifacts, failure mode, recoverability, population, and recurrence evidence. This is evidence acquisition, not assigned urgency.	INTEROP and downstream owners; representative builds and fixtures.	Medium

Table 39 – Proposed mitigation work packages, continued

ID	Phase / owner	Records	Scope and recommendation coverage	Dependencies	Effort
RM-2G	2 / PRIV	CORE-PRIV-001 conditionally	Where RM-1D cannot meet policy through minimization and recipient controls, deploy whole-file or equivalent all-metadata protection and test its workflow and recovery behavior. Until verification, avoid the affected sharing boundary or maintain tested access restriction within the Near-term clock. Covers CORE-PRIV-S4.	RM-1D; supported encryption design, named key custodian, protected key lifecycle, backup, and tested recovery procedure.	Large
RM-3A	3 / CORE	CORE-SEC-001, CORE-SEC-002, APP-SEC-004, IND-SAF-001, IND-SAF-002; CORE-SEC-003 evidence	Create the versioned machine-readable invariant registry and mandatory checked arithmetic, bounds, allocation, and traversal primitives; import the RM-1E catalog and map every declared invariant to an always-active enforcement point. Covers CORE-S1 and CORE-S3.	RM-1B; RM-1E and RM-2C are coordination inputs for their applicable tranches, and work can begin before either catalog is complete.	Large
RM-3B	3 / CORE	CORE-SEC-001, CORE-SEC-002	Migrate prioritized structures from direct object construction to bounded raw decode, semantic validation, and construction. Start the assert-masking perimeter with release-reachable sole guards ordered by post-assert consequence, not by raw candidate count. Covers CORE-S2.	RM-3A; migrate in independently releasable component tranches.	Large
RM-3C	3 / CORE	CORE-SEC-001, CORE-SEC-002, CORE-TOOL-001	Make class closure auditable through structure-aware fuzzing, permanent malformed-input regression, parser-review gates, tool coverage, and public metrics. Run the corpus in assertion-enabled and -DNDEBUG builds and block any file-derived invariant enforced solely by an assertion. Covers CORE-S0, CORE-S5-CORE-S7, CORE-S9, and CORE-S12.	RM-3A for invariant identifiers; harness and corpus work may begin in Phase 1.	Large
RM-3D	3 / EXT	APP-SEC-005; EXT-PRIV-001 conditionally	Establish explicit plugin policy, allowlisted paths and identities, manifests, resource limits, and tested signature enforcement; add extension privacy documentation and introspection where the bounded evidence supports it. Covers CORE-S10, APP-S5, CORE-PRIV-S2, VOL-S1-VOL-S3, FILTER-S1, FILTER-S2, FILTER-S5, VFD-S1, VFD-S3, and VFD-S4.	RM-2A for profile policy; privacy-specific prioritization uses RM-2E.	Large

Table 39 – Proposed mitigation work packages, continued

ID	Phase / owner	Records	Scope and recommendation coverage	Dependencies	Effort
RM-3E	3 / REL	SUP-SAFE-001; all records supported	Publish source and binary SBOMs, pinned dependency and action policy, signed artifacts, verifiable provenance, and runtime/build feature reporting; maintain a producer/consumer profile matrix with deterministic diagnostics and repair, rebuild, or migration paths for supported mismatches. Covers CORE-S11, APP-S6, and SUP-S1.	RM-1A; mismatch-specific treatment uses evidence from RM-2F.	Large
RM-3F	3 / PRIV	EXT-PRIV-001 conditionally	Implement extension privacy-lifecycle controls supported by RM-2E: metadata and cache minimization, privacy-aware connector development, whole-file protection where filter metadata are sensitive, secure temporary-artifact handling, and retention/deletion controls. Covers CORE-PRIV-S3, VOL-S4, FILTER-S3, FILTER-S4, and VFD-S2.	RM-1D, RM-2E; encryption work also requires the key-management and recovery controls in RM-2G.	Large
RM-3G	3 / CORE	CORE-SAFE-001, CORE-SAFE-002	Deliver the durability controls: an ENOSPC notification path at the VFD boundary with caller-selected response, a durable suspect-file state, and optional capacity reservation and pre-flush checks; publish the crash-consistency guarantee per layout and driver; then implement the selected crash-proofing mechanism. Covers CORE-SAFE-S1 and CORE-SAFE-S2. Sequence the small, independently useful items first — callback and pre-flush check — ahead of the format-visible and architectural ones.	RM-2D supplies the harness and the baseline measurements; a pass-through VFD wrapper is an acceptable proving ground before any public API commitment.	Large
RM-3H	3 / INTEROP	LONG-SAF-001	Provide per-file non-core dependency reporting; define and document an archival profile that removes removable dependencies; archive codec specifications and test vectors for those retained; require deposit records to state whether an artifact is self-contained or a view onto an external system. Covers LONG-S1.	RM-2C for the fixture and reader-matrix machinery; RM-3E for provenance reporting; an archive or data-repository partner to define the corpus.	Medium

Table 39 – Proposed mitigation work packages, continued

ID	Phase / owner	Records	Scope and recommendation coverage	Dependencies	Effort
RM-3I	3 / INTEROP	FMT-SAF-001	Maintain a declared subset of the file format as a machine-readable normative description from which validators and fixtures are generated, and adopt a standing adjudication procedure for specification-versus-implementation disputes covering ruler, timeline, dispositions, and where rulings are recorded. Covers FMT-S1. Declaring the description normative is a governance act, not a technical one, and requires an explicit GOV decision — otherwise it becomes a fourth artifact to disagree with.	RM-2C for fixtures and adjudicated cases; existing machine-readable format work is consumable as data only, under Section C.3.	Large
RM-4A	4 / GOV	All records	Operate the release gates and metrics, re-run applicability and residual-risk reviews, maintain the differential corpus, and close or re-plan exceptions. Sustains CORE-S0, CORE-S6-CORE-S8, CORE-S12, CORE-S11, IND-S1, APP-S6, APP-S7, and SUP-S1.	Begins with Phase 1 evidence and absorbs each accepted package; it does not wait for all Phase 3 work to finish.	Medium (recurring)

C.5. Minimum Viable Subset Under Constrained Capacity

The register above is scoped to risk, not to budget. It proposes nine *Large* packages, several of which are multi-release architectural programs. A plan that cannot be started is not a control, so this subsection identifies what to fund first if the full program cannot be resourced.

What this subsection is and is not. This is a funding and sequencing aid. It is **not** a risk rank, and it does not revise any inherent rating, response route, or maximum horizon. Section 1.7 forbids effort from lowering urgency, and nothing here does so: Tier A below is precisely the work whose omission would breach a response clock, and it is therefore not discretionary at any funding level. Tiers B and C order the *prevention* program, which has program commitments rather than finding-response clocks. Deferring a Tier C item is permissible only with the recorded, expiring exception required by Section C.8, and each deferral carries the consequence stated in Table 40.

Tier A — not discretionary. RM-0A (when triggered), RM-1A, RM-1B, RM-1C, RM-1D, and RM-1E. Every one is *Medium* effort. Together they are the entire Phase 0-1 response obligation: contain a confirmed Critical-equivalent path, establish the population, treat the known affected scope, and bound the assert-masking perimeter. RM-1A is first among equals — without a census, no other package can demonstrate that it treated anything, and the version-and-path gap recorded in Appendix D cannot be closed.

Tier B — highest prevention coverage per unit of effort. Five items return the most durable control per engineer-week, and each is independently useful if the rest of Phase 3 never ships. This ordering reflects the effort revised in Table 10: because the invariant registry, the profile semantics, the corpus, and the fuzz-harness design already exist and can be consumed as data, four of the five are now Small or Medium rather than Large. CORE-S1 in particular moves up — adopting an existing invariant vocabulary and mapping it to enforcement points is a materially smaller job than designing the vocabulary from nothing, and it is the prerequisite the rest of the registry-driven work builds on.

Tier C — defer explicitly, with the consequence recorded. The remaining Large programs. Deferring them is a legitimate capacity decision; representing them as unnecessary is not.

Table 40 – Capacity-constrained ordering of the prevention program

Item	Why it is high leverage	Consequence if deferred
Tier B — fund these five before any other Phase 2-3 work		
CORE-S6 via RM-1B, RM-3C (Small, re-scoped)	The malformed-input corpus is the cheapest durable control in the program and a prerequisite for verifying every other one. It also carries the debug/-DNDEBUG parity harness that CORE-SEC-003 needs, so it is already required by Tier A.	Every subsequent fix becomes unverifiable and every closed defect can silently regress. Nothing else in Phase 3 can produce acceptance evidence.

Table 40 – Capacity-constrained ordering of the prevention program, continued

Item	Why it is high leverage	Consequence if deferred
CORE-S7 via RM-1E, RM-3C (Medium)	The review checklist and its automated rules are configuration, not architecture. They stop reintroduction of the recurring class at the point of change, including the assert-only-guard pattern, at a fraction of the cost of migrating existing code.	The class keeps being reintroduced at the same rate it is repaired, and the invariant work in RM-3A is continually eroded by new code.
CORE-S3 via RM-2A, RM-3A (Large; tranche it)	Checked arithmetic and allocation address the step at which a validation failure becomes memory corruption. It is the highest defect-class coverage per line changed, and it can be delivered structure-by-structure rather than as a single migration.	The dominant root-cause category in Table 7 remains untreated even where decode-side checks are added.
CORE-S4 via RM-2A (Medium, re-scoped)	Security profiles are the only item that removes work from every downstream simultaneously, and they are the control the ecosystem has explicitly asked for. Shipping the profile and the external-resource policy first, with resource budgets following, delivers most of the value early.	Each downstream continues to reimplement partial, divergent policy; APP-S1 and APP-S2 stay unimplementable as shared controls; RM-0A containment remains the only available answer.
CORE-S1 via RM-3A (Medium, re-scoped)	An invariant vocabulary, record-family decomposition, and per-family coverage map already exist and are consumable as data. Adopting them converts the registry from a design problem into an enforcement-mapping problem, and every other registry-driven control — review gates, fuzz targeting, metrics denominators — keys off the identifiers it establishes.	CORE-S7 and CORE-S12 lose their denominator: there is nothing to check a change against and nothing to measure coverage over. Deferring this while funding the others makes the others unmeasurable.
Tier C — defer with a recorded, expiring exception		

Table 40 – Capacity-constrained ordering of the prevention program, continued

Item	Why it is high leverage	Consequence if deferred
CORE-S2 (RM-3B)	The parser migration is the actual class-elimination control, and it is the one item the existing design work does <i>not</i> make cheaper: a design reduces uncertainty, not migration cost. With CORE-S1 funded in Tier B, the registry can still be seeded opportunistically from RM-1E and RM-1B findings while migration waits.	Class elimination cannot be claimed. Local repairs remain local; the CORE-S0 class-prevention records stay open indefinitely and must be reported as such.
CORE-S5 (RM-3C, Medium, re-scoped)	Structure-aware fuzzing finds families rather than instances, and a harness design with per-family recipes already exists. The two named gaps are OSS-Fuzz integration and per-family targets; close those rather than rebuilding the portfolio. Borderline Tier B if capacity allows.	New variants of known families continue to arrive as individually reported defects.
CORE-S10, CORE-S11 (RM-3D, RM-3E)	Plugin signing and full provenance are valuable but have low marginal effect while CORE-S4 plugin policy is absent. Hash allowlists and protected paths are the interim control.	APP-SEC-005-class exposure is treated per deployment rather than structurally; release provenance stays unverifiable.
CORE-PRIV-S2 (RM-2G)	Conditional by construction: reached only where RM-1D minimization and recipient controls cannot meet policy. Requires key custody and tested recovery before it may be credited.	Affected sharing boundaries are avoided or access-restricted instead, within the CORE-PRIV-001 Near-term clock. This is an acceptable interim posture.
IND-S1, SUP-S1 (RM-2C, RM-3E)	Both are Large and both are genuinely needed, but their findings are Moderate and their populations are bounded and known. Preserve the #6409 and #6534 fixtures now; build the full matrix later.	The two Planned interoperability findings still require their bounded treatment on the 90-day clock; only the general corpus and profile matrix are deferred.

Reading the subset honestly. Funding Tier A and Tier B is a coherent program: it treats every known affected population, bounds the assert perimeter, and installs the three controls that make future work verifiable and the one control that helps the whole downstream ecosystem. It does **not** eliminate the recurring class, and a report of progress should not say that it does. Under CORE-S0, the class-prevention record remains open, and under CORE-S12 the tranche denominator must show the untreated remainder rather than reporting only on the delivered subset.

C.6. Acceptance and Closure Evidence

A work package is complete only when its artifacts are linked from the implementation record and reproduced or otherwise independently verified by the assigned ASSURE role before acceptance by the accountable role. Shipping code or publishing guidance alone does not establish control coverage or reduce residual risk.

Table 41 – Required work-package acceptance evidence

Package	Required observable evidence	Closure decision
RM-0A	Configuration or code diff; deployment proof; negative fixtures for every supported external-resource form; sandbox escape test; credential and secret-rotation record where indicated.	APP confirms containment; GOV records whether incident handling remains open.
RM-1A	Machine-readable inventory recording version, linkage, enabled features, production build mode, assertion state, scope tag, trust boundary, authority, exposure, owner, census boundary and completeness attestation, and evidence date for each applicable scope.	GOV accepts applicability and creates dated treatment or non-applicability records.
RM-1B	Supported-version matrix; patch or isolation record; minimized fixtures; assertion-enabled and -DNDEBUG ASan/UBSan/LSan and controlled-error results for affected core and tool paths.	CORE accepts technical evidence; GOV records current residual risk.

Table 41 – Required work-package acceptance evidence, continued

Package	Required observable evidence	Closure decision
RM-1C	Application version and rollout evidence; tests proving safe-mode code rejection, external-path rejection, allocation-budget enforcement, and plugin-path ownership; a canary plugin placed in the current directory, an environment-selected path, and a user-writable searched path must not load or execute, and startup evidence must record the effective plugin-loading state.	APP accepts each deployed boundary; GOV records current residual risk.
RM-1D	Metadata/data-flow inventory, classification and recipient matrix, and before/after inspection through both HDF5 APIs and raw-byte search showing the effect of minimization and recipient controls.	PRIV signs the disclosure-boundary decision and whether RM-2G is required; GOV records current residual risk.
RM-1E	Versioned source-site inventory recording branch, function, predicate, file-byte provenance, dominating always-active guard, first post-assert sink, release reachability, fixture, assertion-enabled and -DNDEBUG result, supported-version scope, consequence, and disposition; paired negative tests for every confirmed site; promoted finding and SLA child records where an adverse path is supported.	CORE accepts the bounded catalog and local repairs; GOV confirms that supported scenarios are rated and routed and that inconclusive sites remain open.
RM-2A	API and policy documentation; default-state tests; hostile-file integration corpus; allocation, traversal, metadata, filter-expansion, external-resource, and plugin tests; a file-replacement test proving preflight and runtime enforcement cannot diverge; tool-default tests.	CORE accepts the core control; GOV records the scope for which it may be credited.
RM-2B	Per-downstream policy and integration tests; before-allocation budget tests for extreme, overflow-boundary, cumulative, and valid boundary cases; external-resource rejection; file-replacement test; deployment evidence.	APP accepts each independently governed integration; GOV records credited coverage and current residual risk.
RM-2C	Versioned positive and negative fixture corpus with writer, reader, format-bound, and expected-result metadata; supported-reader matrix; exact release/backport verification; acceptance of the sufficient nonminimal fixture and predictable rejection of an under-sized comparator; corrected writer round-trip; complete in-scope legacy enumeration; and repaired-file pre/post checksums, provenance, and supported-reader acceptance.	INTEROP accepts conformance results and documents unresolved compatibility exceptions.

Table 41 – Required work-package acceptance evidence, continued

Package	Required observable evidence	Closure decision
RM-2D	Fault-injection matrix and logs covering <i>both</i> the storage-exhaustion and abrupt-termination triggers, recording API sequence, version, VFD/filesystem/buffering profile, error surfaced, durable file state, reopen and repair result, any divergence between acknowledged and read-back data, data uniqueness, and affected workload boundary.	CORE and the data owner submit the result to GOV, which records current residual risk for CORE-SAFE-001 and CORE-SAFE-002; an inconclusive study remains open and is re-planned.
RM-3G	Fault-injection results showing a distinguishable ENOSPC-class error at every injected point for every supported VFD and buffering profile; suspect-state set and cleared under test; published per-layout crash-consistency guarantee; and a matrix of writer-kill points recording open/repair/fail outcomes and any divergence between acknowledged and read-back data.	CORE accepts the technical evidence; GOV records current residual risk for CORE-SAFE-001 and CORE-SAFE-002. A run in which the library reported success and the file is later found inconsistent blocks acceptance.
RM-3I	Machine-readable description of the declared subset; generated valid and invalid fixtures with reference-library results; a divergence log in which every entry is classified as a library defect or a description defect; the adopted adjudication procedure; and a dispute ledger recording disposition, authoritative artifact, change made, and elapsed time to ruling.	INTEROP accepts the technical evidence; GOV decides whether the description is declared normative and records that decision. An unruled dispute is reported open, never closed by concession.
RM-3H	Generated dependency report per file in the defined corpus, confirmed complete by opening with each declared dependency absent; archival-profile files reading correctly in a reference build with all non-core plugins disabled; and at least one demonstrated independent reproduction of a retained codec from its archived specification and test vectors.	INTEROP accepts; GOV records residual risk for LONG-SAF-001.

Table 41 – Required work-package acceptance evidence, continued

Package	Required observable evidence	Closure decision
RM-2E	Reproducible extension data-flow test, classified population, recipients, storage and network endpoints, cache/log/artifact retention, control evidence, and bounded consequence or negative result.	PRIV and EXT submit the result to GOV; a negative disposition requires a documented method, tested scope and population, and retained results, while an inconclusive study remains open.
RM-2F	Exact producer and consumer build/runtime profiles, reproducible fixture and diagnostic, affected artifact inventory, recoverability, population, and recurrence results.	REL and INTEROP submit the result to GOV; a negative disposition requires a documented method and tested scope, while an inconclusive study remains open.
RM-2G	Ciphertext and metadata inspection, unauthorized-open test, partial-I/O workflow test, protected key inventory, backup/restore evidence, and successful loss-and-recovery exercise.	PRIV and the key custodian accept confidentiality and recoverability; GOV records credited scope and current residual risk.
RM-3A	Versioned invariant files; code-to-invariant mapping; checked primitive APIs; migration inventory; tests showing the prioritized structures reject single-invariant violations predictably in assertion-enabled and -DNDEBUG builds; proof that no declared invariant relies solely on an assertion.	CORE accepts each declared tranche; missing structures or enforcement mappings remain open.
RM-3B	Per-component raw-decode/validate/construct boundary; no native object before validation; malformed and valid compatibility corpus results; assertion-enabled and -DNDEBUG sanitizer and fuzz evidence.	CORE closes only named migrated components, not the parser class globally.
RM-3C	PR, nightly, weekly, release-candidate, and CVE-closure jobs; retained minimized seeds; coverage by structure; review-checklist records; metric dashboard; assertion-enabled/-DNDEBUG parity; and proof that a new or declared file-derived invariant enforced only by an assertion fails the release gate.	CORE and GOV approve the release gate; every bypass is named and expiring.

Table 41 – Required work-package acceptance evidence, continued

Package	Required observable evidence	Closure decision
RM- 3D	Plugin manifest and policy; protected allowlist or keystore; tests rejecting unsigned, untrusted, writable-path, over-budget, and wrong-profile plugins; extension data-flow documentation and introspection tests.	EXT accepts technical coverage; PRIV accepts privacy claims; GOV records residual risk for affected findings only.
RM- 3E	Release SBOMs, checksums, signatures, provenance verification, pinned automation, dependency scans, a consumer-visible build/runtime feature report, and a versioned producer/consumer profile matrix with expected outcomes and a finite supported-profile catalog, fixture set, and retained CI execution results. Every incompatible supported pair has a tested deterministic repair, rebuild, migration, controlled rejection with a supported alternative, or accountable exception.	REL accepts the release evidence; mismatches become dated defects or scoped findings.
RM- 3F	Tests and artifacts showing minimized connector metadata and caches, secure temporary-buffer/file handling, enforced retention and deletion, and, where required, ciphertext inspection plus key-loss/recovery testing for filter metadata.	PRIV accepts the bounded lifecycle claims after ASSURE verification; unsupported claims remain open.
RM- 4A	Cadenced gate results in production-equivalent build modes, trend metrics, updated finding mappings, expired-exception report, and signed residual-risk decisions for the release or review period.	GOV accepts, further reduces, avoids, transfers, or escalates each verified residual risk.

C.7. Finding and Evidence-Action Coverage

The mapping below prevents a roadmap phase from being declared complete while an in-scope register record has silently disappeared. A formal finding closes only after its stated affected scope is treated or shown not applicable and its current residual risk is reassessed. Closing a work package does not change the historical inherent rating. The package column distinguishes the minimum treatment needed to reassess the concrete finding from continuing class-prevention and sustainment work. A later architectural package does not hold a finding open unless it is selected and credited as that finding's treatment; its

own program commitment remains open independently. Evidence records are complete only after the bounded study is dispositioned; they are not assigned a tier merely because the investigation is scheduled, and an inconclusive study remains open and is re-planned.

Table 42 – Register-to-roadmap coverage

Register record	Treatment; continuing prevention	Record-level completion condition
CORE-SEC-001	Treatment: RM-1A, RM-1B. Prevention: RM-2A, RM-3A–RM-3C, RM-4A.	Affected populations are patched or contained and residual scope and risk are recorded. Credited structural tranches pass separately; unimplemented components are not claimed as covered.
CORE-SEC-002	Treatment: RM-1A, RM-1B. Prevention: RM-2A, RM-2C, RM-3A–RM-3C, RM-4A.	Affected populations are patched, contained, or shown to reject or bound file-controlled work; the residual decision states whether compatibility-policy work is credited or remains open.
CORE-SEC-003	Evidence: RM-1A, RM-1E. Conditional treatment: RM-1B. Prevention: RM-3A–RM-3C, RM-4A.	The supported source and build perimeter is bounded and each candidate has a verified guard, reachability, build-mode result, consequence, and disposition. Every supported adverse path is promoted and routed without waiting for global migration; an inconclusive site remains open, and the staged prevention program closes only its named tranches.
CORE-TOOL-001	Treatment: RM-1A, RM-1B. Prevention: RM-2A, RM-3C, RM-4A.	Affected tools are patched or isolated and reject the fixture without sanitizer findings; continuing tool gates remain tracked independently.
CORE-PRIV-001	Treatment: RM-1A, RM-1D; conditional RM-2G. Sustain: RM-4A.	The actual metadata population and recipients are bounded; minimization and recipient controls pass API and raw-byte inspection; when policy requires encryption, key recovery also passes; remaining disclosure is accepted or further treated.

Table 42 – Register-to-roadmap coverage, continued

Register record	Treatment; continuing prevention	Record-level completion condition
CORE-SAFE-001	Containment: RM-1A. Evidence and treatment: RM-2D, RM-3G.	Affected deployments have tested operational containment recorded; the fault-injection study establishes the failure and recovery distribution; the notification path and suspect state pass their acceptance tests; residual risk is reassessed. An inconclusive study remains open and re-planned.
CORE-SAFE-002	Containment: RM-1A. Evidence and treatment: RM-2D, RM-3G.	Containment recorded; the crash-consistency guarantee is published per layout and driver; the writer-kill matrix runs and its results are published per supported release; any acknowledged-versus-read-back divergence is tracked as a defect. Structural crash-proofing closes only its named, delivered mechanism.
LONG-SAF-001	Treatment: RM-3H. Prevention: RM-2C, RM-3E, RM-4A.	The defined corpus has complete dependency reports; archival-profile files open with all non-core plugins disabled; retained dependencies have reproducible specifications. Closure applies to the defined corpus only and never to “archived HDF5 data” generally.
APP-SEC-001	Treatment: RM-0A, RM-1A, RM-1C. Prevention: RM-2A, RM-2B, RM-4A.	The defined census boundary has a completeness attestation; every equivalent deployment found within it is contained, external-resource tests pass, and current residual risk is recorded. Closure of one instance does not close other or unscoped populations; the historical incident tier remains unchanged and later shared architecture stays open independently.
APP-SEC-002	Treatment: RM-1A, RM-1C. Sustain: RM-4A.	Affected loader versions are removed or contained and safe-mode negative tests pass in each deployed boundary.

Table 42 – Register-to-roadmap coverage, continued

Register record	Treatment; continuing prevention	Record-level completion condition
APP-SEC-003	Treatment: RM-1A, RM-1C; RM-0A when a Critical-equivalent service exposure is confirmed. Prevention: RM-2A, RM-2B, RM-4A.	Affected loaders reject or sandbox all supported external-resource forms and deployment evidence supports the residual decision.
APP-SEC-004	Treatment: RM-1A, RM-1C. Prevention: RM-2A, RM-2B, RM-4A.	Affected loaders enforce tested overflow-safe per-object and aggregate budgets before allocation; later reusable APIs remain independent program work.
APP-SEC-005	Treatment: RM-1A, RM-1C. Prevention: RM-3D, RM-4A.	Affected versions are fixed or contained; startup state and planted CWD, environment-path, and writable-path canaries prove that untrusted plugins do not load or execute.
FMT-SAF-001	Adjudication: RM-2C. Prevention: RM-3I, RM-4A.	Every identified dispute has a recorded ruling naming the authoritative artifact and the change made; the declared subset is machine-readable and its divergences are classified without a residual category; the adjudication procedure is adopted and its ledger is current. Closure applies to the declared subset only, never to the format as a whole.
IND-SAF-001	Treatment: RM-1A, RM-2C. Prevention: RM-3A, RM-4A.	Exact supported release/backport artifacts accept the sufficient nonminimal fixture and predictably reject the undersized invalid comparator.
IND-SAF-002	Treatment: RM-1A, RM-2C. Prevention: RM-3A, RM-4A.	Corrected writer output round-trips; the complete defined legacy population is inventoried; and repaired files preserve recorded checksums and provenance and pass supported readers, or have an accountable deterministic disposition.

Table 42 – Register-to-roadmap coverage, continued

Register record	Treatment; continuing prevention	Record-level completion condition
EXT-PRIV-001	Evidence: RM-2E. Conditional treatment: RM-3D, RM-3F.	The bounded study is completed; a negative disposition identifies its tested method, scope and population, an inconclusive result remains open, and a formal rating and treatment follow only if a concrete affected population and harm are supported.
SUP-SAFE-001	Evidence: RM-1A, RM-2F. Conditional treatment: RM-3E.	The bounded mismatch study is completed and routed or records a documented negative result over the tested profile population; an inconclusive result remains open and re-planned.

C.8. Change Control and Residual Risk

Each implementation record should contain the package identifier, named accountable person, responsible implementer, independent verifier, release or deployment approver, affected scope, linked findings and recommendations, linked finding-and-scope SLA child records, trigger event, trigger owner, actual trigger date, recommended maximum horizon, approved committed date or release, hard and coordination dependencies with their owner and state, effort estimate and assumptions, status, evidence links, target residual tier and status with its scope and assumptions, named risk- tolerance authority, exception identifier, reassessment trigger, and the last decision date. Status should use *Proposed*, *Accepted*, *In progress*, *Blocked*, *Implemented pending verification*, *Verified*, *Deferred under exception*, *Risk accepted*, or *Superseded*. A missed due-date event is recorded immutably as *Overdue*; a later active exception may add *Deferred under exception* for the exact scope but does not erase the miss. Blocked, deferred, accepted, or superseded work requires an explanatory decision record.

Any horizon, gate, control, compatibility, or residual-risk exception requires a named GOV approver, rationale, affected scope and unmet requirement, interim or compensating controls with evidence, monitoring owner and cadence, issue date,

expiration date, and automatic reopen or reassessment triggers. Exceptions do not revise inherent risk and cannot be indefinite. No exception permits a known, uncontained Critical-equivalent external-resource path to continue untrusted processing: the path is contained or the job is stopped; an exception may govern only later treatment or architecture.

After acceptance testing, the implementer supplies the artifacts, ASSURE verifies them, the package owner attests readiness, and the release or deployment approver authorizes rollout. The named GOV risk authority then reassesses current residual risk using Section 1.7, identifies the controls actually credited, and records a decision to accept, further reduce, avoid, or transfer the remaining risk. Target residual risk is not marked achieved before that review, and the verified current residual is recorded as the later result. Each reassessment is a dated, immutable snapshot with its scope, assumptions, verifier, and evidence; it does not overwrite the original inherent assessment.

The roadmap authority should review Phases 0–2 at least weekly while open, Phase 3 at each planned release gate, and Phase 4 at its stated cadence. New findings enter the response route first and are then mapped to the earliest work package capable of meeting that route; they are not assigned a phase from recommendation numbering or implementation convenience. Event-driven reassessment is also required after a control failure, new incident or reproducer, materially changed HDF5 or downstream release/default, changed trust boundary, privilege, automation, data sensitivity or dependency, fuzz or telemetry regression, and exception expiry. For recurring RM–4A work, each review produces a dated point-in-time verification snapshot rather than a permanent completion claim. A later gate failure, event trigger, or affected release reopens the scoped record and creates a new SLA child and reassessment snapshot while preserving the prior history.

D. CVE History Summary

Source, selection rule, and known limits of this table

Selection. The entries below were drawn from a public CVE-record search for “hdf5” against the MITRE CVE index, retrieved 21 May 2026 [72], supplemented by the HDF Group CVE test suite [40]. The population is therefore *every identifier that names HDF5 in that index at that retrieval date*, not a curated set of confirmed HDF5 defects. It consequently includes duplicate, rejected, unreproducible, and not-filed-against-HDF5 entries, which are marked as such. This is a records-based enumeration, not a prevalence estimate: it does not support statements about how often any class occurs across the installed ecosystem, and Section 1.10 applies in full.

Retrieval date versus evidence cutoff. Because the underlying search pre-dates the report’s 27 August 2026 evidence cutoff, identifiers published between those dates were added individually rather than by re-running the search. Five such records appear at the head of the table. Any HDF5-related identifier published in that window and not separately cited in this report is **outside** this table. Re-running the search at the cutoff date is an RM-1A census input.

The version-and-path gap. This table records affected *area* and root cause; with the exceptions noted in the final column, it does **not** establish affected versions, fixed-in versions, or fix commits. That is a material limit, because CORE-SEC-001 and CORE-SEC-002 define their assessed populations as deployments matching an affected version-and-path pair represented here. Until the final column is populated, **those scope statements cannot be evaluated by a reader and cannot be used to bound a deployment census**. Populating affected version, fixed-in version, fix commit, and reachability for every row is an explicit RM-1A deliverable and a precondition for starting the Near-term treatment clock on a concrete population. Cells reading *not established* are evidence gaps, not statements that a version is unaffected.

Scope tags. The final column also carries the scope tag proposed in APP-S7. The tag records which boundary the identifier belongs to; it assigns no priority, and each bounded scenario is still rated and routed under Section 1.7.

CWE classification. Each reported failure mode now carries the Common Weakness Enumeration identifiers for the weakness it manifests, drawn from the SSP SIG's top-CWE list for HDF5 contexts where the list covers the case and from CWE directly where it does not [53]. Three cautions apply.

First, the tags classify the *reported failure mode* — the observable outcome — not the root cause. Second, and more important for reading this table, **almost every row also manifests CWE-20, improper input validation**, and its HDF5-relevant children: CWE-1284 improper validation of a specified quantity, CWE-1285 of a specified index, position, or offset, CWE-1286 of syntactic correctness, and CWE-1287 of a specified type. That is the finding of this appendix rather than an annotation on it, so it is stated once here instead of repeated seventy times: the out-of-bounds writes, over-reads, and arithmetic faults tagged below are overwhelmingly *consequences* of a file-derived value being used before it was validated. The recurring class named in Section 3 is CWE-20; the tags in the table are what it turns into. Third, three rows are marked *not classified* — one whose root cause the source record leaves undetermined, one rejected duplicate, and one not filed against HDF5.

Known defects in the source records. Two artifacts of the source data are preserved rather than silently corrected. CVE-2025-6516 appears in two rows — metadata cache image and address-length decode — because the source records associate it with both paths; whether these are one defect or two is unresolved. Several rows aggregate identifiers that share a root-cause summary but may not share affected versions; those groupings must be split during RM-1A before any row is used to establish a deployment's applicability.

Table 43 – HDF5 CVE history summary

CVE(s)	Affected area	Reported failure mode	Root cause summary	Scope tag; version and fix evidence
CVE-2026-34734	h5dump datatype conversion (H5T__conv_struct)	heap use-after-free CWE-416	a crafted file drives the tool's type-conversion path into a use-after-free. Reported with AddressSanitizer-detected termination; the advisory states that practical code-execution exploitability is unknown. This is the tool finding CORE-TOOL-001, not evidence that every libhdf5 consumer reaches the path.	HDF5-official-tool Affected: 1.14.1-2 and earlier per the advisory [28, 85]. Fixed version and commit not established (RM-1A).
CVE-2026-2492	TensorFlow HDF5 plugin search path	local privilege escalation / arbitrary code loading CWE-427	the HDF5 integration searched an unsecured location for plugins, so a low-privileged local user could place a library that loads with the target user's authority. Rated at the application boundary as APP-SEC-005; the mechanism is HDF5 dynamic plugin loading, the defect is the search-path policy.	HDF5-plugin-loading Affected: TensorFlow 2.17.0; vendor fix cited [59, 92, 79]. Not an HDF5-release defect; no HDF5 version range applies.
CVE-2026-1669	Keras model loading via HDF5 external dataset references	arbitrary local file read / disclosure CWE-73	an embedded external dataset reference selected a local path during model load, disclosing a file readable by the loader process. Rated at the application boundary as APP-SEC-003; external raw storage is a documented HDF5 capability, not a parser defect.	HDF5-external-reference Affected: Keras >=3.0.0, <3.12.1 and >=3.13.0, <3.13.2 [70, 64]. No HDF5 version range applies.
CVE-2026-0897	Keras weight loading from model.weights.h5	memory exhaustion / interpreter crash CWE-400	a valid HDF5 member declared an extreme dataset shape and weight loading allocated from it without an adequate budget. Rated at the application boundary as APP-SEC-004; HDF5 metadata may legitimately declare a logical shape far larger than the useful payload.	HDF5-downstream-resource-policy Affected: Keras >=3.0.0, <=3.12.0 and >=3.13.0, <3.13.2 [69, 62]. No HDF5 version range applies.
CVE-2025-9905	Keras legacy .h5/.hdf5 model loading	arbitrary code execution CWE-502	legacy HDF5 model loading did not honor safe_mode=True, so a Lambda layer's serialized code executed in the loader process. Rated at the application boundary as APP-SEC-002; this is application deserialization layered on HDF5, not an HDF5 parser finding.	HDF5-downstream-deserialization Affected: Keras 3.0.0 through 3.11.2 [68, 63]. No HDF5 version range applies.
CVE-2025-7069, CVE-2025-6270, CVE-2025-2914	file-space info / free-space manager	heap buffer overflow CWE-787	file-derived file-space metadata, especially page size / section data, was accepted without sufficient sanity checks. Fix rejects invalid page size and corrupted free-space info early.	HDF5-core-parser Fix evidence: early page-size sanity check [35]. Affected and fixed versions not established (RM-1A).

Table 43 – HDF5 CVE history summary, continued

CVE(s)	Affected area	Reported failure mode	Root cause summary	Scope tag; version and fix evidence
CVE-2025-6858, CVE-2025-6817, CVE-2025-2913, CVE-2025-2912	object-header / metadata-cache cascade	null dereference, resource consumption, use-after-free, heap overflow CWE-787, CWE-416, CWE-476, CWE-400	corrupted object-header continuation metadata caused later cache/free-list/object-message failures. Root cause is bad continuation-message size/state not rejected at decode time.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2025-6856, CVE-2025-6816, CVE-2025-2923	object-header continuation messages	heap buffer overflow / use-after-free CWE-787, CWE-416	crafted object header with continuation message pointing back to itself led to under-sized internal buffer allocation. Fix compares expected and actual object-header chunk counts during deserialization.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2025-6750, CVE-2024-33874	mtime object-header messages	heap buffer overflow CWE-787	old/new modification-time messages were not properly decoded; an invalid message size could produce a zero-size buffer passed to an encoder. Fix decodes both mtime formats and rejects invalid size.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2025-6269, CVE-2025-6516	metadata cache image / address decode	heap buffer overflow CWE-787	cache-image reconstruction used insufficient bounds and input validation. Fix adds buffer-size tracking, overflow checks, input validation, and cleanup on failure.	HDF5-core-parser Fix evidence: buffer-size tracking, overflow and input validation, cleanup [34]. Versions not established (RM-1A).
CVE-2025-44905, CVE-2025-2308, CVE-2024-29159, CVE-2024-32615, CVE-2016-4331	scale-offset / n-bit filters	heap overflow / buffer overrun / arbitrary code execution CWE-787, CWE-502	filter decoders trusted encoded precision, byte counts, or scale-offset parameters from the file. Root cause is insufficient bounds checking in decompression/filter parameter handling.	HDF5-core-parser Versions and fix commit not established (RM-1A).

Table 43 – HDF5 CVE history summary, continued

CVE(s)	Affected area	Reported failure mode	Root cause summary	Scope tag; version and fix evidence
CVE-2025-44904, CVE-2024-32614, CVE-2024-32605, CVE-2024-29165, CVE-2019-9151, CVE-2018-14035, CVE-2018-13875, CVE-2018-11205	vectorized memory copy / scatter-fill paths	heap/stack overflow or over-read CWE-787, CWE-125	memory-vector copy routines copied between mismatched source/destination selections or trusted element counts too far. Root cause is insufficient validation of vector extents and copy sizes before memcpy-style operations.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2025-2926	object-header cache checksum serialization	null pointer dereference CWE-476	object-header checksum/serialization path could operate on incomplete or invalid object-header state. Root cause is missing null/state validation before serialization.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2025-2925	memory management wrapper	double free CWE-415	reallocation/error path mishandled ownership of mem. Root cause is ambiguous ownership/cleanup semantics around failed or edge-case realloc.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2025-2924, CVE-2024-32613, CVE-2024-32612	local heap free-list deserialization	heap buffer overflow CWE-787	local-heap free-block metadata from the file was trusted. Root cause is insufficient validation of free-block size/offset before rebuilding heap structures.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2025-2915, CVE-2018-13867	file accumulator	heap overflow / out-of-bounds read CWE-787, CWE-125	accumulator overlap/offset math accepted invalid ranges. Root cause is weak range validation for file accumulator reads/frees.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2025-2310, CVE-2019-9152	datatype / attribute string decoding	heap buffer overflow / out-of-bounds read CWE-787, CWE-125	string duplication from metadata used file-derived length or unterminated data. Root cause is unsafe string length handling during datatype/attribute decode.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2025-2309, CVE-2024-29163	datatype bit operations	heap buffer overflow CWE-787	bit-copy / bit-find logic trusted bit offsets, lengths, or precision metadata. Root cause is insufficient bounds checking in datatype bitfield operations.	HDF5-core-parser Versions and fix commit not established (RM-1A).

Table 43 – HDF5 CVE history summary, continued

CVE(s)	Affected area	Reported failure mode	Root cause summary	Scope tag; version and fix evidence
CVE-2025-2153	shared-message deletion	heap buffer overflow CWE-787	shared-message deletion path could process corrupted shared-message/object-header state. Root cause appears to be insufficient validation of shared-message metadata and cleanup state before deletion.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2025-26200, CVE-2024-33877, CVE-2017-17507	compound datatype conversion	heap overflow / out-of-bounds read/write CWE-787, CWE-125	compound datatype conversion trusted complex datatype layout details. Root cause is unsafe datatype conversion when member layout, unused bits, or conversion metadata are malformed.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2024-33876	point selection deserialization	heap overflow CWE-787	serialized point-selection metadata was trusted. Root cause is missing bounds/count validation when rebuilding point selections.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2024-33875, CVE-2019-8396	dataset layout encode	heap/buffer overflow CWE-787	dataset layout metadata could encode inconsistent dimensions/storage sizes. Fix family adds dataset layout size checks, multiplication overflow checks, and file-bound checks.	HDF5-core-parser Fix evidence: layout size, multiplication-overflow, and file-bound checks [36]. Versions not established (RM-1A).
CVE-2024-33873	dataset scatter memory	heap overflow CWE-787	scatter operation accepted inconsistent memory/file selection sizes. Root cause is insufficient validation of selected element counts and destination capacity.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2024-32624, CVE-2024-32610	reference datatype memory nulling	heap overflow CWE-787	reference-memory initialization/nulling used malformed datatype/reference metadata. Root cause is missing size/type validation before clearing reference memory.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2024-32623	generic array fill	heap overflow CWE-787	array fill operation trusted element count/size metadata. Root cause is missing multiplication/range validation before filling memory.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2024-32622, CVE-2024-29158, CVE-2024-29161 note	free-list array allocation	out-of-bounds read / stack overflow CWE-787, CWE-125	array allocation/free-list path accepted invalid counts or sizes. Root cause is weak validation of allocation quantity derived from file metadata.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2024-32621, CVE-2024-29162, CVE-2024-29157	global heap read	heap/stack overflow CWE-787	global heap object size/offset metadata could be inconsistent with actual buffer. Root cause is insufficient validation of heap object boundaries before read.	HDF5-core-parser Versions and fix commit not established (RM-1A).

Table 43 – HDF5 CVE history summary, continued

CVE(s)	Affected area	Reported failure mode	Root cause summary	Scope tag; version and fix evidence
CVE-2024-32620, CVE-2025-6516, CVE-2021-45830, CVE-2018-13866	address-length decode	heap/stack overflow / over-read CWE-787, CWE-125	encoded file addresses/lengths could exceed expected bounds. Root cause is insufficient validation of decoded address/length byte counts and destination capacity.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2024-32619, CVE-2018-14031	datatype copy/reopen	heap overflow / over-read CWE-787, CWE-125	datatype copy path accepted malformed nested datatype metadata. Root cause is missing validation while copying or reopening datatype structures.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2024-32618	native datatype conversion	heap overflow CWE-787	native-type construction trusted malformed datatype metadata. Root cause is inadequate validation during recursive datatype normalization.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2024-32617	H5MM_xstrdup	heap overflow CWE-787	unsafe string duplication from file-derived metadata. Root cause is missing bounded string handling.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2024-32616	datatype object-header encode helper	heap overflow CWE-787	datatype message encode helper accepted inconsistent datatype-message size/state. Root cause is missing encode-time size validation.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2024-32611	attribute release table	uninitialized value / DoS CWE-908	attribute-table bookkeeping conflated allocated capacity and actual attribute count. Fix separates current count and maximum capacity and cleans partial tables on error.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2024-32608, CVE-2024-32607	attribute close	memory corruption / segfault CWE-787	attribute close/release path could operate on partially initialized or corrupted attribute state. Root cause is unsafe cleanup after failed attribute-table construction or decode.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2024-32606	h5tools string printing	uninitialized dereference / DoS CWE-824	tool-side string formatting could dereference uninitialized values. No commit URL listed, so fix review was not possible.	HDF5-official-tool No commit URL in the source record; fix not reviewed. Versions not established (RM-1A).
CVE-2024-29166	link-info decode	buffer overrun CWE-787	object link-info metadata decode trusted malformed encoded size/flags. No commit URL listed.	HDF5-core-parser No commit URL in the source record; fix not reviewed. Versions not established (RM-1A).

Table 43 – HDF5 CVE history summary, continued

CVE(s)	Affected area	Reported failure mode	Root cause summary	Scope tag; version and fix evidence
CVE-2022-26061, CVE-2022-25972, CVE-2022-25942, CVE-2020-10809, CVE-2018-17436, CVE-2018-17433	gif2h5 tool / GIF parser	heap overflow, out-of-bounds write/read, invalid write CWE-787	legacy GIF conversion code parsed attacker-controlled GIF/HDF inputs unsafely. Commentary says the tool was removed rather than surgically hardened.	HDF5-legacy-tool Disposition: gif2h5 removed rather than hardened. Removing release not verified (RM-1A).
CVE-2021-46244	datatype copy completion	divide by zero / DoS CWE-369	datatype copy logic failed to guard arithmetic denominators or zero-sized datatype state.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2021-46243	datatype object-header decode	untrusted pointer dereference / DoS CWE-822	datatype decode accepted malformed pointer/metadata state. Root cause is missing validation before dereference.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2021-46242, CVE-2020-10810	metadata cache pin/unpin	use-after-free / null dereference CWE-416, CWE-476	cache entry lifecycle operations did not robustly handle malformed or unexpected cache state.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2021-45833	chunk file-map hyperslab	stack overflow / DoS CWE-787	chunk/hyperslab mapping trusted malformed dataspace/chunk metadata. Root cause is inadequate bounds checking in chunk mapping.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2021-45832	error stack internals	stack overflow / DoS CWE-787	listed as cannot reproduce; no fix commit. Treat root cause as unconfirmed.	HDF5-core-parser Disposition: recorded as cannot reproduce; no fix commit. Root cause unconfirmed.
CVE-2021-45829	unspecified HDF5 path	segfault / DoS <i>not classified</i>	Root cause undetermined.	HDF5-core-parser Disposition: root cause undetermined; no fix commit.
CVE-2021-37501	h5dump string sprint	buffer overflow / DoS CWE-787	h5dump formatting path wrote beyond buffer while rendering file-derived data.	HDF5-official-tool Versions and fix commit not established (RM-1A).
CVE-2021-36977	mat file I/O library using HDF5	heap overflow CWE-787	not filed against HDF5. Root cause belongs to downstream integration/context, not the HDF5 fix set.	not-HDF5 Not filed against HDF5; downstream integration of HDF5 1.12.0 as reported. No HDF5 fix set applies.
CVE-2021-31009	Apple platform issue	multiple HDF5 issues <i>not classified</i>	not filed against HDF5; fixed by Apple by removing HDF5 from affected components.	not-HDF5 Not filed against HDF5; vendor addressed it by removing HDF5 from the affected components.

Table 43 – HDF5 CVE history summary, continued

CVE(s)	Affected area	Reported failure mode	Root cause summary	Scope tag; version and fix evidence
CVE-2020-18494, CVE-2020-18232	dataspace close	buffer overflow / possible RCE CWE-787	malformed file could corrupt dataspace cleanup/close state. Root cause is unsafe lifecycle handling of malformed dataspace metadata.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2020-10812	file reference count query	null pointer dereference / DoS CWE-476	file query path failed to validate file/reference state before dereference.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2020-10811, CVE-2018-14033	layout decode	heap over-read CWE-125	dataset layout message decoder trusted encoded layout fields. Root cause is missing bounds checks around decoded layout data.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2019-8398, CVE-2019-8397	datatype size/close	out-of-bounds read CWE-125	datatype object state could be malformed before size/close operations. No commit URL listed.	HDF5-core-parser No commit URL in the source record; fix not reviewed. Versions not established (RM-1A).
CVE-2018-17439	dataspace extent dimensions during HDF to GIF conversion	stack overflow CWE-787	conversion tool path trusted dataspace dimensions from file.	HDF5-legacy-tool Disposition: HDF-to-GIF conversion path removed. Removing release not verified (RM-1A).
CVE-2018-17438	selected I/O	divide by zero / DoS CWE-369	selected-I/O path failed to guard divisor/count metadata from crafted file. Tool-removal commit, so the listed fix is coarse.	HDF5-core-parser Disposition: tool-removal commit, so the listed fix is coarse and does not establish parser-path closure.
CVE-2018-17437	datatype decode helper	memory leak / DoS CWE-401	error path leaked allocations when datatype metadata was malformed.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2018-17435	attribute decode	heap over-read CWE-125	attribute message decoder trusted encoded sizes/offsets.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2018-17434	h5repack filter application	divide by zero / DoS CWE-369	tool-side filter setup failed to guard zero-valued filter/chunk parameters.	HDF5-official-tool Versions and fix commit not established (RM-1A).
CVE-2018-17432	simple dataspace encode	null pointer dereference / DoS CWE-476	dataspace encode path accepted invalid dataspace state before dereference.	HDF5-core-parser Versions and fix commit not established (RM-1A).

Table 43 – HDF5 CVE history summary, continued

CVE(s)	Affected area	Reported failure mode	Root cause summary	Scope tag; version and fix evidence
CVE-2018-17237, CVE-2018-17233, CVE-2018-11207, CVE-2018-11203, CVE-2017-17508	chunking / b-tree / datatype arithmetic	divide by zero / SIGFPE / DoS CWE-369	crafted file could set chunk, b-tree, datatype, or layout parameters to zero. Root cause is missing arithmetic precondition checks.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2018-17234, CVE-2018-13873, CVE-2018-11204	object-header chunk deserialize	memory leak / over-read / null dereference CWE-125, CWE-476, CWE-401	object-header chunk deserialization accepted malformed chunk metadata and mishandled error cleanup.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2018-16438	external links	out-of-bounds read CWE-125	external-link query trusted malformed external-link metadata.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2018-15672, CVE-2018-14032	duplicate/rejected IDs	not applicable <i>not classified</i>	rejected duplicate CVEs; no HDF5 root cause to summarize beyond the referenced canonical CVEs.	duplicate/rejected Rejected duplicate identifiers; no HDF5 root cause or fix set beyond the canonical CVEs.
CVE-2018-15671	property-list callback lookup	excessive stack consumption / DoS CWE-674	recursive or malformed property-list callback state allowed stack exhaustion.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2018-14460	simple dataspace decode	heap over-read CWE-125	dataspace message decoder trusted encoded extent/rank metadata.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2018-14034	filter pipeline reset	out-of-bounds read CWE-125	malformed filter-pipeline metadata caused unsafe reset/read behavior. No commit URL listed.	HDF5-core-parser No commit URL in the source record; fix not reviewed. Versions not established (RM-1A).
CVE-2018-13876, CVE-2018-13874	sec2 VFD read path	stack overflow CWE-787	low-level file-driver read/memset path could be driven out of bounds by malformed file metadata. No commit URL listed.	HDF5-core-parser No commit URL in the source record; fix not reviewed. Versions not established (RM-1A).
CVE-2018-13872	group entry decode	heap overflow CWE-787	group-entry decoder trusted malformed symbol-table/group metadata. No commit URL listed.	HDF5-core-parser No commit URL in the source record; fix not reviewed. Versions not established (RM-1A).
CVE-2018-13871	free-list block allocation	heap overflow CWE-787	free-list block allocator accepted invalid allocation size/count.	HDF5-core-parser Versions and fix commit not established (RM-1A).

Table 43 – HDF5 CVE history summary, continued

CVE(s)	Affected area	Reported failure mode	Root cause summary	Scope tag; version and fix evidence
CVE-2018-13870, CVE-2018-13869	link decode	heap over-read / overlapping memcpy CWE-125	link-message decoder trusted malformed link metadata, including overlapping source/destination regions.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2018-13868	old fill-value decode	heap over-read CWE-125	old fill-value decoder trusted encoded fill-value size. No commit URL listed.	HDF5-core-parser No commit URL in the source record; fix not reviewed. Versions not established (RM-1A).
CVE-2018-11206	fill-value decode	out-of-bounds read / possible disclosure CWE-125, CWE-73	fill-value message decoders trusted encoded size/type fields.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2018-11202	hyperslab spans	null pointer dereference / DoS CWE-476	hyperslab span construction failed to validate malformed selection state.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2017-17509	group cache entry decode vector	out-of-bounds write CWE-787	group-entry vector decoder trusted file-derived counts/array bounds. No commit URL listed.	HDF5-core-parser No commit URL in the source record; fix not reviewed. Versions not established (RM-1A).
CVE-2017-17506, CVE-2017-17505	filter pipeline decode	out-of-bounds read / null dereference CWE-125, CWE-476	pipeline decoder failed to validate malformed filter counts or fields. No commit URL listed.	HDF5-core-parser No commit URL in the source record; fix not reviewed. Versions not established (RM-1A).
CVE-2016-4333	array allocation / initialization loop	out-of-bounds write CWE-787	file-controlled value could modify loop termination while initializing an allocated array.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2016-4332	object-header message flags	heap out-of-bounds write / code execution CWE-787	library failed to check whether a message type supported a flag before casting to an incompatible structure.	HDF5-core-parser Versions and fix commit not established (RM-1A).
CVE-2016-4330	dataspace array rank/dimensions	heap overflow / code execution CWE-787	file-supplied array dimensionality was not checked against allocated space.	HDF5-core-parser Affected: 1.8.16 as reported by the primary analysis [26]. Fixed version and commit not established (RM-1A).

E. Software Supply Chain Details

The `HDF5_ALLOW_EXTERNAL_SUPPORT` modes apply to filter-related dependencies. Some test and example configurations have their own source-fetch paths, such as KWSys for the API test driver and h5py for Python examples. MPI, HDFS, ROS3, OpenSSL, Java, documentation, packaging, and developer tools must be provided by the build environment.

Table 44 – Dependency source modes

Mode	Relevant options	Meaning
System packages	<code>HDF5_ALLOW_EXTERNAL_SUPPORT=NO</code>	CMake uses <code>find_package()</code> , <code>find_library()</code> , or <code>find_program()</code> to locate installed dependencies. This is the default.
Git fetch	<code>HDF5_ALLOW_EXTERNAL_SUPPORT=GIT</code> plus a dependency-specific <code>*_USE_EXTERNAL=ON</code> option	CMake downloads source from the configured <code>*_GIT_URL</code> and <code>*_GIT_TAG</code> values and builds it with HDF5.
Tarball fetch	<code>HDF5_ALLOW_EXTERNAL_SUPPORT=TGZ</code> plus a dependency-specific <code>*_USE_EXTERNAL=ON</code> option	CMake downloads from <code>*_TGZ_ORIGPATH</code> and <code>*_TGZ_NAME</code> , or uses <code>TGZPATH</code> when the dependency-specific <code>*_USE_LOCALCONTENT=ON</code> option is set.

Table 45 – Baseline build

Dependency	Trigger	Notes
CMake 3.26 or newer	Always	Required by the top-level <code>cmake_minimum_required()</code> .
C compiler and native build tool	Always	The top-level project is a C project.
Platform C library, headers, and system calls	Always	CMake probes standard headers, types, functions, byte order, large-file support, file locking, <code>pread/pwrite</code> , and related platform features.
Platform link libraries	Platform-dependent	CMake adds libraries only when probes show they are needed: <code>m</code> , <code>dl</code> , <code>rt</code> , <code>posix4</code> , <code>ucb</code> , <code>ws2_32</code> , <code>wsock32</code> , and <code>shlwapi</code> .

PkgConfig	Optional probe	Used only when available. It can help some find modules, such as zlib-ng and libaec, but it is not a hard requirement for the default build.
Threads	Found in every configure; CMake searches for C11 threads, Win32 threads, or pthreads. When found, HDF5 records thread support and links Threads::Threads; HDF5_ENABLE_THREADSAFE, HDF5_ENABLE_CONCURRENCY, and HDF5_ENABLE_SUBFILING_VFD require it.	

Table 46 – HDF5 feature triggers

Feature or component	Build options	Dependency	Notes
Parallel HDF5	HDF5_ENABLE_PARALLEL=ON	MPI C with MPI-IO, MPI standard 3.0 or newer	Adds MPI::MPI_C to public link libraries. Parallel filtered writes and large parallel I/O are enabled only when the required MPI symbols are present. Parallel tests also need an MPI launcher such as mpiexec.
Parallel Fortran	HDF5_ENABLE_PARALLEL=ON, HDF5_BUILD_FORTRAN=ON	MPI Fortran	The Fortran build adds MPI Fortran libraries. The mpi_f08 module is used when available.
Thread-safe library	HDF5_ENABLE_THREADSAFE=ON	Threads	Unsupported with parallel, Fortran, C++, or high-level library unless HDF5_ALLOW_UNSUPPORTED=ON. On Windows, static libraries are rejected.

Multi-threaded concurrency	HDF5_ENABLE_CONCURRENCY=ON	Threads	Experimental. Mutually exclusive with HDF5_ENABLE_THREADS; has the same unsupported combinations as thread-safety unless overridden with HDF5_ALLOW_UNSAFE=ON.
C++ wrappers	HDF5_BUILD_CPP_LIB=ON	C++ compiler	Unsupported with parallel unless HDF5_ALLOW_UNSAFE=ON. High-level C++ wrappers also require HDF5_BUILD_HL_LIB=ON.
Fortran wrappers	HDF5_BUILD_FORTRAN=ON	Fortran compiler with Fortran 2003 ISO_C_BINDING support	Unsupported with thread-safety or concurrency unless HDF5_ALLOW_UNSAFE=ON. High-level Fortran wrappers also require HDF5_BUILD_HL_LIB=ON.
Java wrappers	HDF5_BUILD_JAVA=ON	Java/JDK, shared HDF5 libraries, Java dependency JARs	Java requires shared HDF5 libraries. Java 11 or newer is required for the JNI implementation. Java tests additionally require the JUnit and Hamcrest JARs.
Java JNI implementation	HDF5_BUILD_JAVA=ON, HDF5_ENABLE_JNI=ON, or any Java version earlier than 25	JNI headers and libraries	JNI is the default Java implementation. HDFS also requires JNI.
Java FFM implementation	HDF5_BUILD_JAVA=ON, Java 25 or newer, HDF5_ENABLE_JNI=OFF	jextract from JEXTRACT_HOME/bin or JAVA_HOME/bin	Generates FFM bindings at configure time.

Java logging/test JARs	HDF5_BUILD_JAVA=ON	slf4j-api, slf4j-nop, slf4j-simple; plus JUnit 4 and Hamcrest for tests	The default paths point to bundled JARs under java/lib. They can be overridden with HDF5_JAVA*_JAR cache variables. HDF5_ENABLE_MAVEN_DEPLOY=ON generates Maven-style artifacts and POM files; the CMake build does not locate or run mvn.
Documentation	HDF5_BUILD_DOC=ON	Doxygen	HDF5_ENABLE_DOXY_WARNINGS=ON makes Doxygen warnings fail the build.
Header regeneration	HDF5_GENERATE_HEADERS=ON	Perl	Perl is optional otherwise. If it is missing, generated headers are not regenerated.
Python examples	H5EXAMPLE_BUILD_PYTHON=ON	Python3 interpreter, Python development files, NumPy, h5py source, pip	The examples CMake project fetches h5py from Git and installs it with python3 -m pip.

Table 47 – HDF5 filter dependency choices

Feature	Build options	Dependency	Source choices
DEFLATE filter	HDF5_ENABLE_ZLIB_SUPPORT=ON, HDF5_USE_ZLIB_NG=OFF	zlib	System package by default; ZLIB_USE_EXTERNAL=ON can build zlib from Git or TGZ. HDF5_USE_ZLIB_STATIC=ON prefers a static library.

DEFLATE through zlib-ng	filter	HDF5_ENABLE_ZLIB_SUPPORT=ON, HDF5_USE_ZLIB_NG=ON	zlib-ng	System package by default; ZLIB_USE_EXTERNAL=ON can build zlib-ng from Git or TGZ. zlib compatibility mode is detected.
SZIP filter		HDF5_ENABLE_SZIP_SUPPORT=ON	libaec with libsz compatibility, or another package selected by LIBAEC_PACKAGE_NAME	System package by default; SZIP_USE_EXTERNAL=ON can build libaec from Git or TGZ. HDF5_USE_LIBAEC_STATIC=ON prefers static libraries.
HDF5 filter project	filter plugins	HDF5_ENABLE_PLUGIN_SUPPORT=ON	Installed h5pl/HDF5 filter plugins package, or the hdf5_plugins project	PLUGIN_USE_EXTERNAL=ON can build the plugin project from Git or TGZ. Plugin support is disabled for the 1.6 API.

Table 48 – HDF5 VFD and VOL connector dependency choices

Feature	Build options	Dependency	Notes
Direct VFD	HDF5_ENABLE_DIRECT_VFD=ON	POSIX O_DIRECT and posix_memalign()	No third-party library. Configuration fails if the platform support is missing.
Mirror VFD	HDF5_ENABLE_MIRROR_VFD=ON	POSIX socket/fork headers and functions	No third-party library. Requires netinet/in.h, netdb.h, arpa/inet.h, sys/socket.h, and fork().

ROS3 VFD	HDF5_ENABLE_ROS3_VFD=ON	aws-c-s3 package	CMake	HDF5 does not fetch this library. Building aws-c-s3 from source can also require AWS C libraries such as aws-c-common, aws-checksums, aws-c-cal, aws-c-io, aws-c-compression, aws-c-http, aws-c-sdkutils, aws-c-auth, and, on Linux, aws-lc and s2n-tls. ROS3 docker-proxy tests additionally require docker and the AWS CLI.
HDFS VFD	HDF5_ENABLE_HDFS=ON	JNI/JVM and Hadoop libhdfs		HADOOP_HOME is used to locate hdfs.h, libhdfs, and the Hadoop version command. Non-MSVC builds also add -pthread.
Subfilng VFD and IOC VFD	HDF5_ENABLE_PARALLEL=ON, HDF5_ENABLE_SUBFILING_VFD=ON	MPI C with MPI_Comm_split_type, plus Threads		Not available on Windows. IOC VFD is built when subfilng is enabled.
External VOL connectors	HDF5_VOL_ALLOW_EXTERNAL=GIT LOCAL_DIR HDF5_VOL_URLnn HDF5_VOL_PATHnn	Connector-defined or with or		Up to ten CMake-based VOL connectors can be added. HDF5 patches connector CMake code to use the in-tree HDF5 build, but each connector may bring its own dependencies.

Table 49 – Tools, Tests, and Security Features

Feature	Build options	Dependency	Notes
Signed plugin verification	HDF5_REQUIRE_SIGNED_PLUGINS=ON	OpenSSL libcrypto 1.1.0 or newer	Adds OpenSSL: : Crypto to HDF5. HDF5_LOCK_PLUGIN_KEYSTORE=ON also requires HDF5_PLUGIN_KEYSTORE_DIR.
h5sign tool	HDF5_REQUIRE_SIGNED_PLUGINS=ON, HDF5_BUILD_TOOLS=ON	OpenSSL libcrypto	Used to sign HDF5 plugins.
Signed-plugin tests	HDF5_REQUIRE_SIGNED_PLUGINS=ON, BUILD_TESTING=ON	openssl executable	Tests generate RSA keys with the OpenSSL command-line tool.
Parallel tools utilities	HDF5_ENABLE_PARALLEL=ON, HDF5_BUILD_PARALLEL_TOOLS=ON	MFU, CIRCLE, DTCMP	CMake looks for headers and libraries, using MFU_ROOT as a hint.
API test driver	BUILD_TESTING=ON, HDF5_TEST_API_ENABLE_DRIVER=ON	KWSys	The test driver fetches KWSys from a tarball or from TGZPATH when KWSYS_USE_LOCALCONTENT=ON.
Shell-script tests	BUILD_TESTING=ON	PowerShell or Bash	CMake uses pwsh/powershell when available; otherwise Unix builds look for bash.
ROS3 VFD tests	HDF5_ENABLE_ROS3_VFD=ON, HDF5_ENABLE_ROS3_VFD_DOCKER_PROXY=ON	Docker daemon and AWS CLI	Used to run S3 proxy tests.

Table 50 – Developer and Packaging Dependencies

Configuration	Build options	Dependency
Analyzer tooling	HDF5_ENABLE_ANALYZER_TOOLS=ON	clang-tidy, include-what-you-use, and cppcheck are probed and used by the analyzer macros.
Source formatting targets	HDF5_ENABLE_FORMATTERS=ON	clang-format and cmake-format.
Code coverage with Clang/IntelLLVM	HDF5_ENABLE_COVERAGE=ON, CODE_COVERAGE=ON	llvm-cov and llvm-profdata; AppleClang uses xcrun llvm-cov and xcrun llvm-profdata.
Code coverage with GCC	HDF5_ENABLE_COVERAGE=ON, CODE_COVERAGE=ON	lcov and genhtml; fallback instrumentation can also link gcov.
Sanitizers	HDF5_ENABLE_SANITIZERS=ON, HDF5_USE_SANITIZER=<kind>	Compiler sanitizer runtime support. Supported values depend on compiler and platform.
AFL support module	Explicit inclusion of config/sanitizer/afl-fuzzing.cmake	afl-cc and afl-c++. This module is available but is not included by the top-level HDF5 build by default.
Dependency graph support module	Explicit inclusion of config/sanitizer/dependency-graph.cmake	Graphviz dot. This module is available but is not included by the top-level HDF5 build by default.
Windows installers	CPack on Windows	NSIS and WiX are probed when available.
Linux packages	CPack on Unix-like systems	dpkg-shlibdeps enables DEB generation; rpmbuild enables RPM generation for non-parallel builds.

Table 51 – External Source Defaults

Dependency	Default version/tag	Default source
------------	---------------------	----------------

zlib	1.3.2 / v1.3.2	https://github.com/madler/zlib.git or https://github.com/madler/zlib/releases/download/v1.3.2/zlib-1.3.2.tar.gz
zlib-ng	2.3.3	https://github.com/zlib-ng/zlib-ng.git or https://github.com/zlib-ng/zlib-ng/archive/refs/tags/2.3.3.tar.gz
libaec	1.1.6 / v1.1.6	https://github.com/MathisRosenhauer/libaec.git or https://github.com/MathisRosenhauer/libaec/releases/download/v1.1.6/libaec-1.1.6.tar.gz
HDF5 filter plugins	master	https://github.com/HDFGroup/hdf5_plugins.git or https://github.com/HDFGroup/hdf5_plugins/releases/download/snapshot/hdf5_plugins-master.tar.gz
KWSys API-test helper	master tarball	https://gitlab.kitware.com/utils/kwsys/-/archive/master/kwsys-master.tar.gz
h5py Python examples	master	https://github.com/h5py/h5py.git
bitshuffle	master or 0.5.2 tarball	https://github.com/kiyo-masui/bitshuffle.git or https://github.com/kiyo-masui/bitshuffle/archive/refs/tags/bitshuffle-0.5.2.tar.gz
Blosc	1.21.6	https://github.com/Blosc/c-blosc.git or https://github.com/Blosc/c-blosc/archive/refs/tags/c-blosc-1.21.6.tar.gz
Blosc zlib helper	develop or 1.3.1 tarball	https://github.com/madler/zlib.git or https://github.com/madler/zlib/releases/download/v1.3.1/zlib-1.3.1.tar.gz
Blosc2	2.17.1	https://github.com/Blosc/c-blosc2.git or https://github.com/Blosc/c-blosc2/archive/refs/tags/c-blosc2-2.17.1.tar.gz

Blosc2 zlib helper	develop or 1.3.1 tarball	https://github.com/madler/zlib.git or https://github.com/madler/zlib/releases/download/v1.3.1/zlib-1.3.1.tar.gz
bzip2	bzip2-1.0.8	https://github.com/libarchive/bzip2.git or https://github.com/libarchive/bzip2/archive/refs/tags/bzip2-bzip2-1.0.8.tar.gz
fpzip	develop or 1.3.0 tarball	https://github.com/LLNL/fpzip.git or https://github.com/LLNL/fpzip/releases/download/1.3.0/fpzip-1.3.0.tar.gz
JPEG	jpeg-9e	https://github.com/libjpeg-turbo/libjpeg-turbo.git or https://www.ijg.org/files/jpegsrc.v9e.tar.gz
LZ4	dev or 1.10.0 tarball	https://github.com/lz4/lz4.git or https://github.com/lz4/lz4/releases/download/v1.10.0/lz4-1.10.0.tar.gz
LZF	3.6 tarball	http://dist.schmorp.de/liblzf/liblzf-3.6.tar.gz
SZ	master or 2.1.12.5 tarball	https://github.com/szcompressor/SZ.git or https://github.com/szcompressor/SZ/releases/download/v2.1.12.5/SZ-2.1.12.5.tar.gz
ZFP	develop or 1.0.1 tarball	https://github.com/LLNL/zfp.git or https://github.com/LLNL/zfp/releases/download/1.0.0/zfp-1.0.1.tar.gz
Zstandard	1.5.7	https://github.com/facebook/zstd.git or https://github.com/facebook/zstd/releases/download/v1.5.7/zstd-1.5.7.tar.gz

F. Resources and Supporting Tooling

The artifacts below are referenced by recommendations in this report as existing starting points. Listing a project here identifies a resource; it is not an assessment of that project, and none was audited.

HDF5 Safety, Security & Privacy (SSP) Special Interest Group [46].

Governance, process, and artifact home for the work this report feeds. It is the natural venue for the roadmap adoption decision described in Appendix C, and for publishing the CORE-S12 metrics dashboard.

HDF5 CVE test suite [40].

An existing collection of vulnerability-triggering inputs. It is the obvious seed for the permanent corpus proposed in CORE-S6, which additionally requires per-case expected-outcome metadata and execution in both assertion-enabled and -DNDEBUG builds. It also contributed to the CVE enumeration in Appendix D.

hdf5-pickles: a machine-readable specification of the HDF5 file format [30].

Directly relevant to three recommendations. It is a candidate source of truth for the invariant registry in CORE-S1; a generator for the positive and negative fixtures required by IND-S1; and the origin of issue #87, the assert-masking observation tracked as CORE-SEC-003.

HDF5 extension skeletons [41].

Templates for VOL connectors, filters, and VFDs. A natural place to embed the privacy-documentation template required by FILTER-S5 and the trust and search-path defaults required by VOL-S3, FILTER-S1, and VFD-S4, so that new extensions inherit them rather than retrofitting them.

HDF5 plugins [43].

The filter-plugin project whose options govern the dependency set discussed in Section 7 and Appendix E. It is the scope boundary for the plugin manifest and allowlist proposed in CORE-S10.